6. Amazon CloudWatch, AWS X-Ray, and Optimization

Introduction to Observability

Monitoring is a verb.

It's something we do.

We collect metrics.

We collect logs.

We monitor network traffic.

When something seems off,
automatic alerts go out to someone or something to try to remediate.

Monitoring is the act of collecting data.

What types of data we collect, what we do with the data,
and if that data is readily analyzed or available is a different story.

Observability is not a verb.

It's not something we do.

Instead, observability is more of a property of a system.

A system can be observable or it can be unobservable.

An observable system allows you to measure
how well the internal workings of a system are, from the outputs of that system.

A system with observability is what you want.

If you cannot observe the outputs of your application
and know whether things are working well or not,
how can you possibly know whether the changes
you are making to your system are working for your end users or not?

One common way to build observability into a system
is to use distributed tracing software.

Tracing software allows you to understand
the flow of traffic through your services.

With API-driven applications,
services call each other to perform tasks.

Tracing allows to collect data on the calls between components
and measure the performance of each individual service
and view the system as a whole regarding latency between these services.

Introduction to X-Ray

X-Ray analyzes and debugs distributed applications,
such as those built using a microservices architecture,
to understand how your application and its underlying services are performing.

X-Ray provides an end-to-end view of requests
as they travel through your application.

And shows a map of your application's underlying components.

Before using X-Ray, there are several concepts that should be covered so that you can have a solid starting foundation.

Segments are the compute resources running your application logic.

These provide the resource's name, details about the request,
and details about the work done.

Segments can break down the data about the work done into subsegments.

Subsegments provide more granular details,
such as timing information and downstream calls, made by an application.

The data that an application sends to X-Ray is used to generate a service graph.

A service graph is a JSON document that contains information about the services and resources that make up the application.

A trace is used to track the path of a request through the application.

It collects all the segments generated by a single request.

X-Ray analyzes an application by
collecting and recording the traces,
and provides you with a service map to see the trace data.

The default sampling rate is fairly conservative,
only recording the first request each second, and 5% of any additional requests.

You have the ability to configure X-Ray to modify the default sampling rule.

The AWS X-Ray daemon is a software application that listens for traffic on UDP port 2000, gathers raw segment data, and relays it to the AWS X-Ray API.

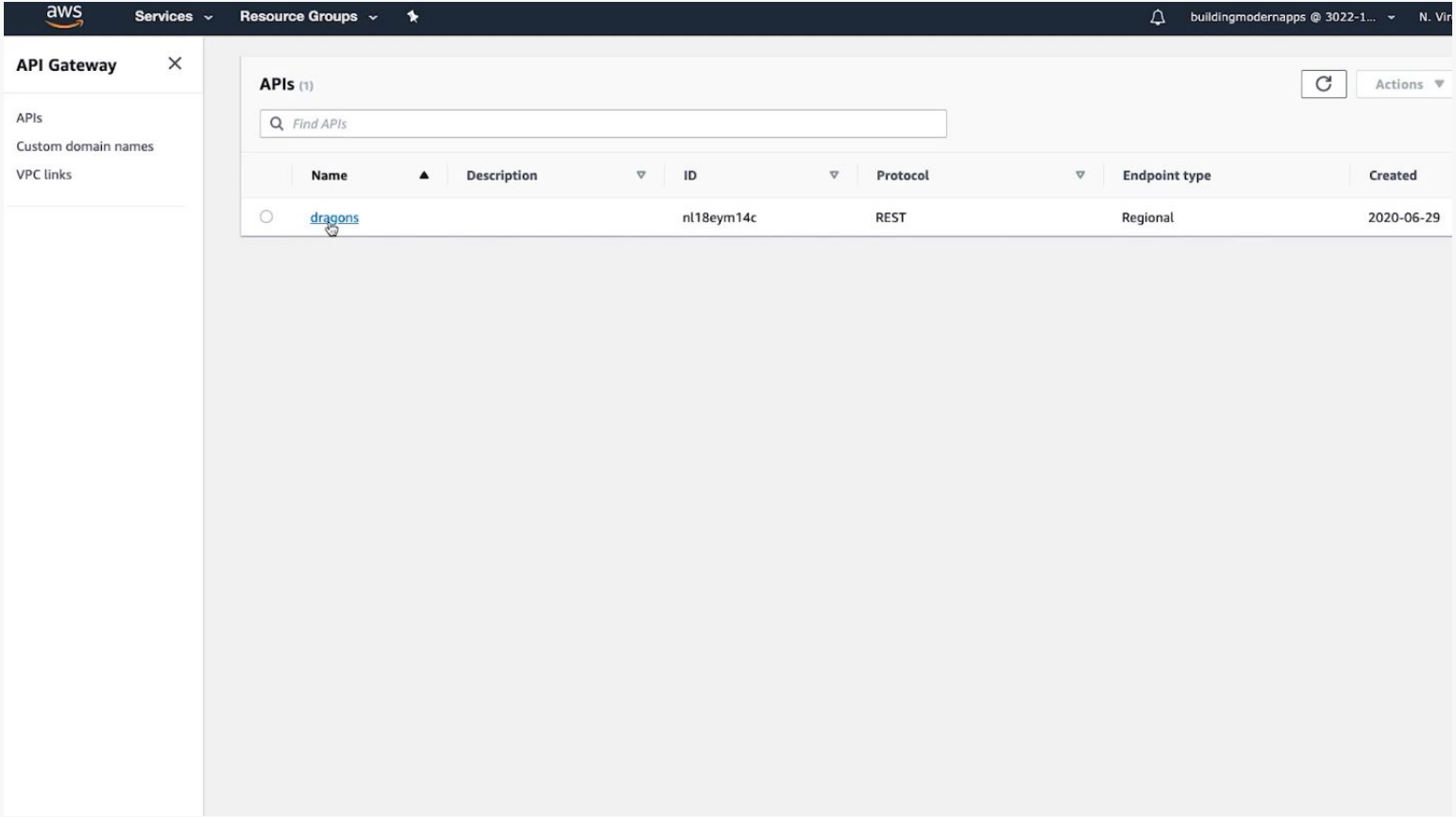
Within Amazon API Gateway, two types of tracing can be done: passive and active.

Passive tracing is the default setting.

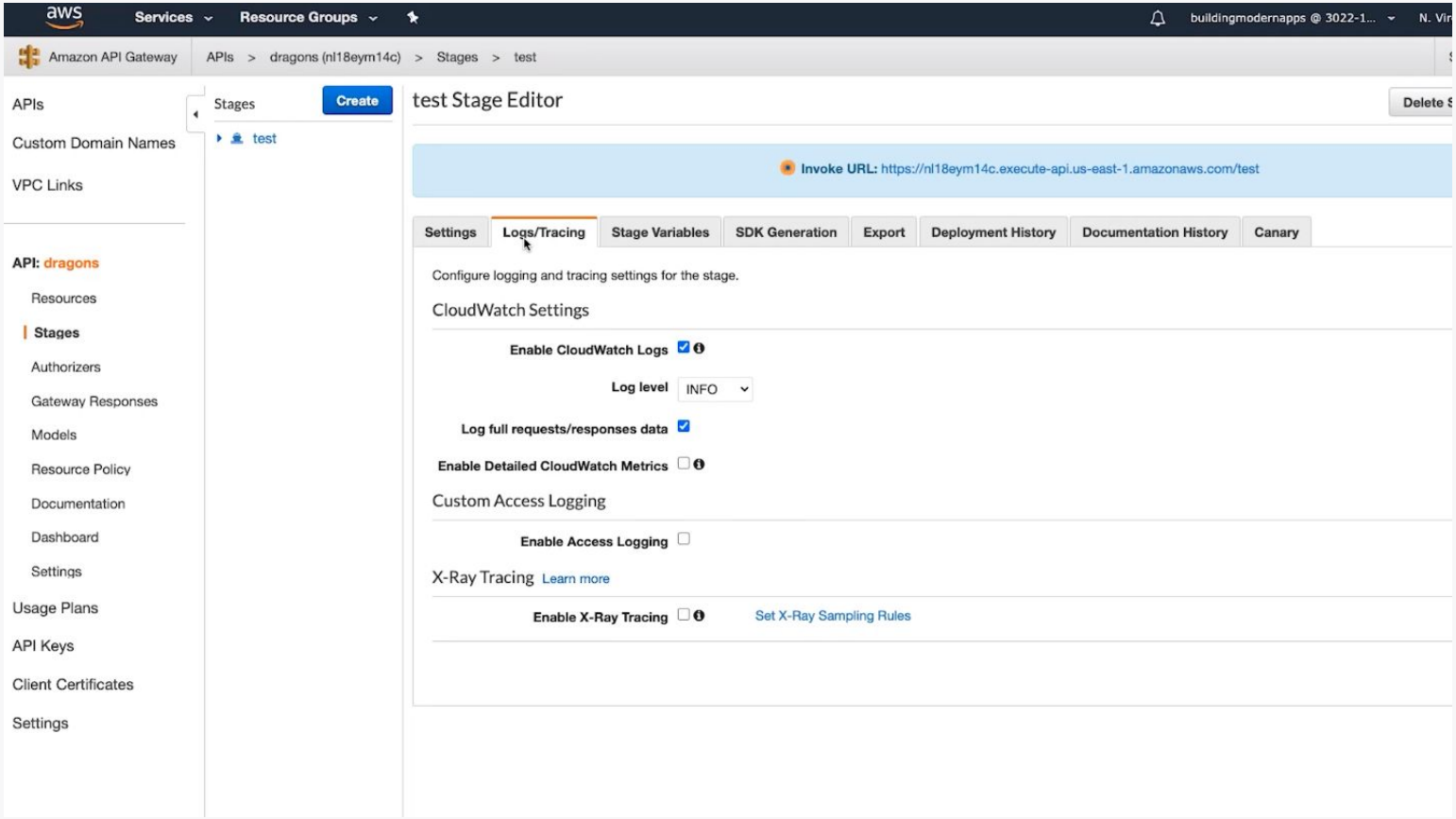
This means that the API Gateway is only traced
if X-Ray has been enabled on an upstream service.

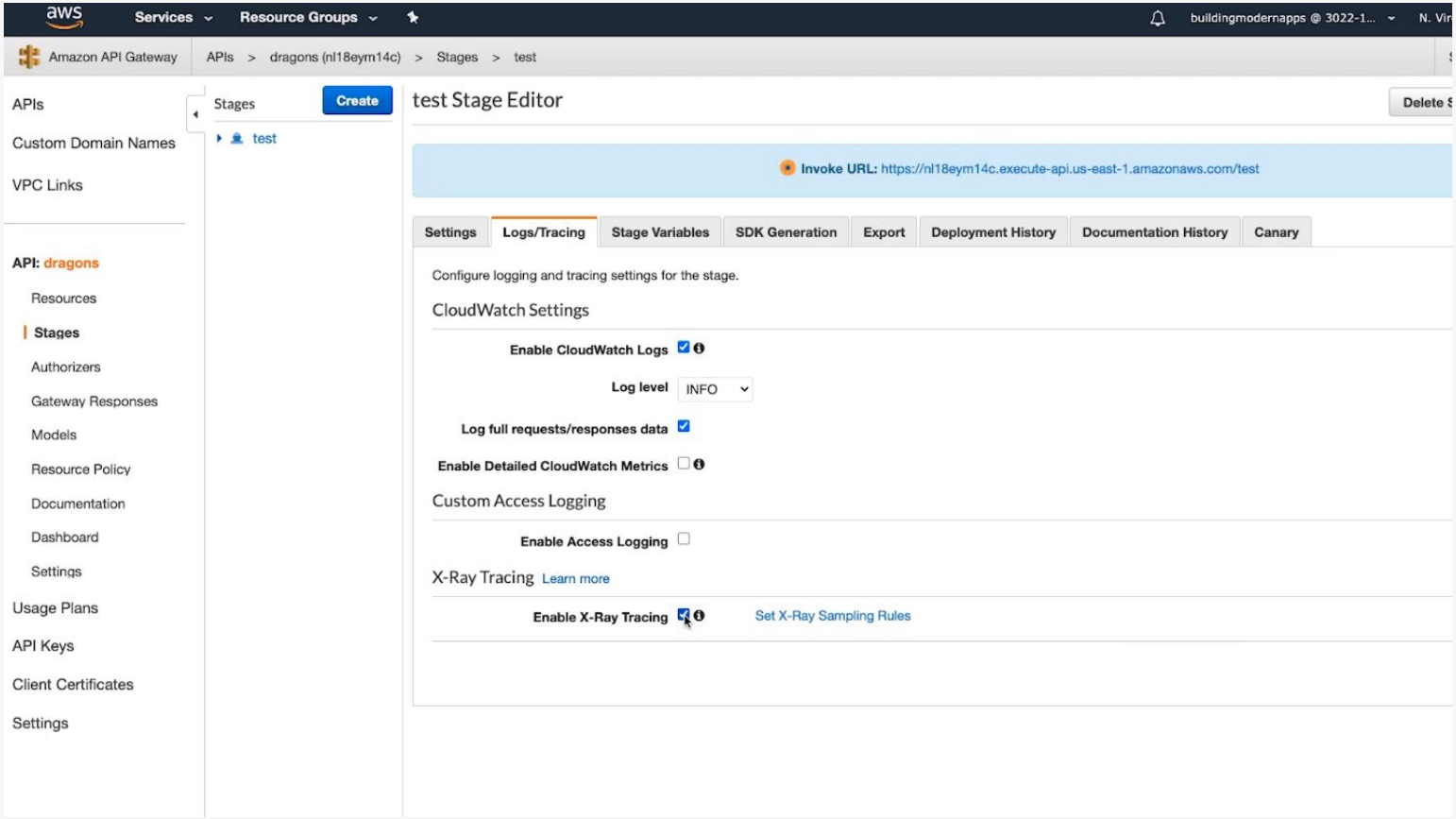
If passive tracing is used, API Gateway will not be sampling requests automatically,
but will only be captured by upstream sampling.

With active tracing,
API Gateway will automatically sample API invocation requests
based on the sampling algorithms set in X-Ray.

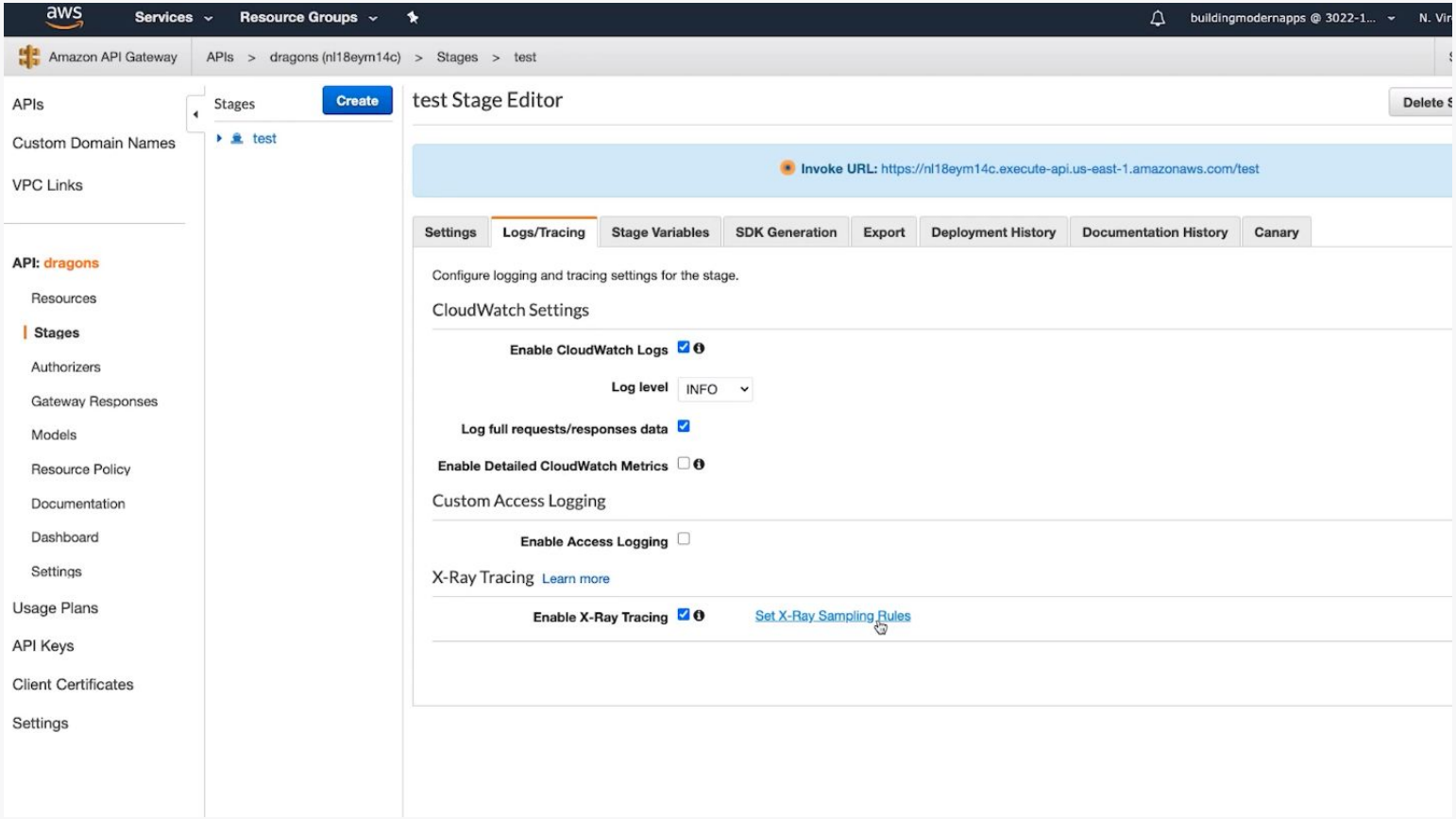


To enable active tracing, you go through the API stages.


aws
Services
Resource Groups
buildingmodernapps @ 3022-1...
N. Vir
Amazon API Gateway
APIs > dragons (nl18eym14c) > Stages > test
APIs
Custom Domain Names
VPC Links
API: dragons
Resources
Stages
Authorizers
Gateway Responses
Models
Resource Policy
Documentation
Dashboard
Settings
Usage Plans
API Keys
Client Certificates
Settings
Stages
Create
test
test Stage Editor
Delete S
Invoke URL: https://nl18eym14c.execute-api.us-east-1.amazonaws.com/test
Settings
Logs/Tracing
Stage Variables
SDK Generation
Export
Deployment History
Documentation History
Canary
Configure logging and tracing settings for the stage.
CloudWatch Settings
Enable CloudWatch Logs
Log level
INFO
Log full requests/responses data
Enable Detailed CloudWatch Metrics
Custom Access Logging
Enable Access Logging
X-Ray Tracing
Learn more
Enable X-Ray Tracing
Set X-Ray Sampling Rules



To enable the tracing, enable X Ray Tracing.



To adjust the sampling rules from the default, choose set X-Ray sampling rules, and go into the X-Ray console and configure the rules there.

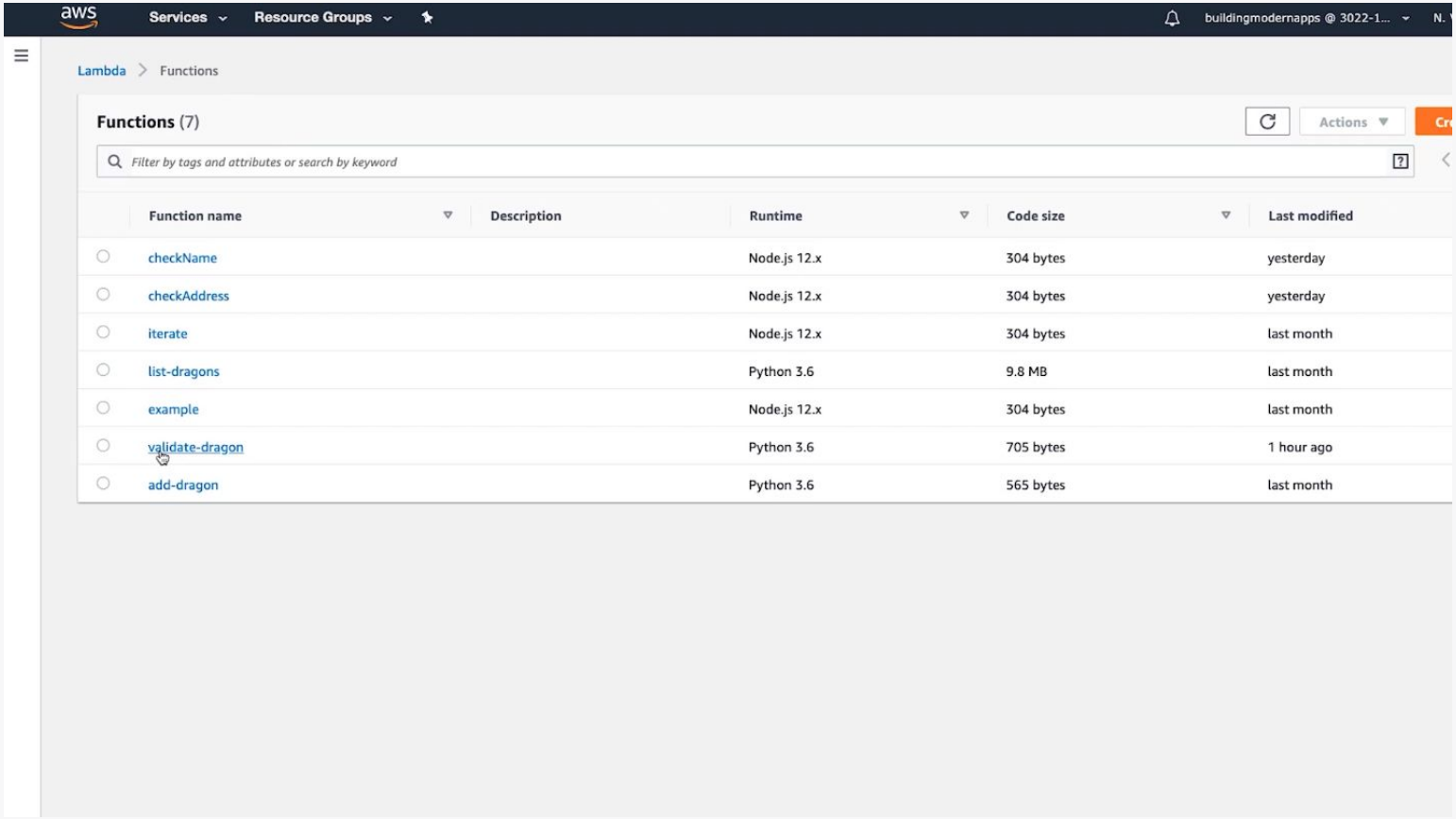
Lambda trace data is sent to X-Ray automatically.

The upstream service such as API Gateway, samples incoming requests, and adds a tracing header that tells Lambda whether to send traces or not.

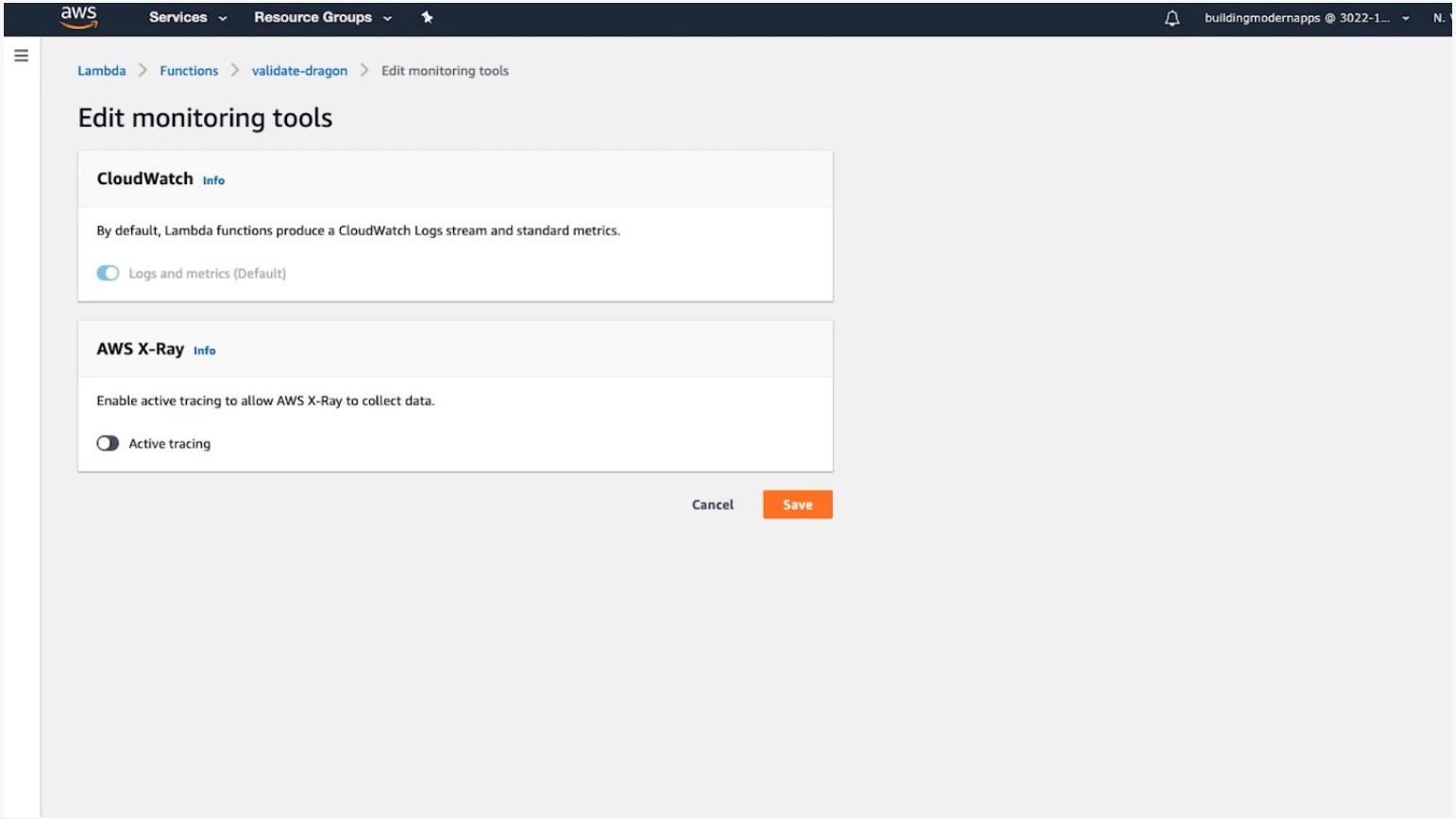
This is the passive method.

Lambda supports both passive and active tracing.

If you want to enable active tracing,
this can be done through the functions configuration.



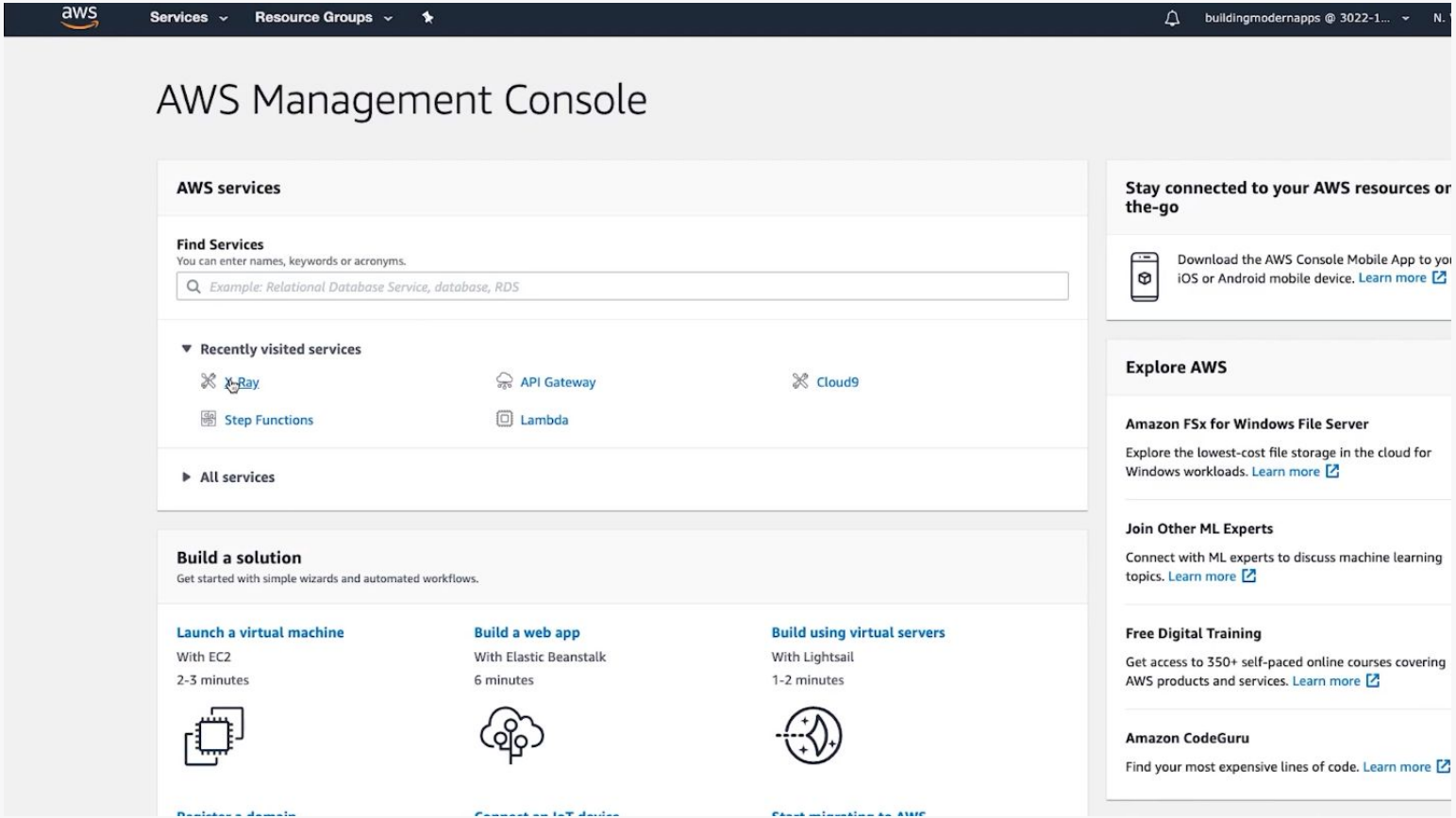
lambda functions



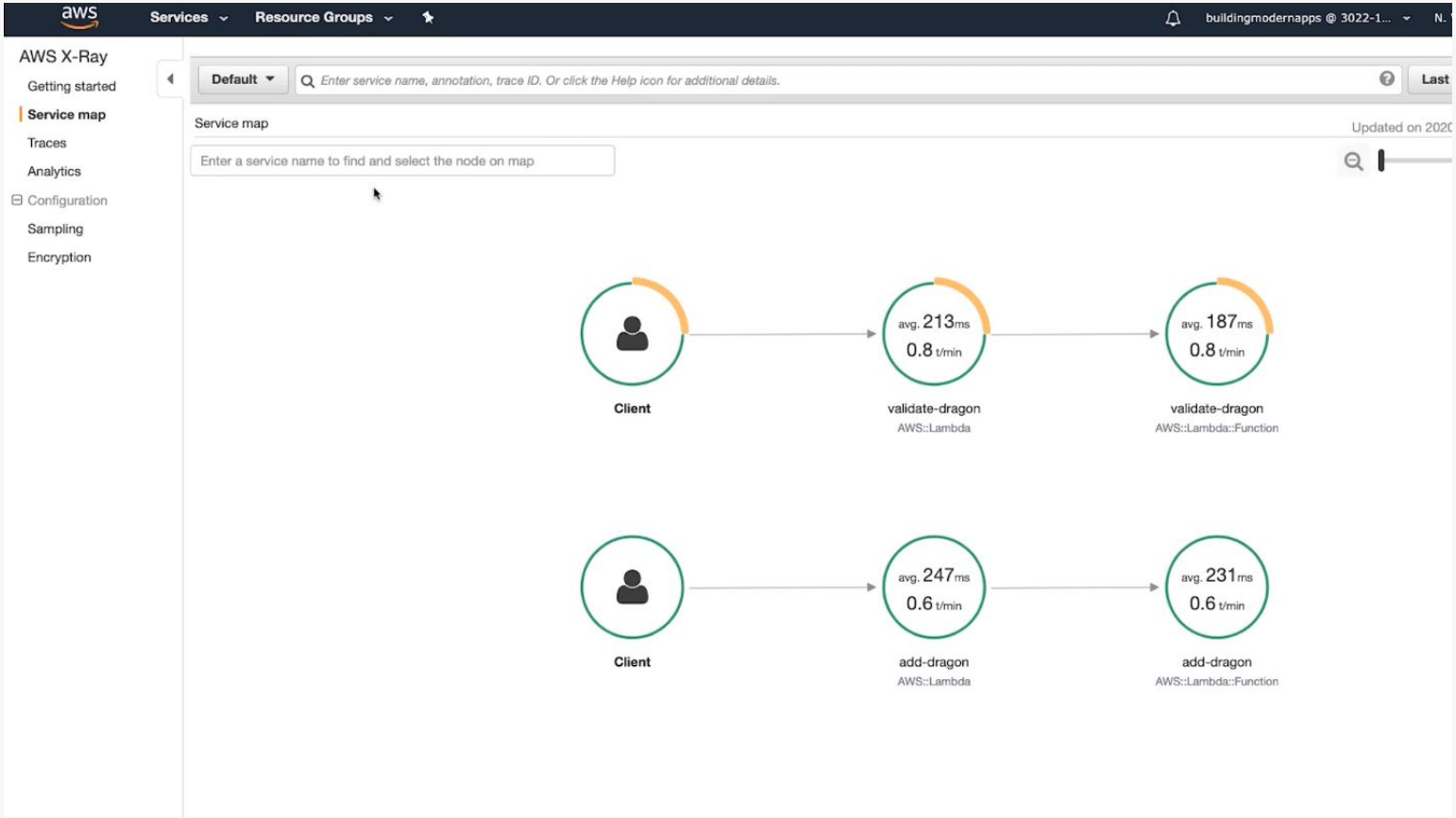
active AWS X-Ray

The active tracing enables Lambda to trace a subset of invocations that have not been sampled by the upstream service.

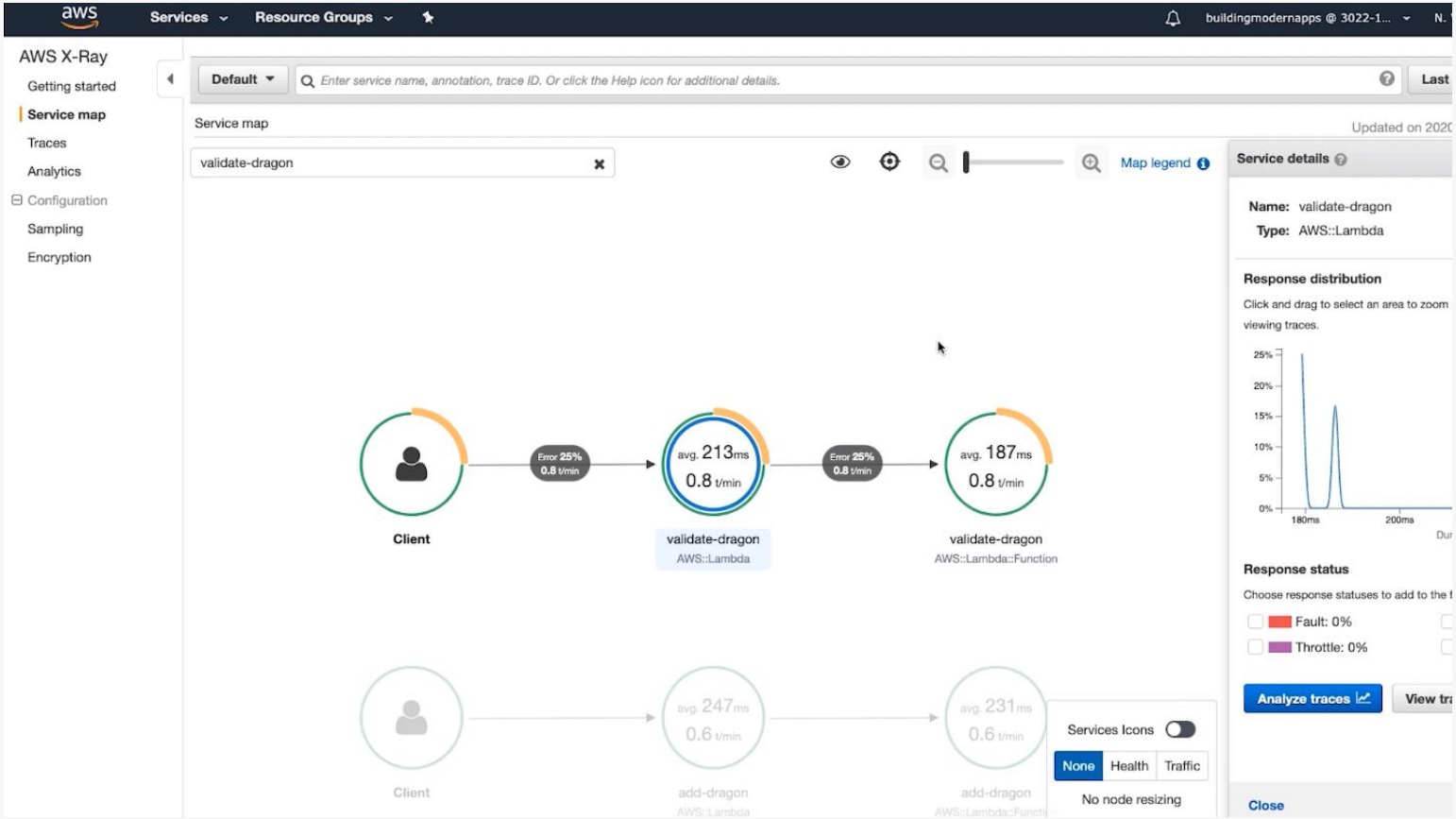
Once that's all done, you can finally see the service map.



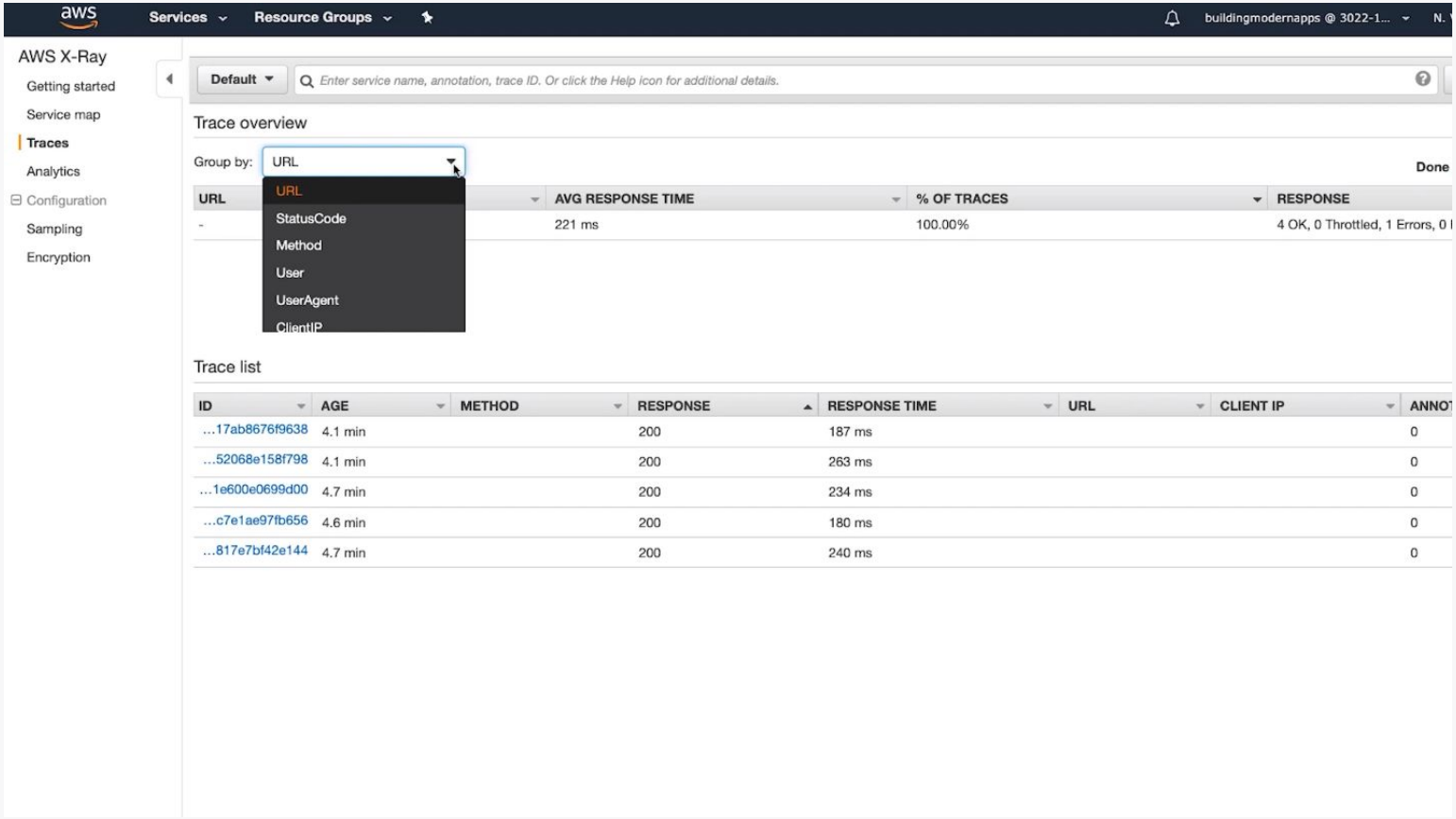
go to the X-Ray


aws
Services
Resource Groups
buildingmodernapps @ 3022-1...
AWS X-Ray
Getting started
Service map
Traces
Analytics
Configuration
Sampling
Encryption
Default
Enter service name, annotation, trace ID. Or click the Help icon for additional details.
Last
Service map
Updated on 2020
Enter a service name to find and select the node on map
avg. 213ms
0.8 t/min
Client
validate-dragon
AWS::Lambda
avg. 187ms
0.8 t/min
validate-dragon
AWS::Lambda::Function
avg. 247ms
0.6 t/min
Client
add-dragon
AWS::Lambda
avg. 231ms
0.6 t/min
add-dragon
AWS::Lambda::Function

Service Map

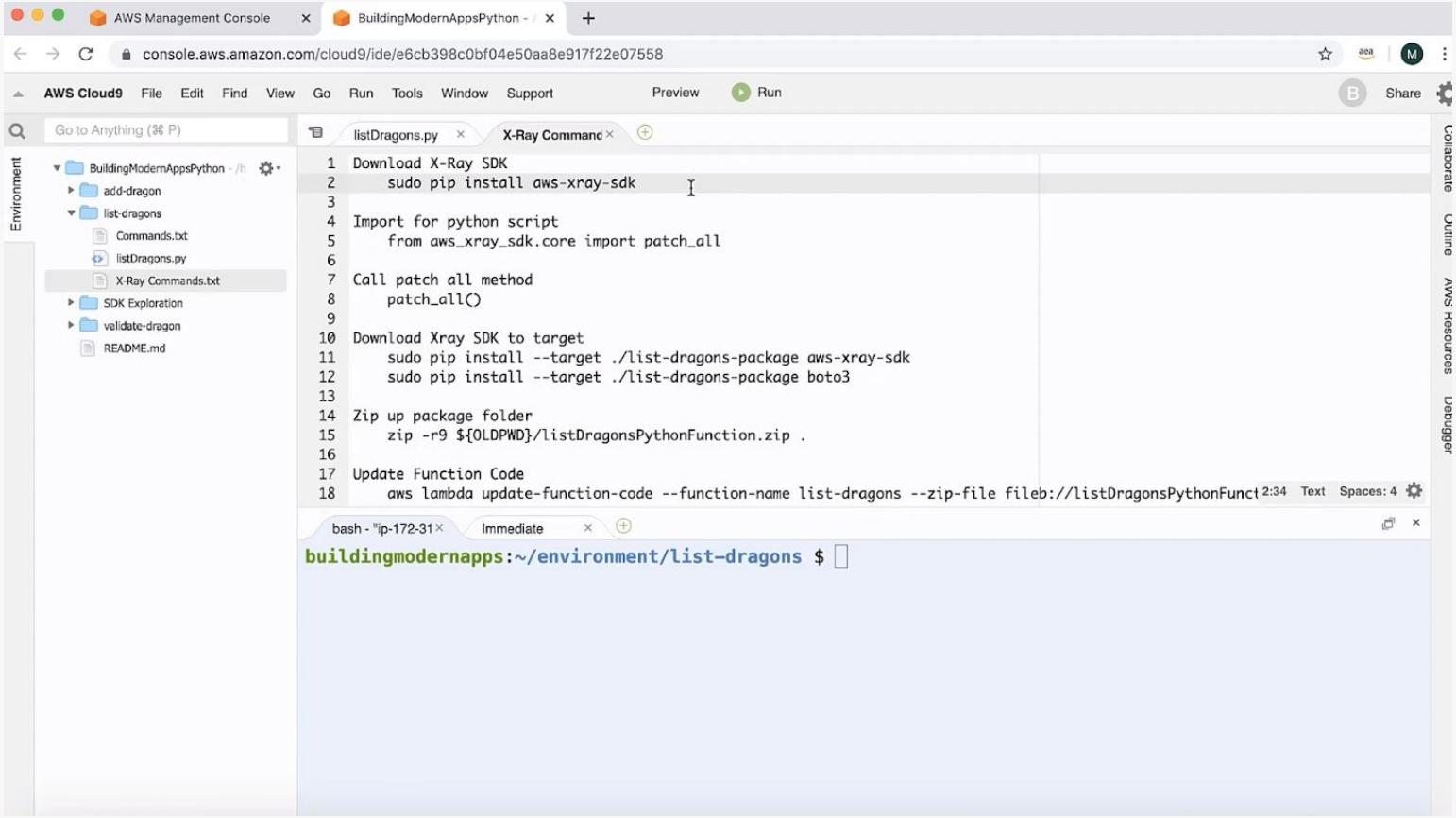


Selecting one of the components brings up the service details.



in the Traces, you can organize data based on URL, status code, method, etc.

There is more that you can do to customize the data you're getting,
but that's a deeper level and will require additional coding.



Steps to add X-Ray to your code

Good Resources:

What is AWS X-Ray? - AWS X-Ray

AWS X-Ray daemon

AWS X-Ray SDK for Python

Amazon CloudWatch Logs

Amazon CloudWatch Logs enables you to
monitor, store and access your log files
from many different sources and enables you to
centralize the logs from all of your systems, applications, any of your services
that you use in a single, highly available service.

CloudWatch Logs enables you to see
all of your logs regardless of their source
as a single and consistent flow of events over time.

You can query them and sort them based on other dimensions,
group them by specific fields,
create custom computations with a query language
and visualize log data in dashboards.

Log event is a record of some activity
recorded by the application or resource being monitored.

log stream is a sequence of log events that share the same source.

Log group is a group of log streams that can share the same retention,
monitoring and access control settings.

You pay for storing log events.

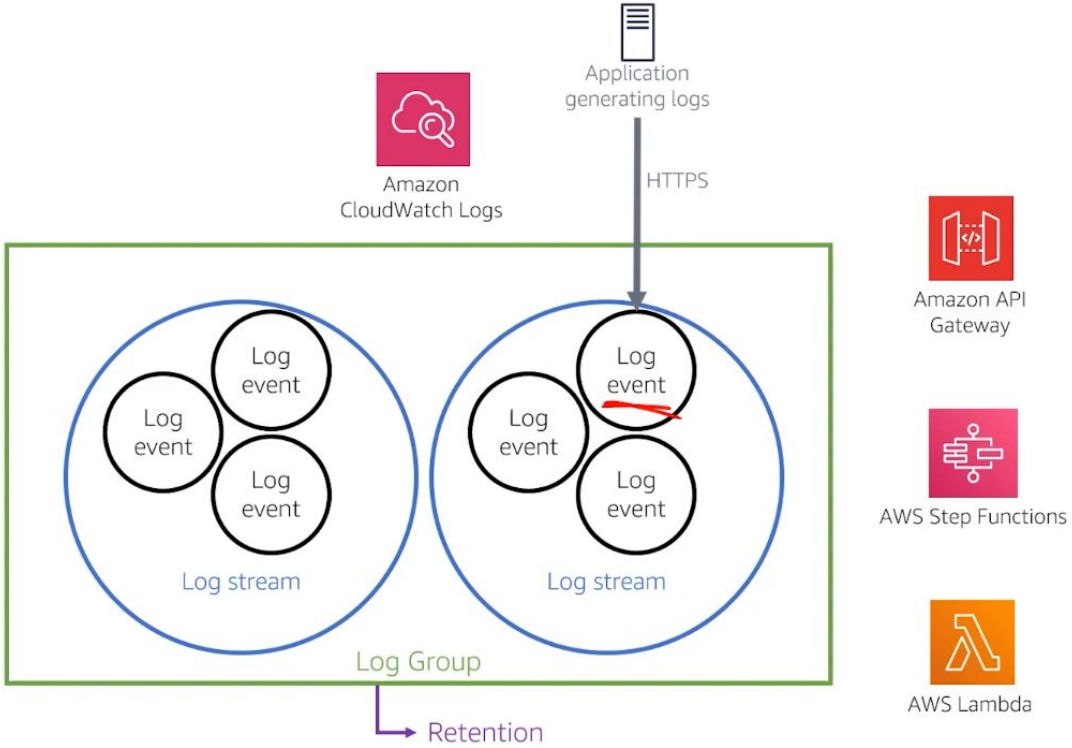
So you may want to get rid of some of these logs after a certain period of time.

The way log events are collected is the same way
as how you communicate with other AWS services
via ???

You pay for storing log events.

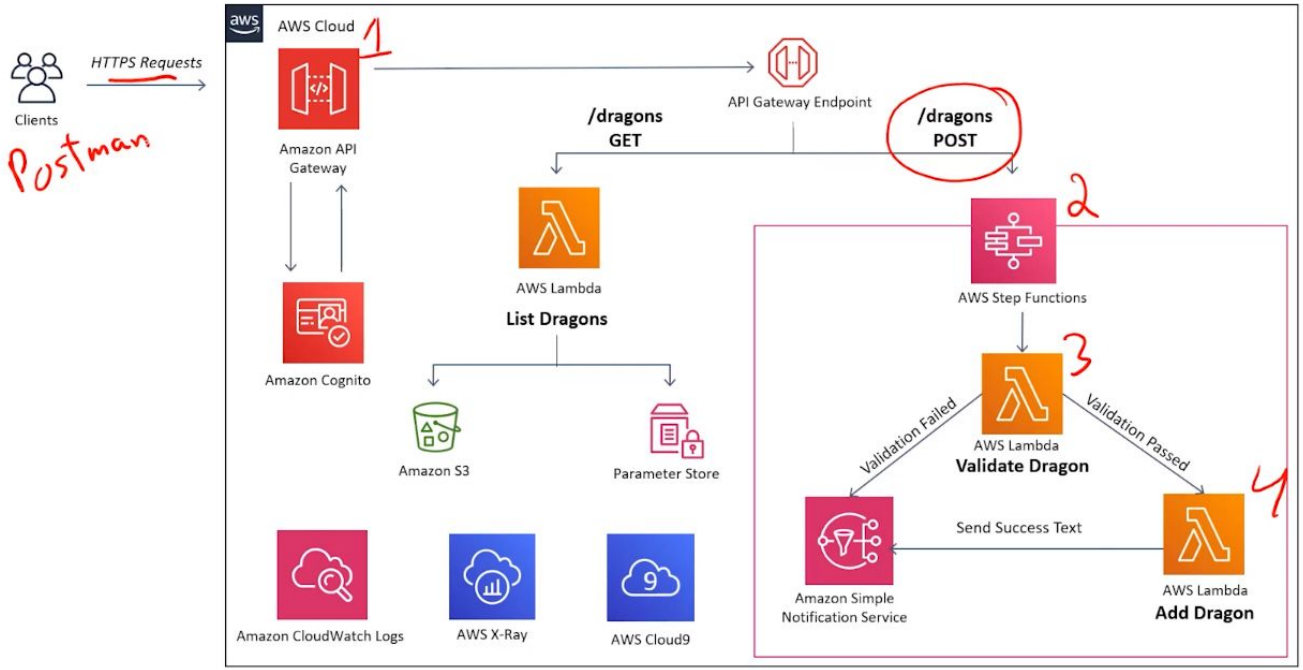
So you may want to get rid of some of these logs after a certain period of time.

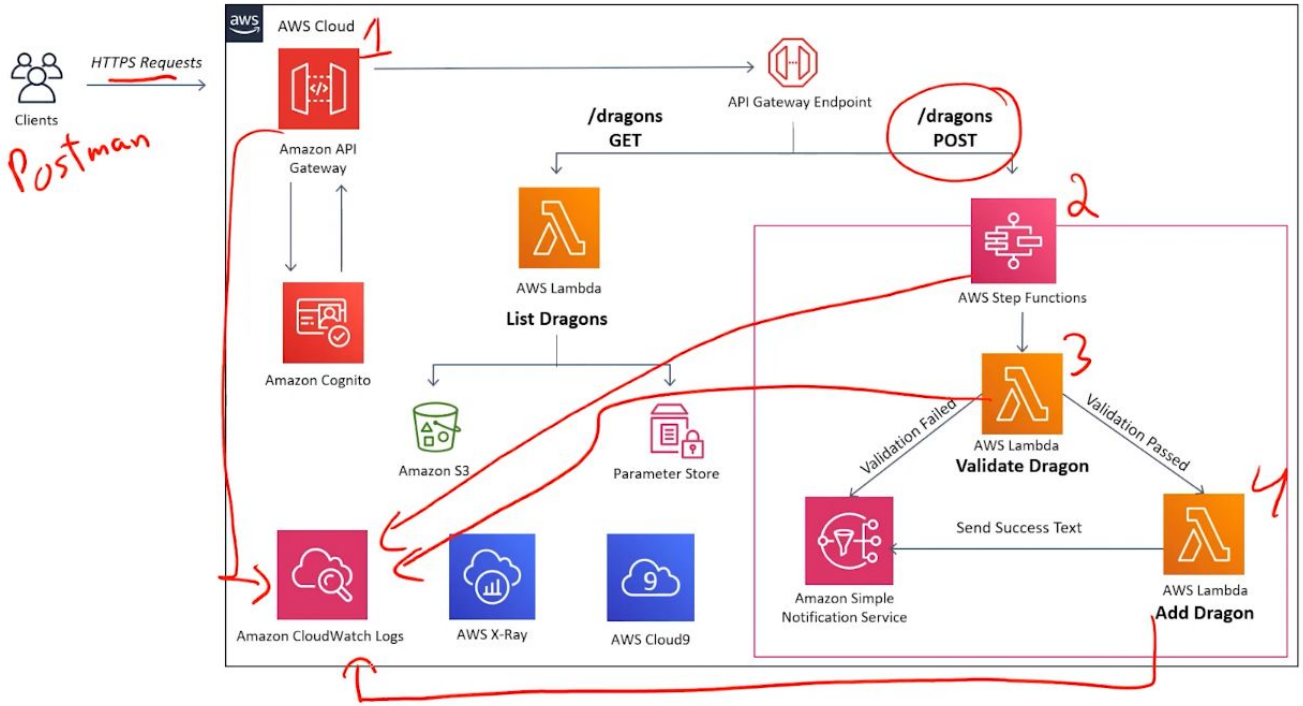
The way log events are collected is the same way
as how you communicate with other AWS services
via an HTTPS request.



all support CloudWatch Logs as a destination for their logs.

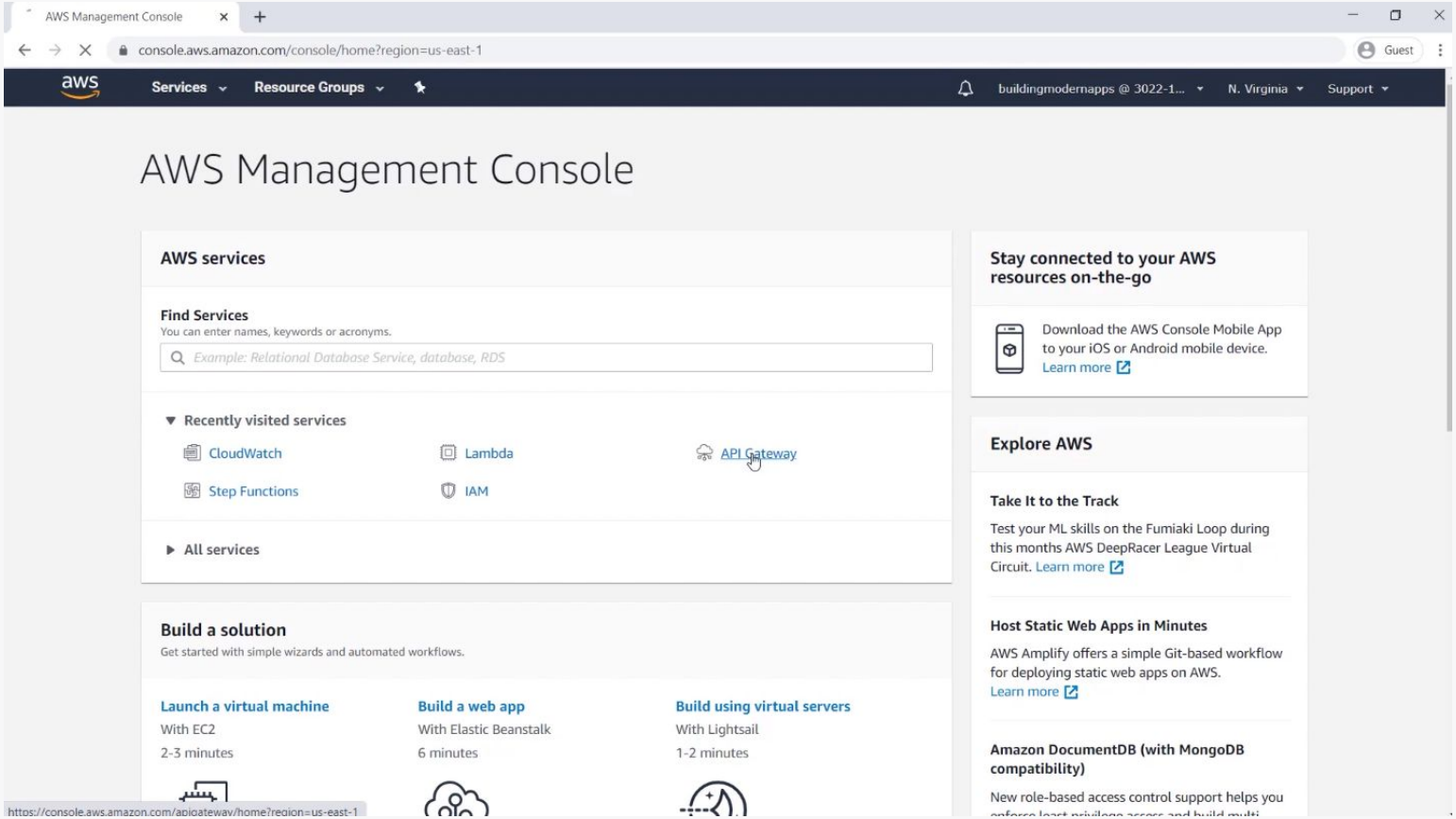
Let's say that we want to troubleshoot the `POST/dragons`.


Clients
HTTPS Requests
Postman
aws
AWS Cloud
1
Amazon API Gateway
API Gateway Endpoint
/dragons
GET
/dragons
POST
Amazon Cognito
AWS Lambda
List Dragons
Amazon S3
Parameter Store
2
AWS Step Functions
3
AWS Lambda
Validate Dragon
Validation Failed
Validation Passed
Send Success Text
Amazon Simple Notification Service
4
AWS Lambda
Add Dragon
Amazon CloudWatch Logs
AWS X-Ray
AWS Cloud9

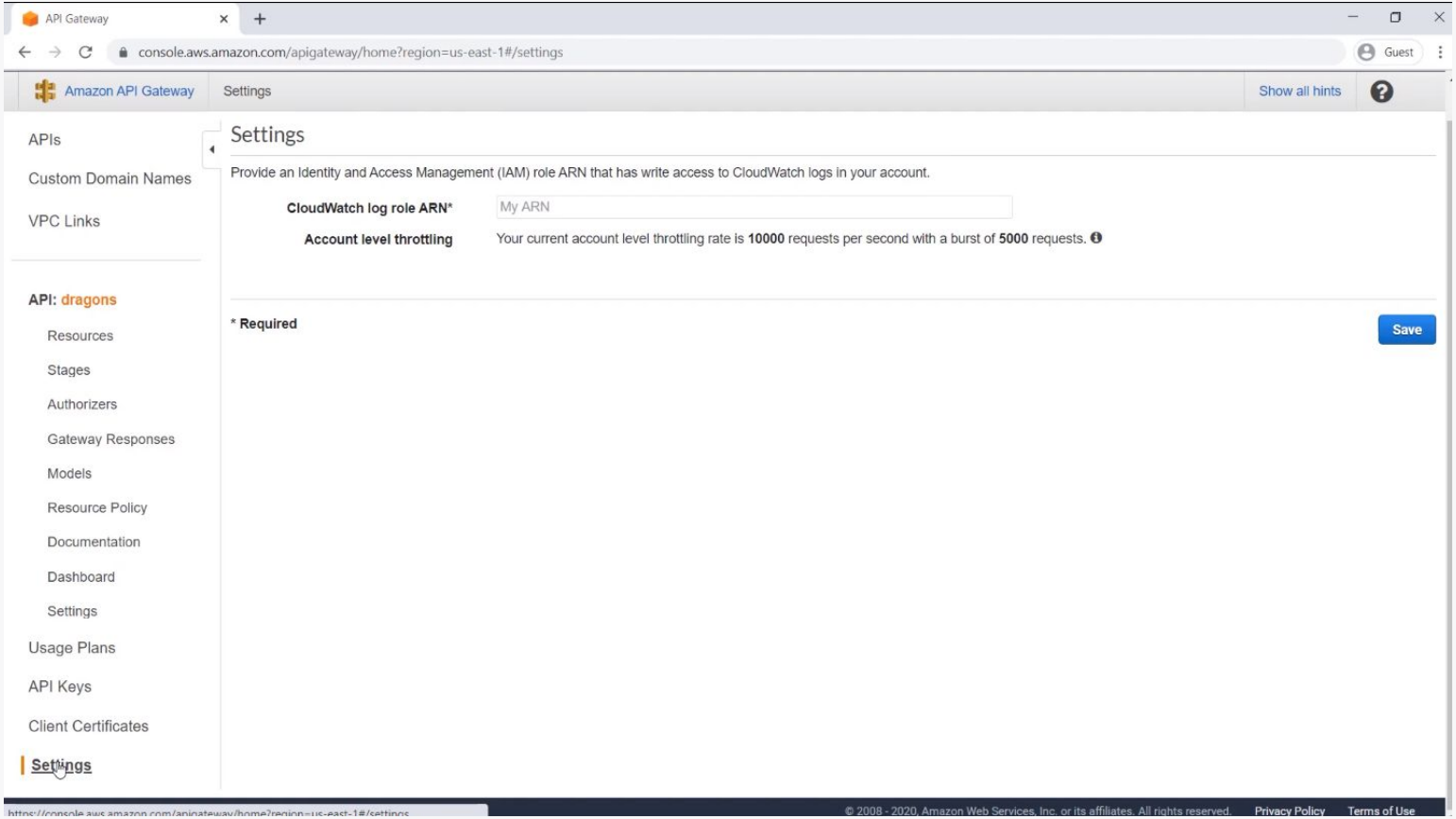

Clients
HTTPS Requests
Postman
AWS Cloud
1
Amazon API Gateway
API Gateway Endpoint
/dragons
GET
/dragons
POST
AWS Lambda
List Dragons
Amazon Cognito
Amazon S3
Parameter Store
2
AWS Step Functions
3
AWS Lambda
Validate Dragon
Validation Failed
Validation Passed
Send Success Text
Amazon Simple Notification Service
4
AWS Lambda
Add Dragon
Amazon CloudWatch Logs
AWS X-Ray
AWS Cloud9

Let's get into the AWS Management Console to configure all of those of the settings.

Remember that this can be done via the CLI and the SDK.



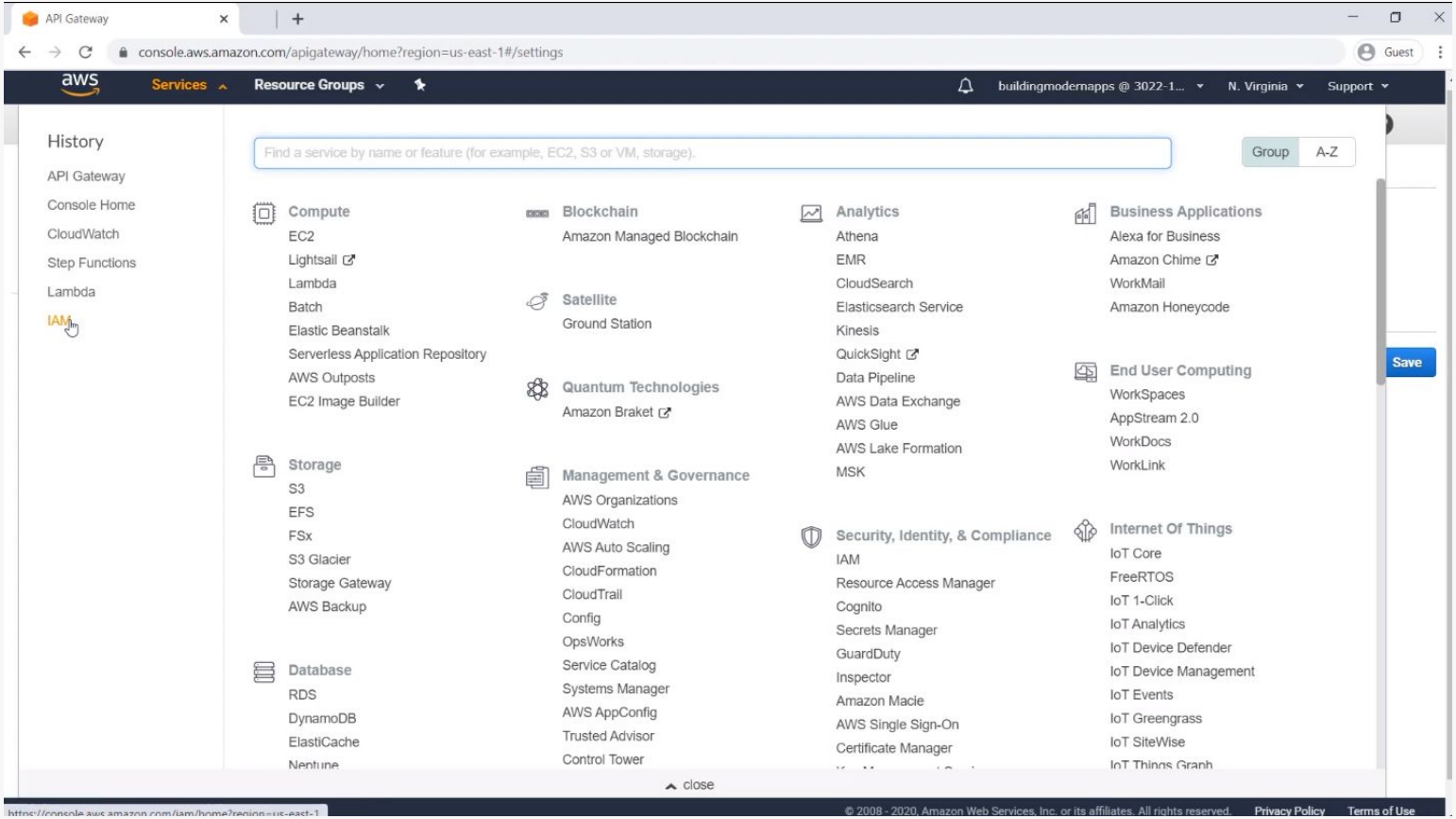
First navigate to API Gateway.



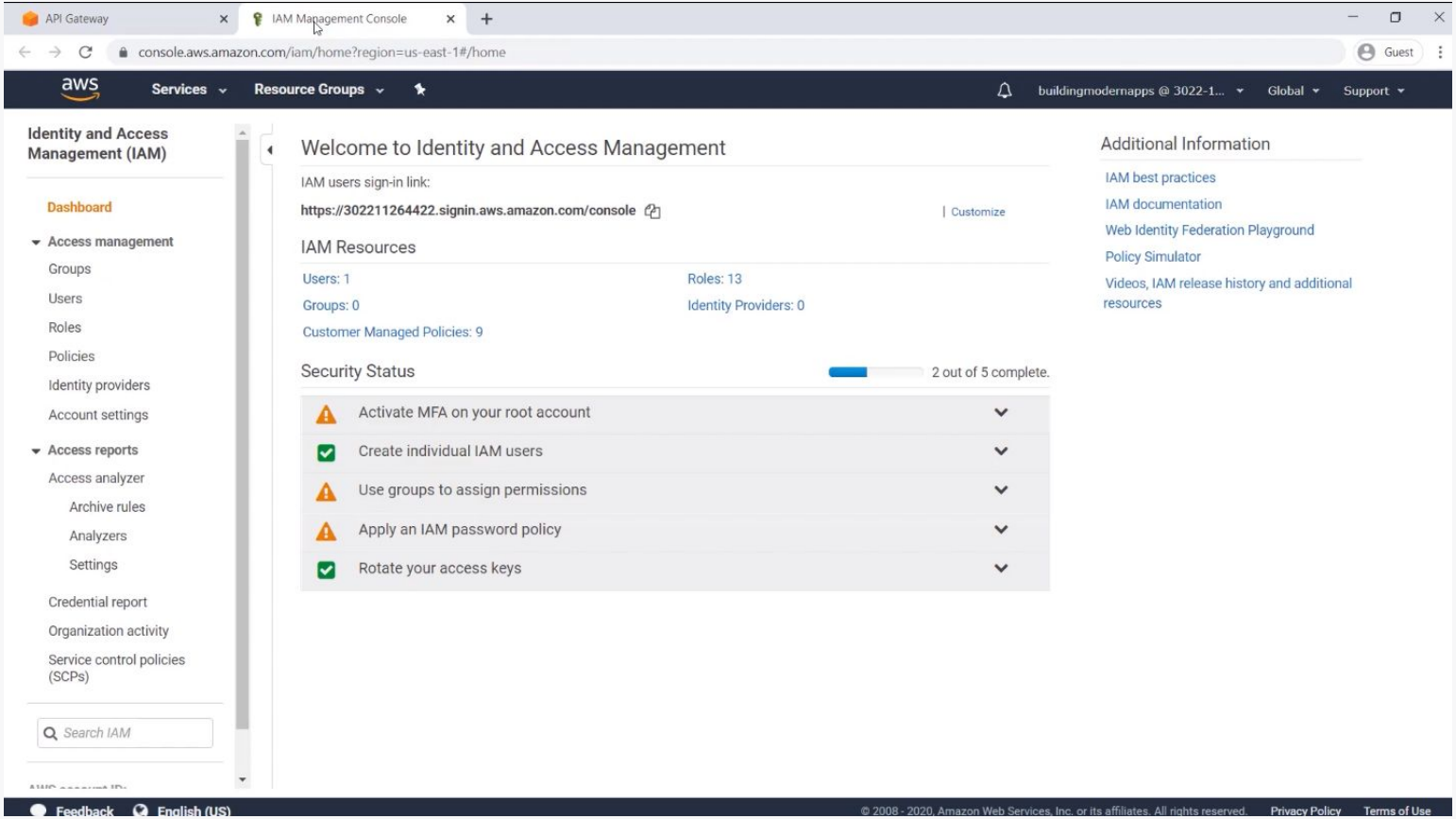
go to the setting

The first thing that it ask is a CloudWatch log role ARN.

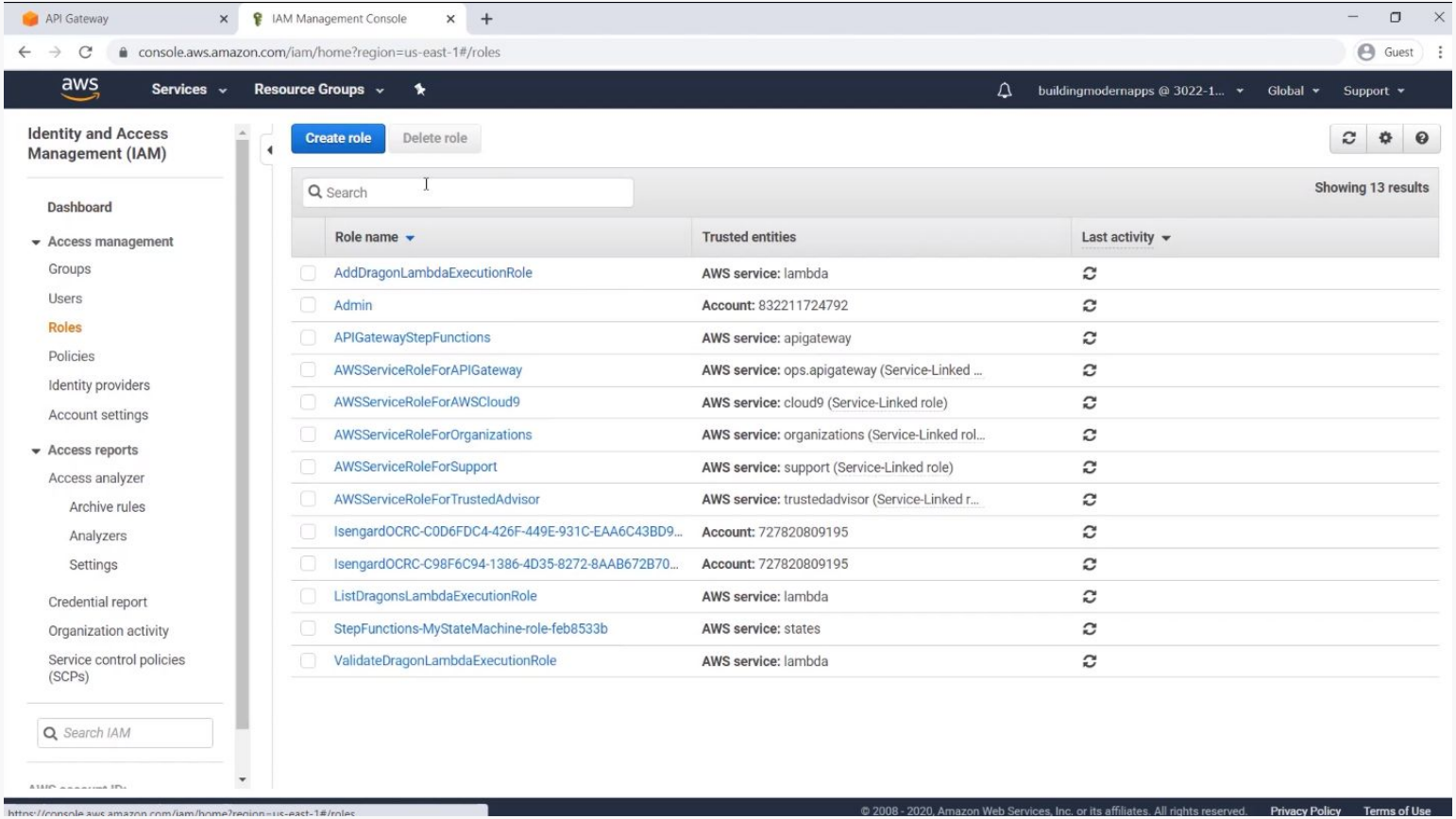
If we don't have one, we must create it first.



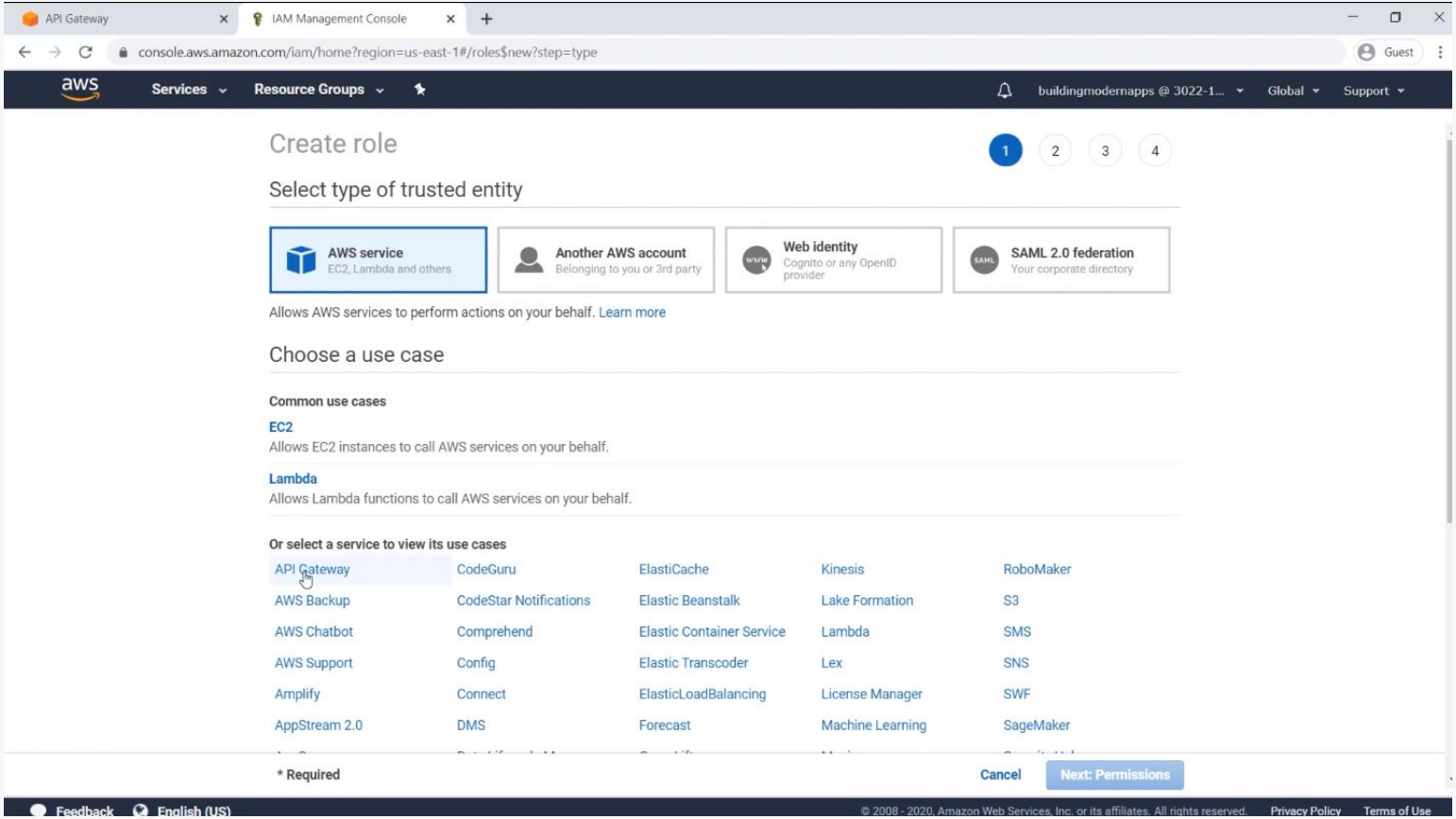
Go to the IAM service


API Gateway
IAM Management Console
console.aws.amazon.com/iam/home?region=us-east-1#/home
Guest
aws
Services
Resource Groups
buildingmodernapps @ 3022-1...
Global
Support
Identity and Access Management (IAM)
Dashboard
Access management
Groups
Users
Roles
Policies
Identity providers
Account settings
Access reports
Access analyzer
Archive rules
Analyzers
Settings
Credential report
Organization activity
Service control policies (SCPs)
Search IAM
Welcome to Identity and Access Management
IAM users sign-in link:
https://302211264422.signin.aws.amazon.com/console
Customize
IAM Resources
Users: 1
Roles: 13
Groups: 0
Identity Providers: 0
Customer Managed Policies: 9
Security Status
2 out of 5 complete.
Activate MFA on your root account
Create individual IAM users
Use groups to assign permissions
Apply an IAM password policy
Rotate your access keys
Additional Information
IAM best practices
IAM documentation
Web Identity Federation Playground
Policy Simulator
Videos, IAM release history and additional resources
Feedback
English (US)
© 2008 - 2020, Amazon Web Services, Inc. or its affiliates. All rights reserved.
Privacy Policy
Terms of Use

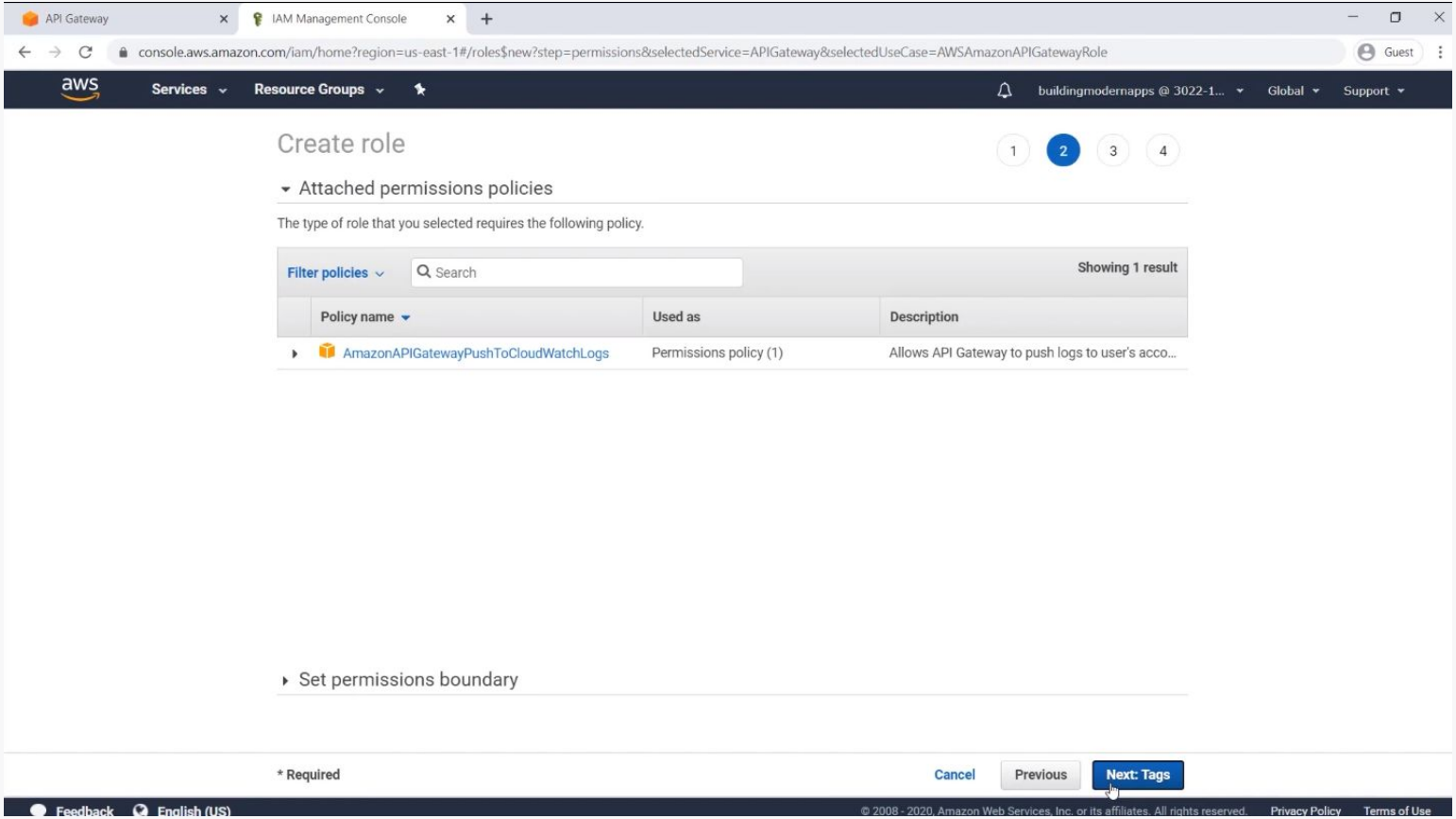
IAM console



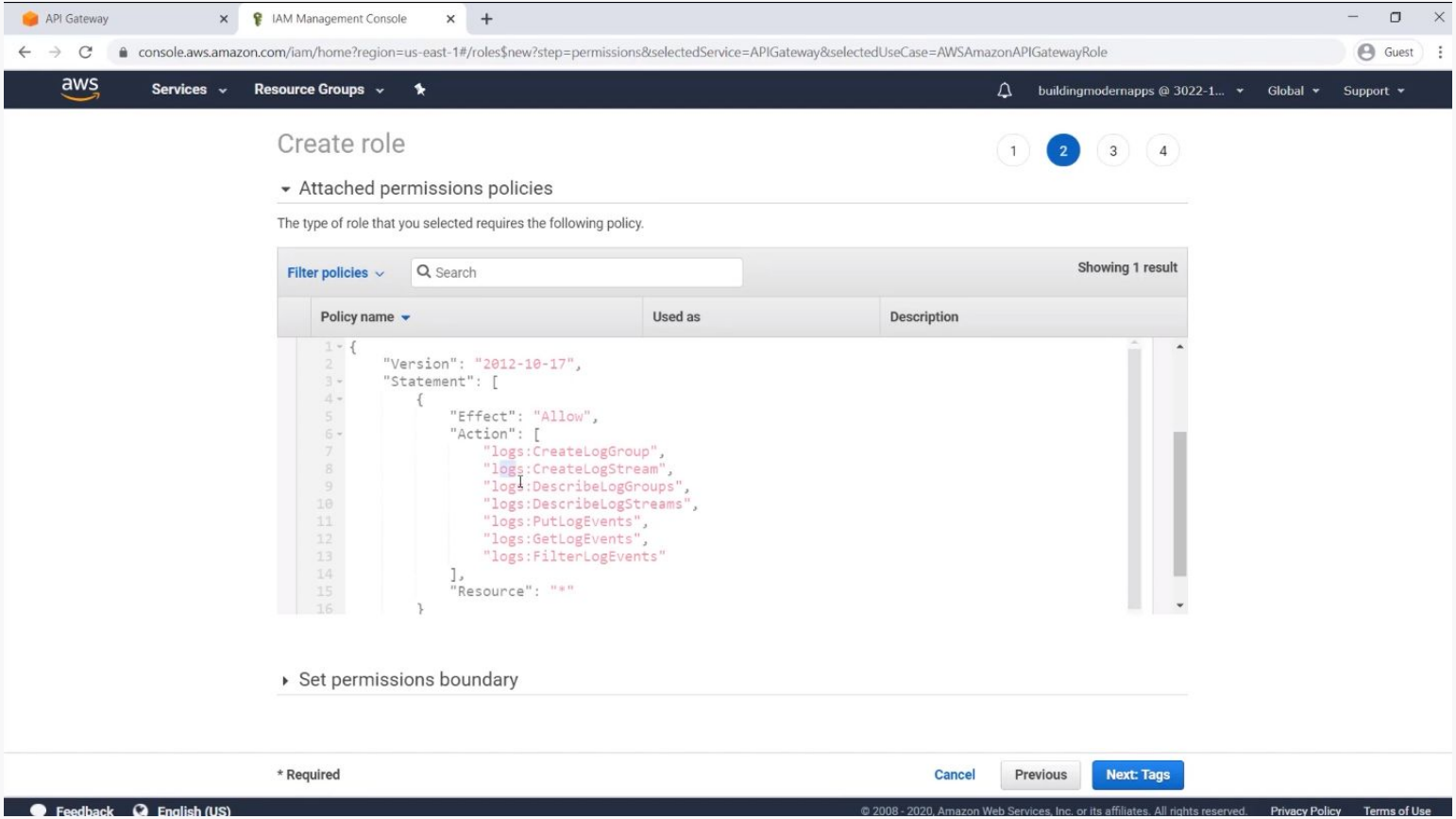
go to the Roles and Create a new role



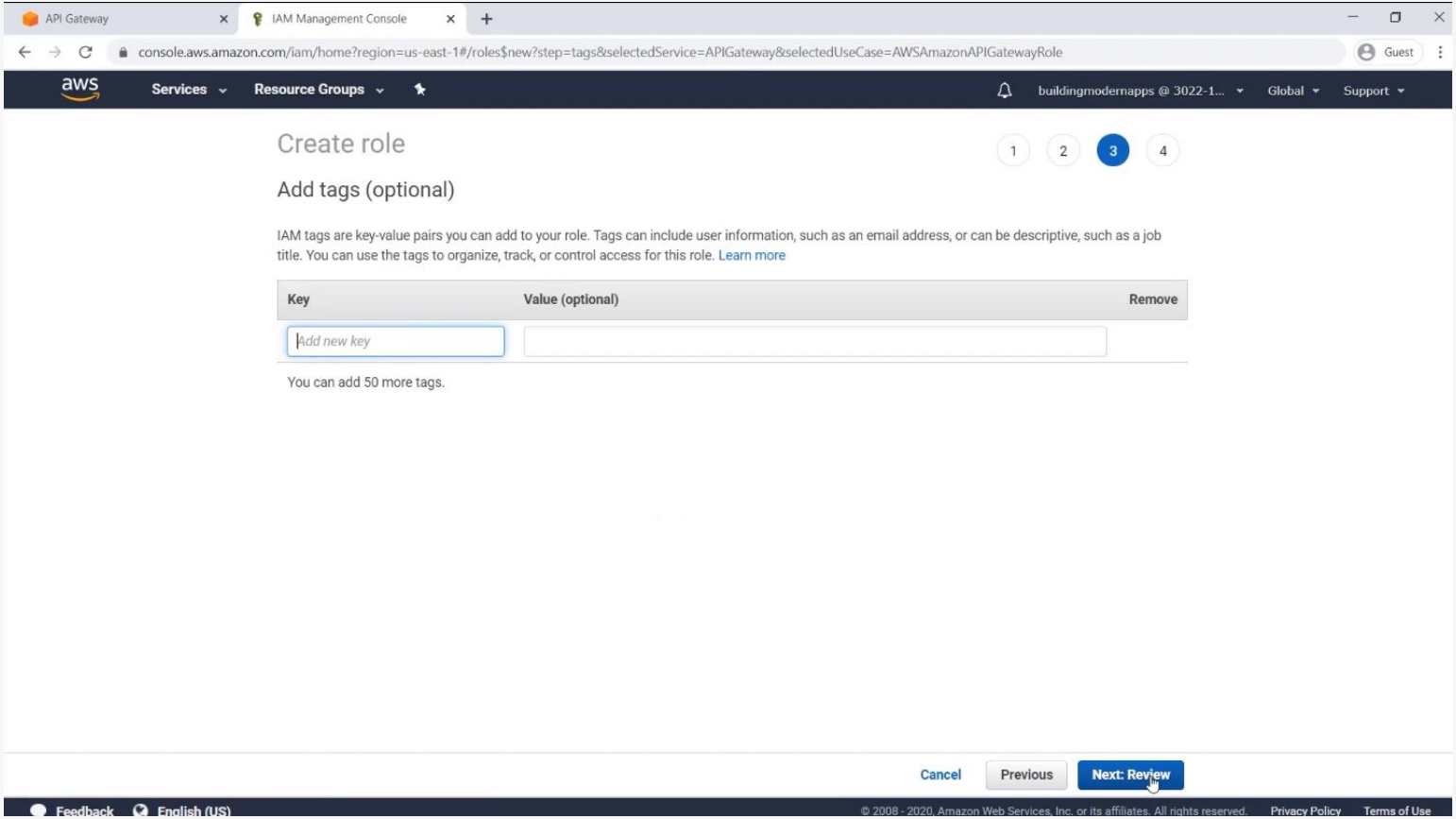
select that this role is for API Gateway



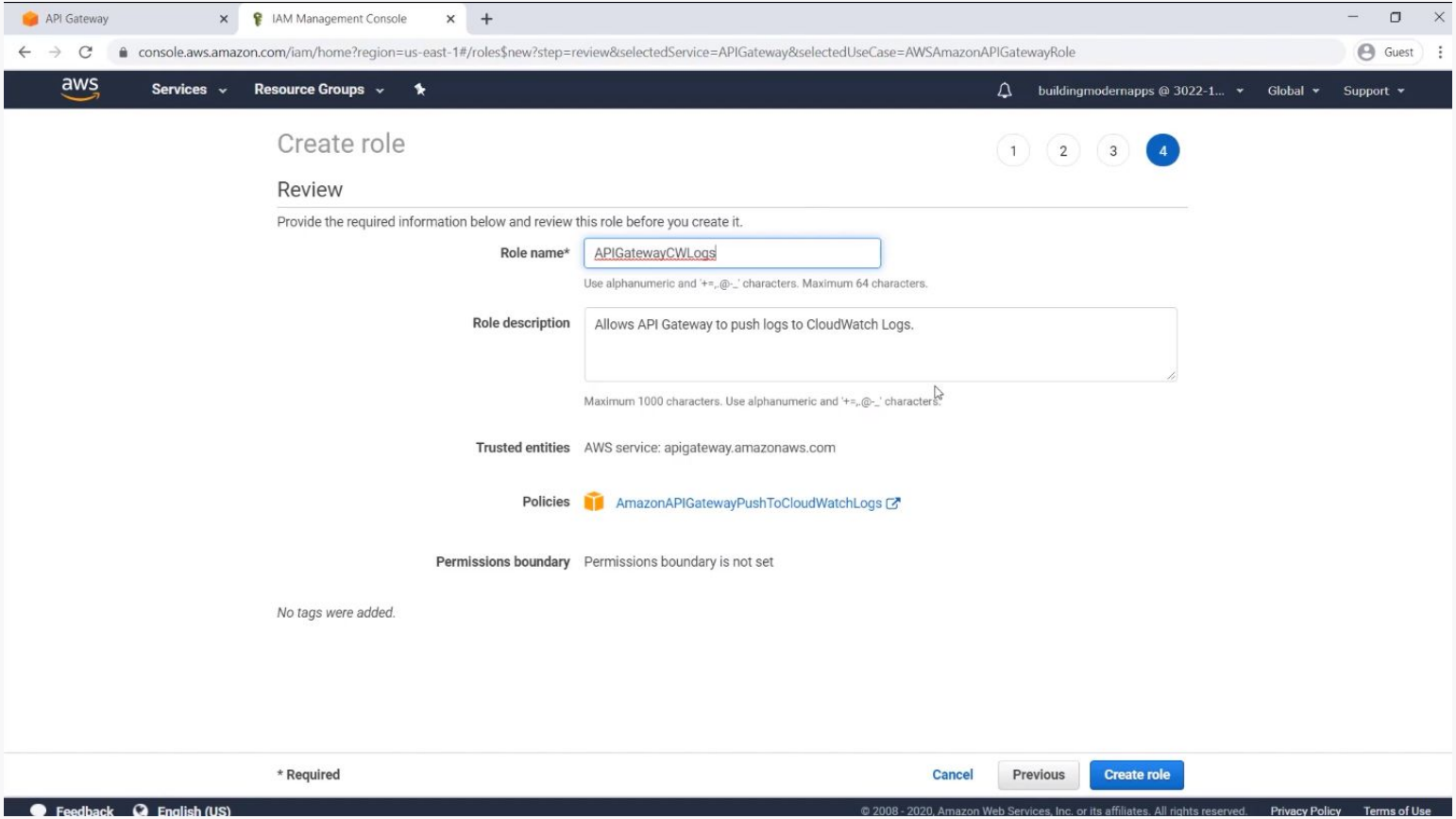
create role for APIGateway to CloudWatchLog.



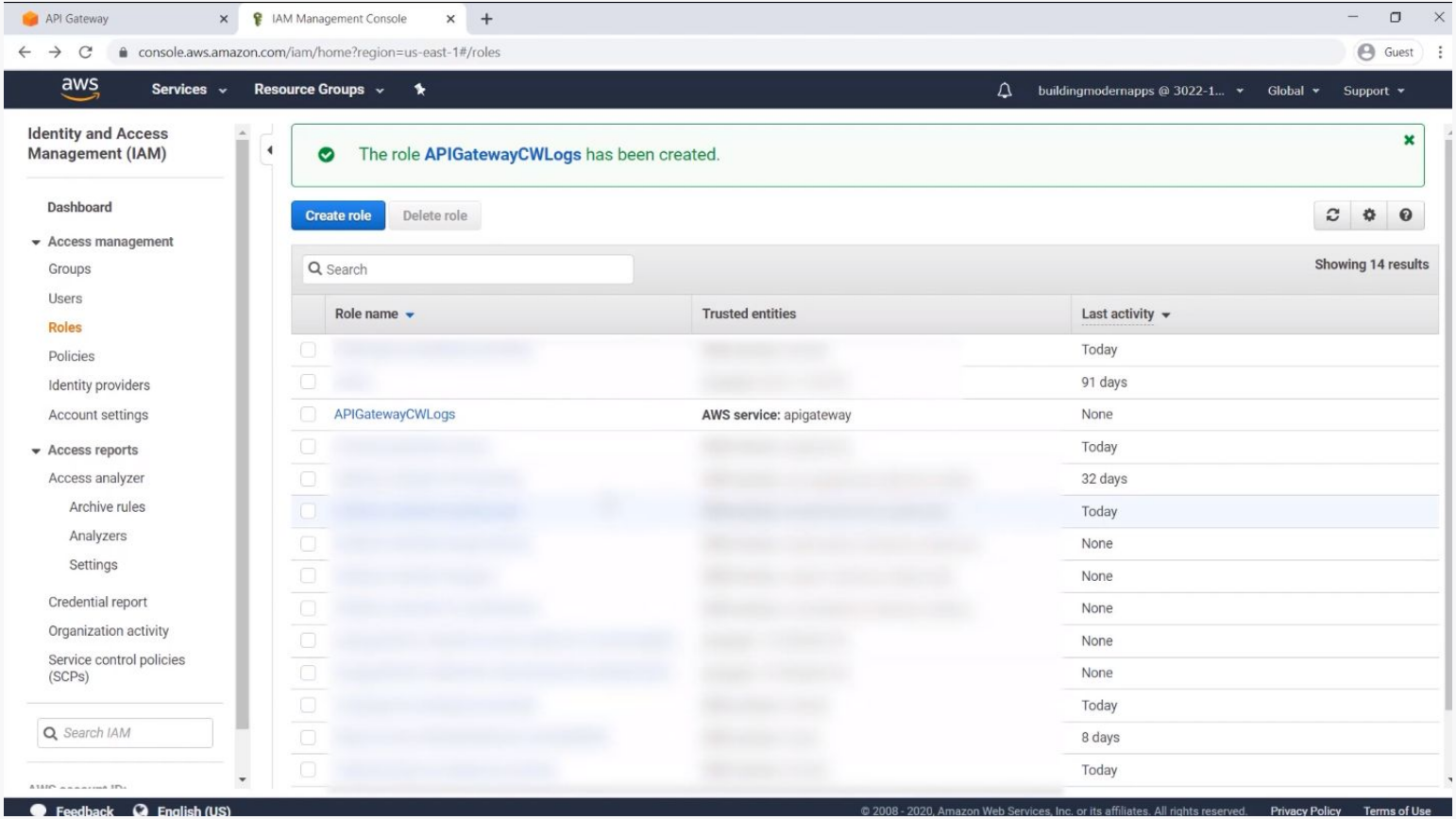
This is the IAM policy inside that IAM role which defines access to different logs



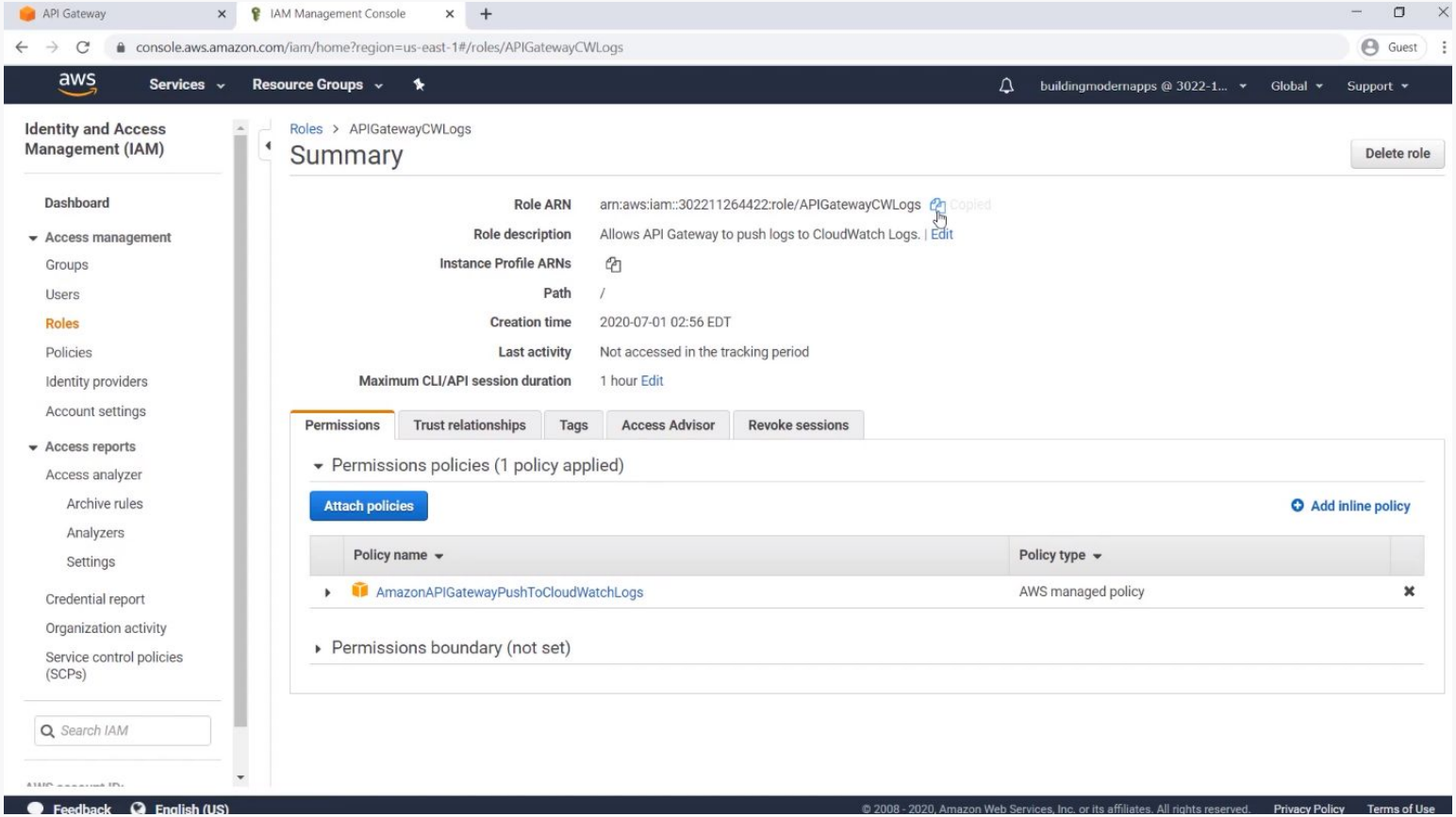
you can leave empty if you don’t need tag



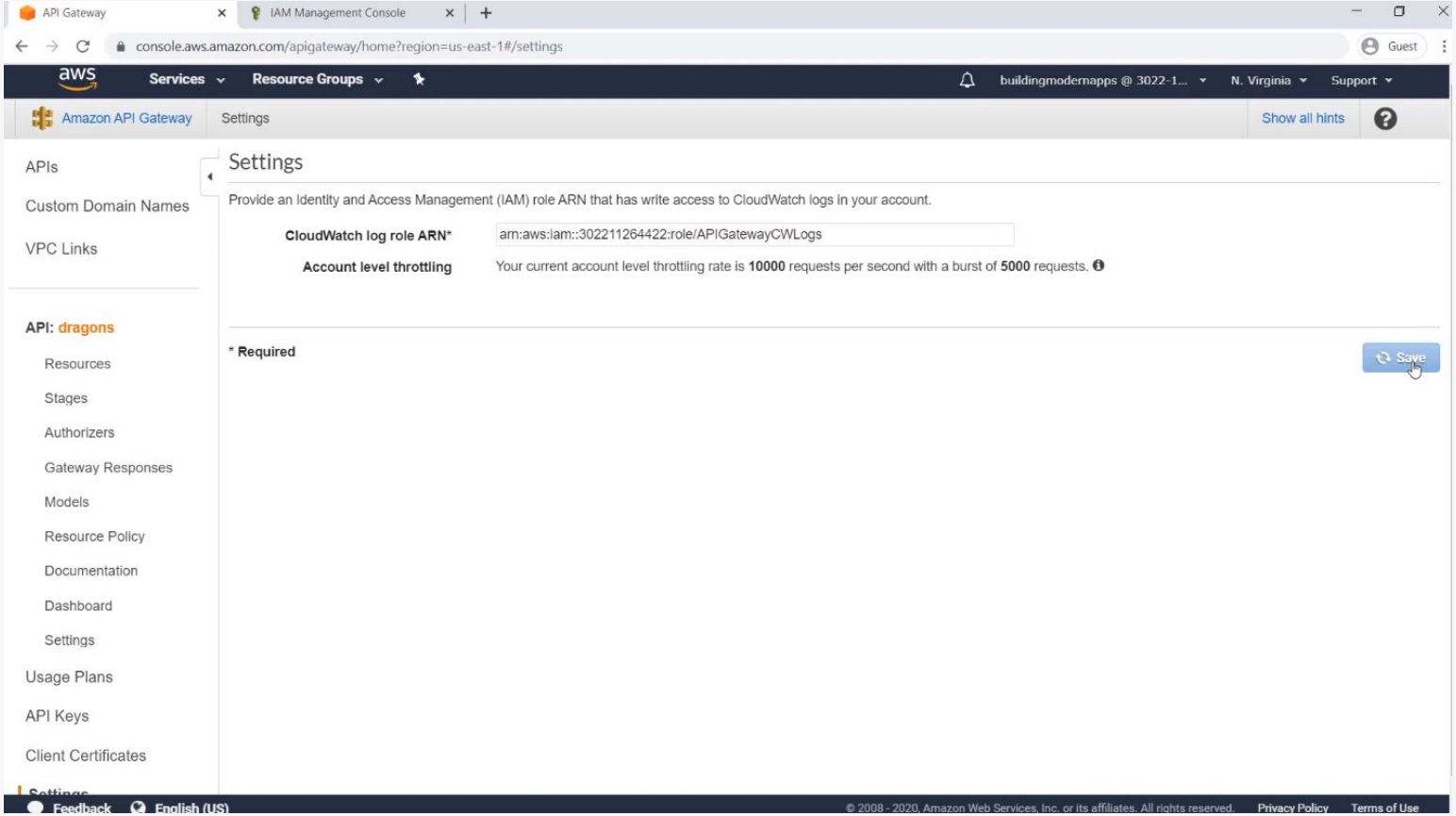
give it a name and later use this name



open the role and copy ARN



copy the Role ARN

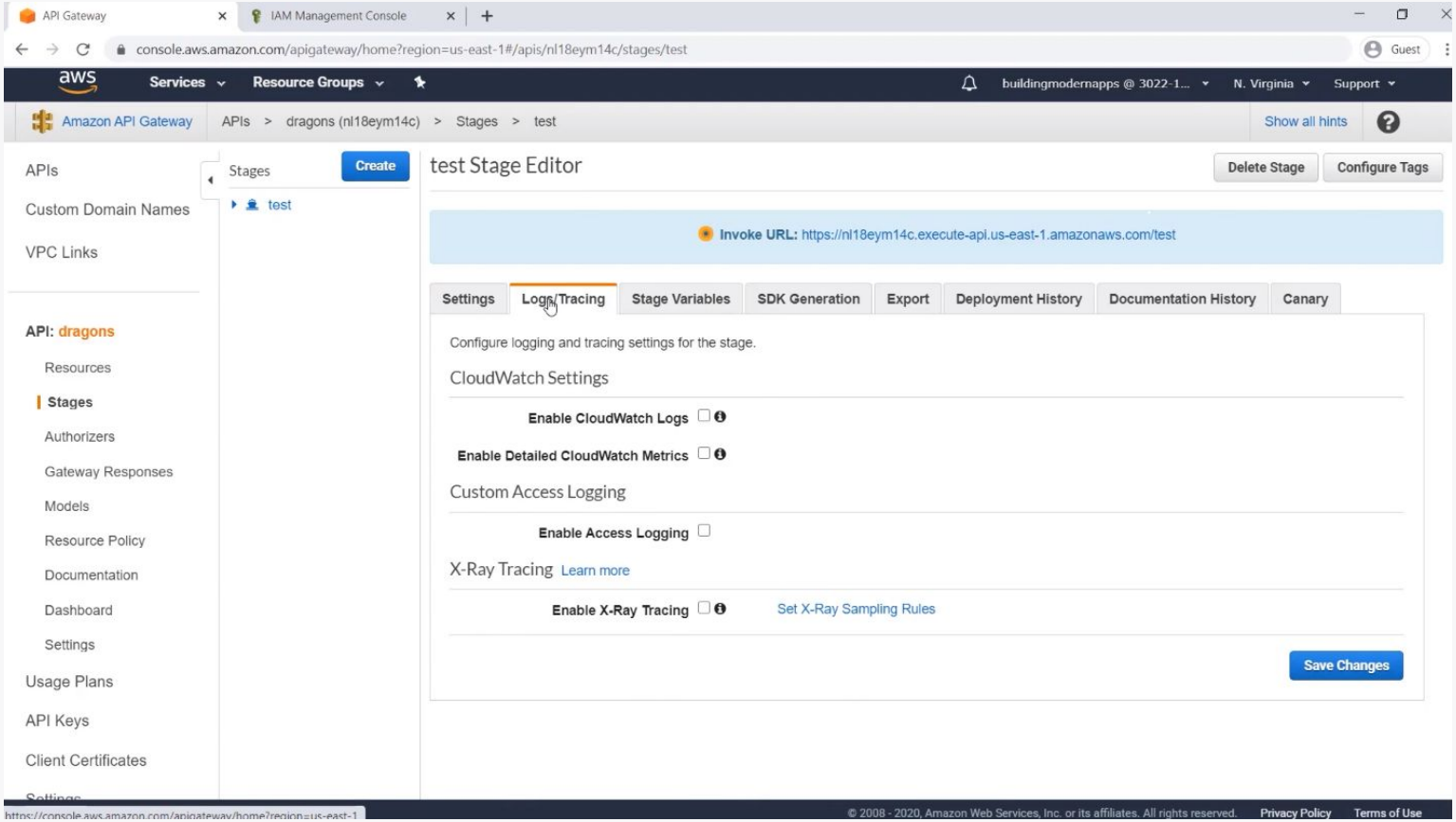


now, back to the API Gateway, we paste the role ARN

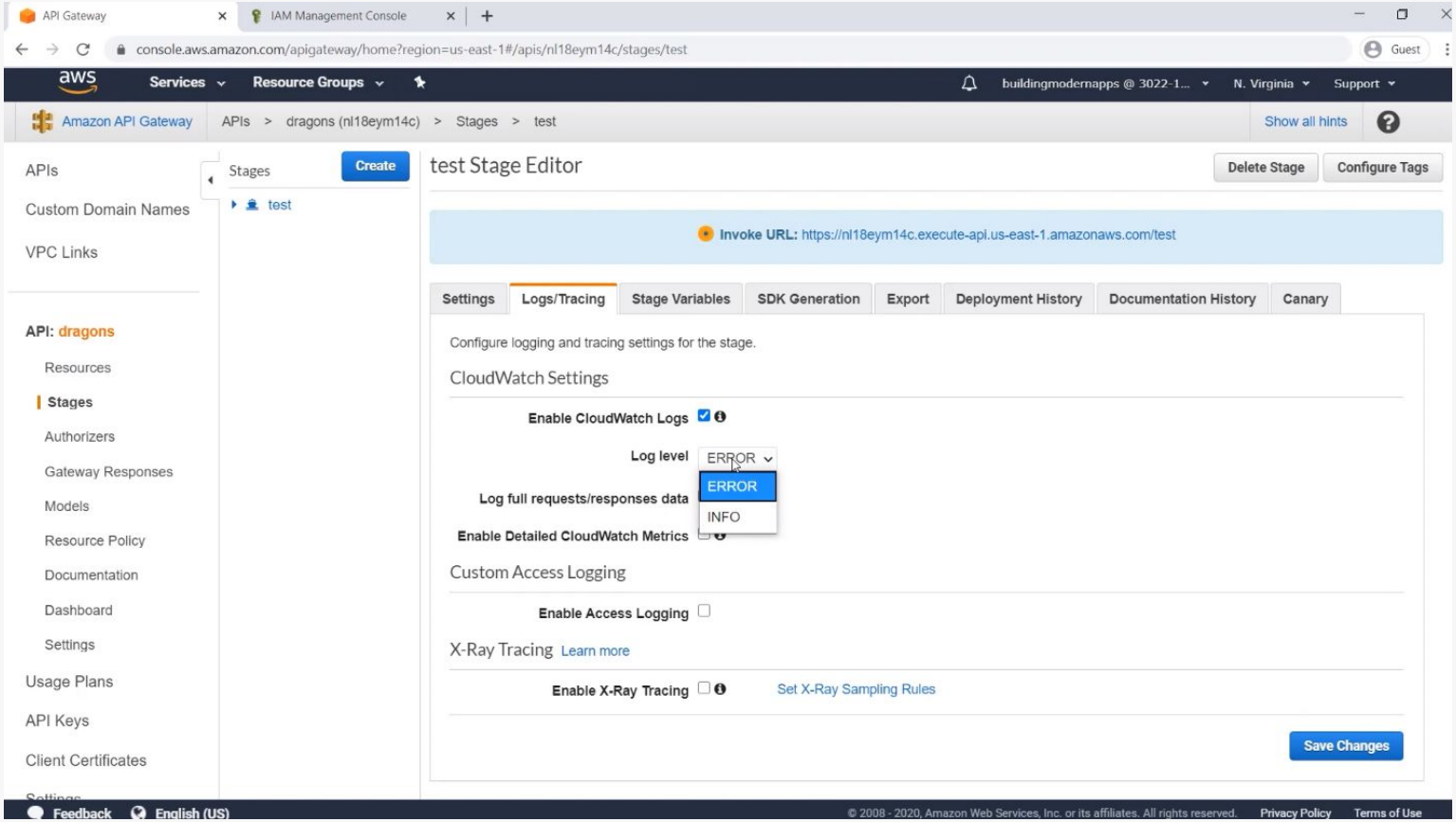
API Gateway is now allowed to send log events to CloudWatch Logs.

we still need to tell the Dragons API, when to send those logs.

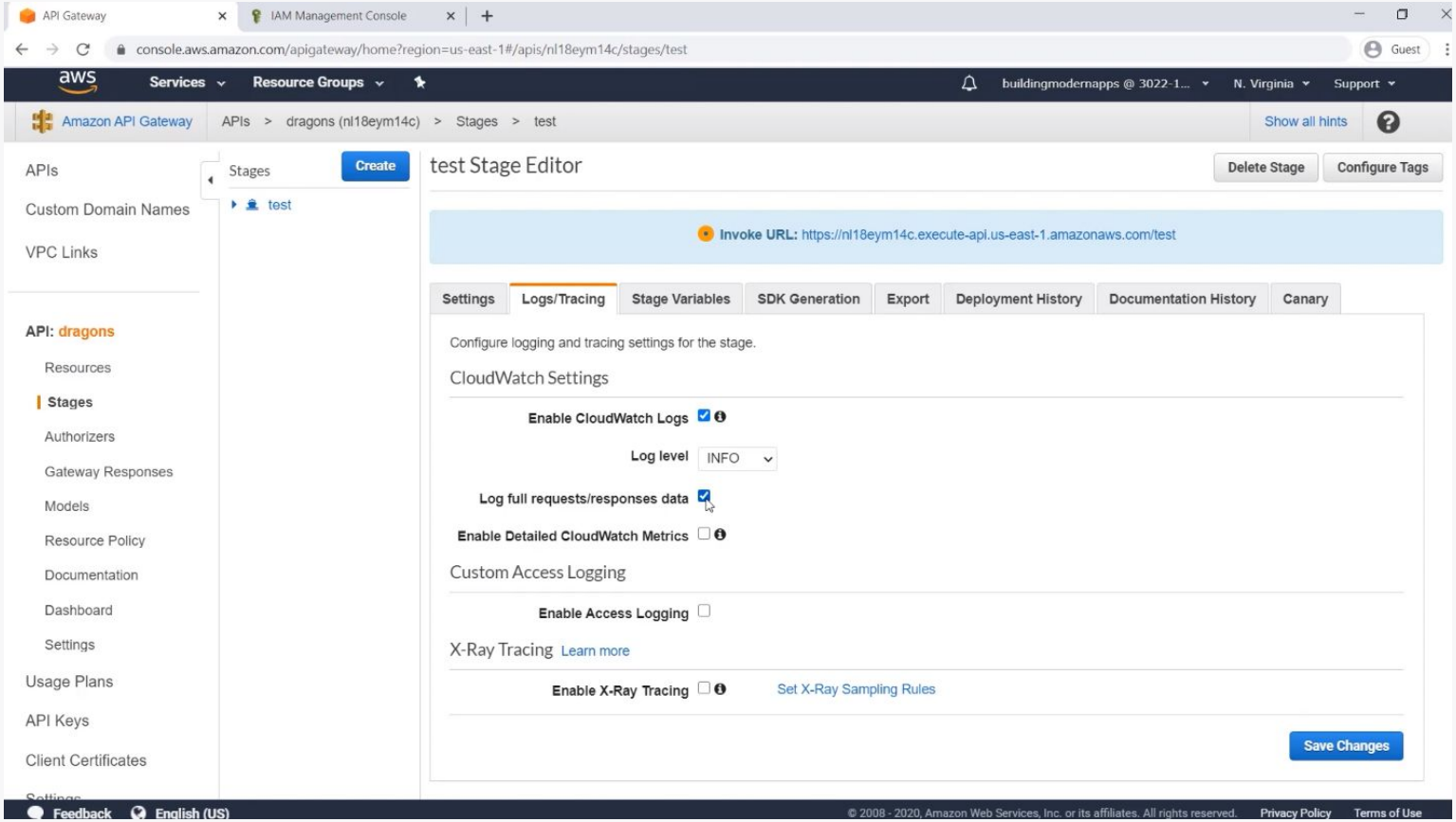
This configuration is done at the Stage level.



in the stage console, enable the CloudWatch logs

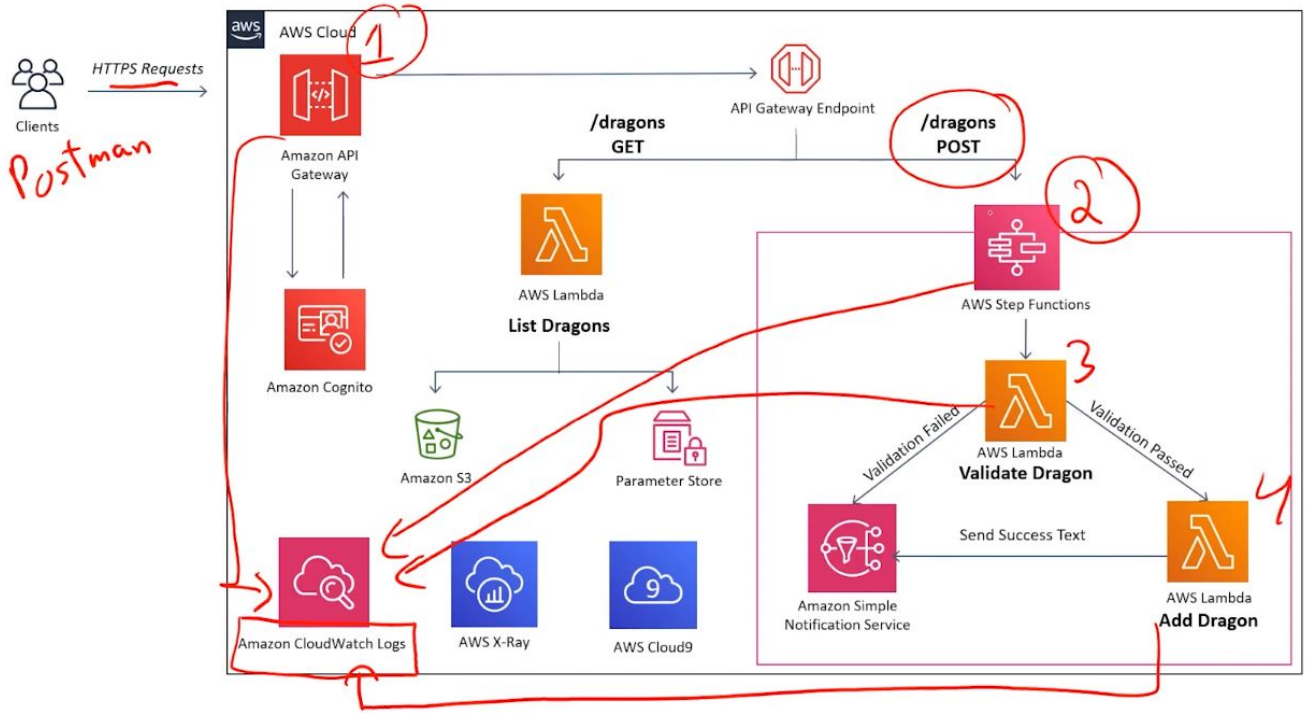


we can select the level of debugging

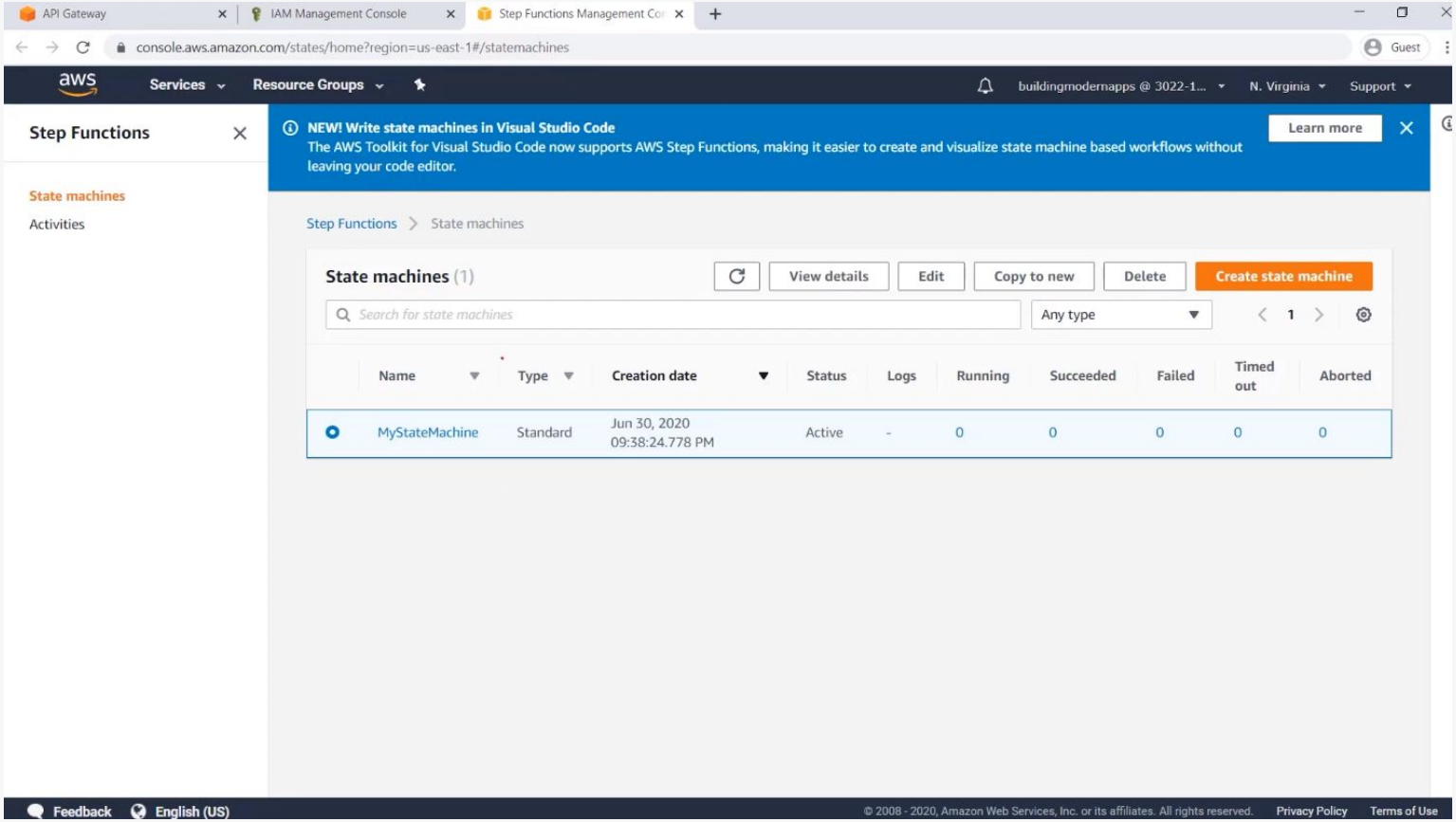


if you want to see the full log of request/response

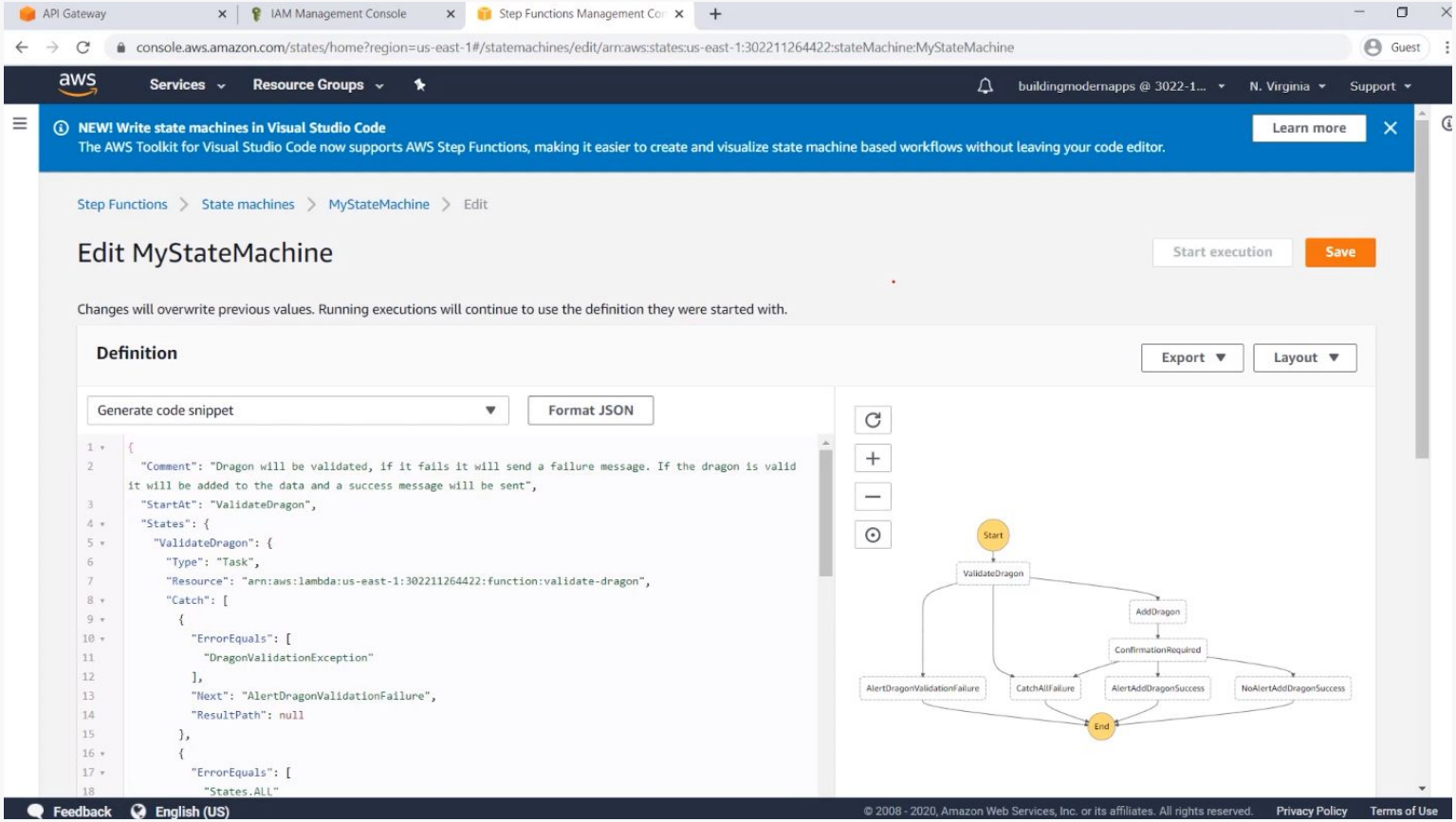
The Dragons API is now configured
to send its execution logs to CloudWatch logs.



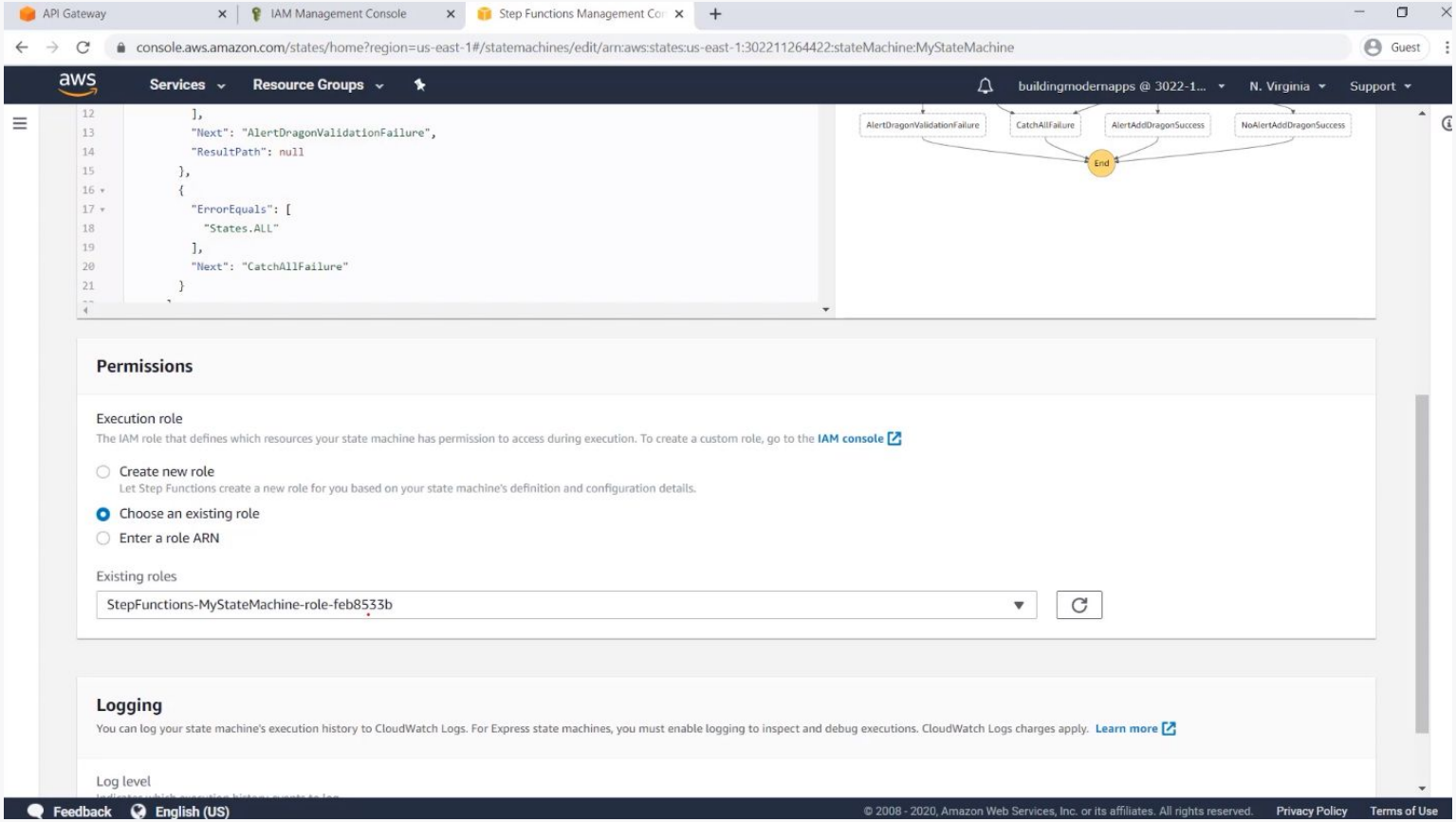
The next step is to configure Step Functions.



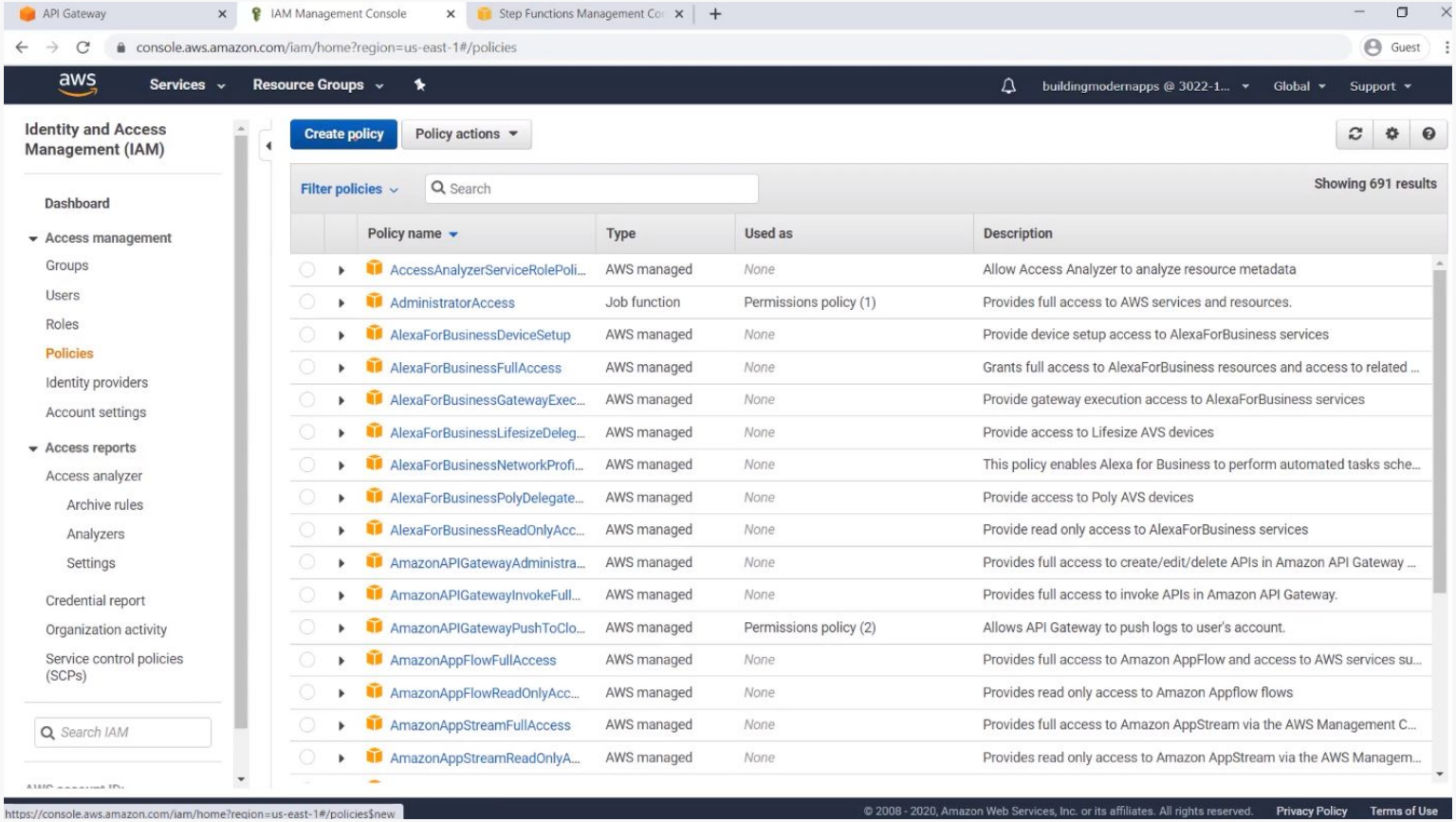
in the Step Function console, select the state machine, and click the Edit



the edit console

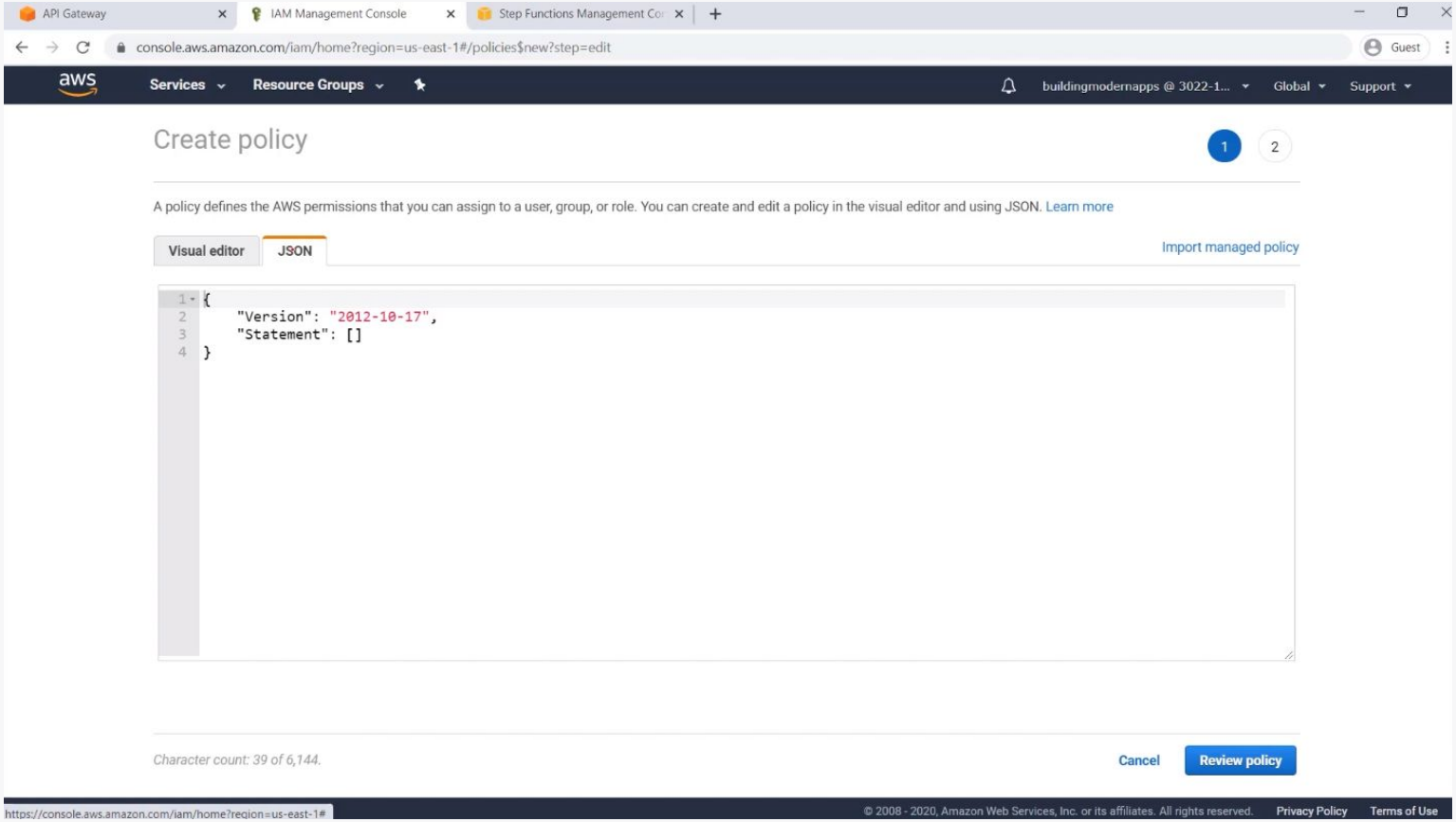


it is already has a role. we need to edit this role to add a policy to allow Step Functions to access the CloudWatch Logs.

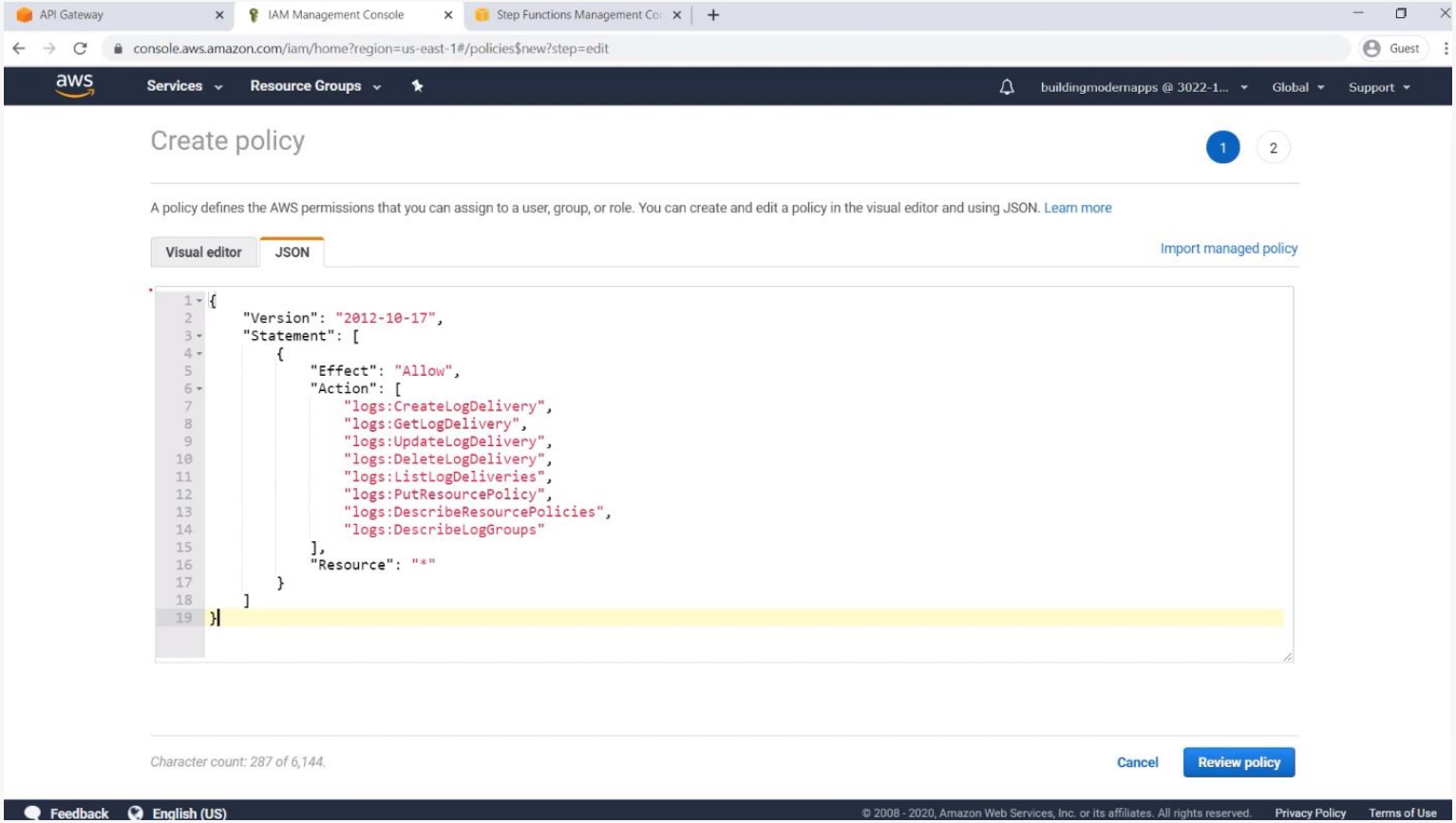


going back to the IAM console, we need to create a brand new IAM policy.

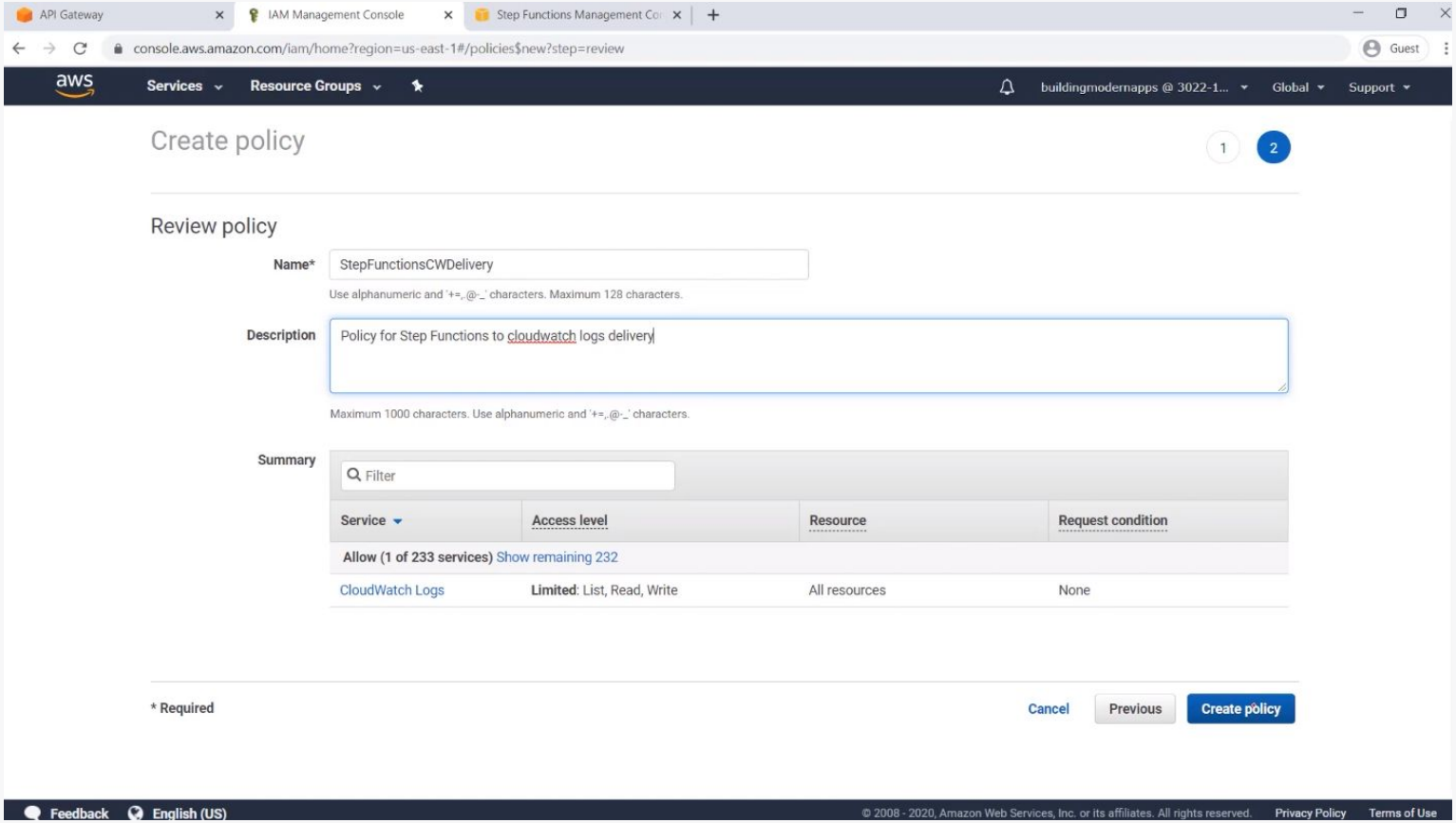
click on create Policy



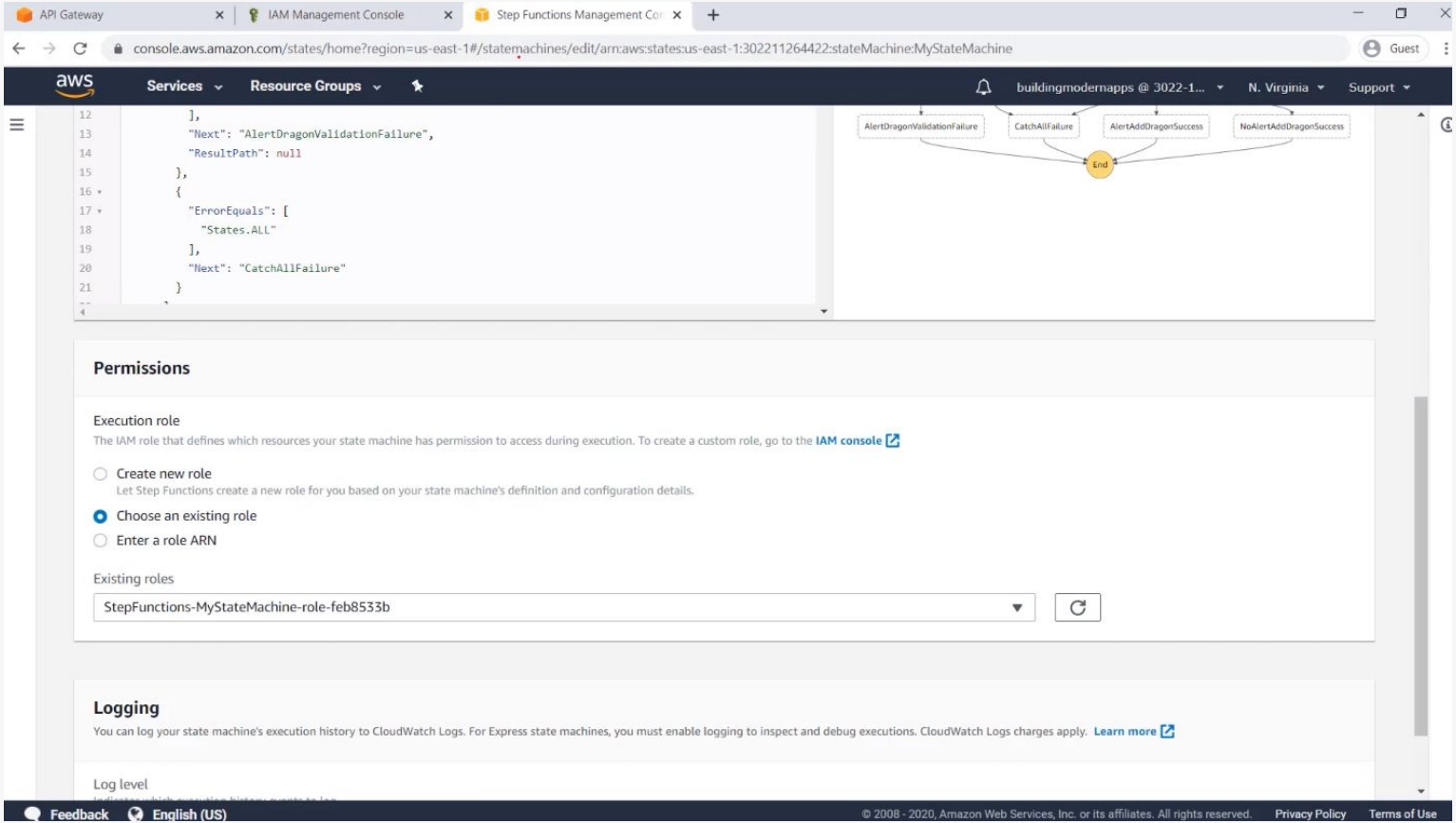
here you can write the policy. you can find the policies in the AWS documentations



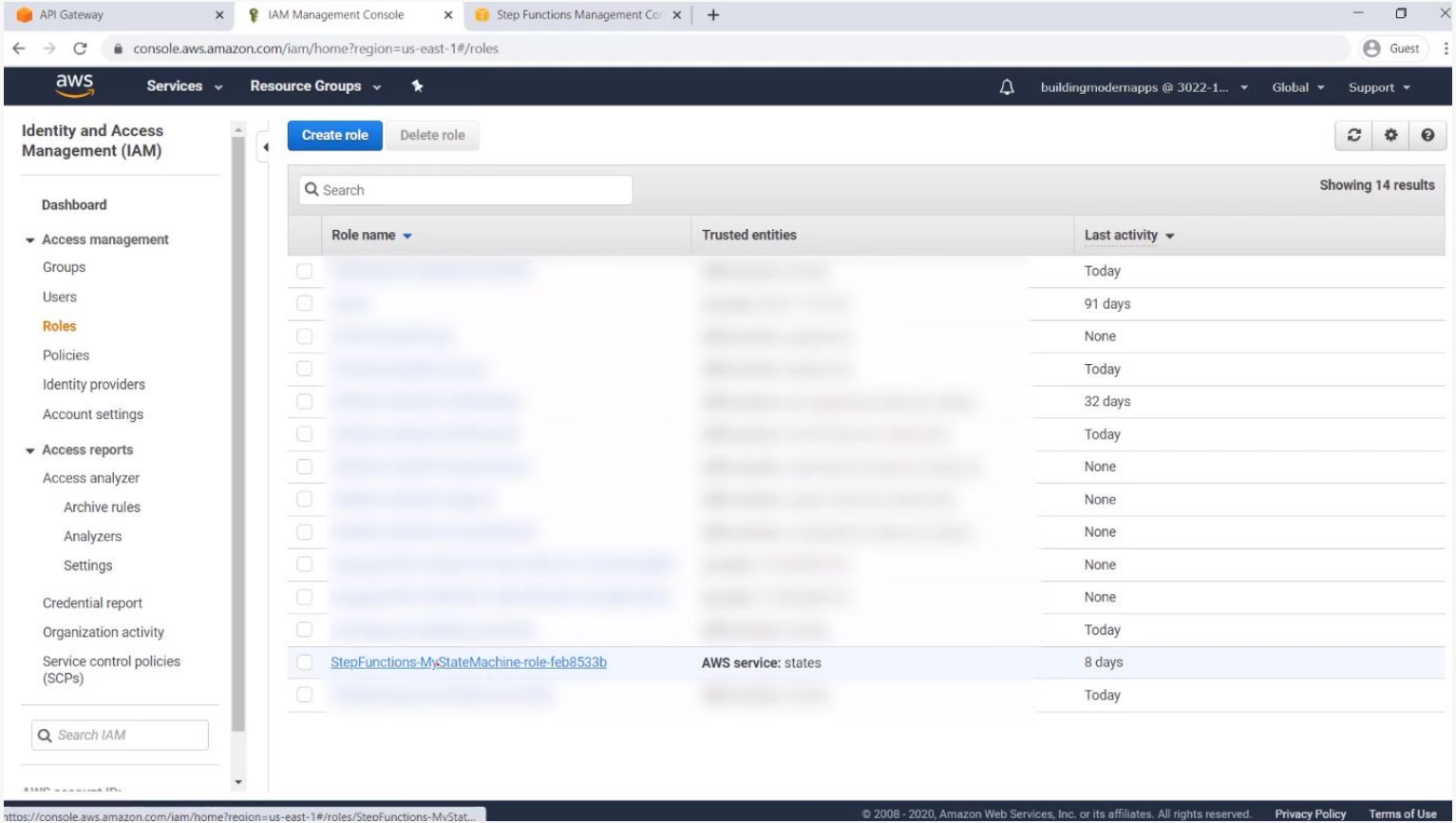
the policy for CloudWatch



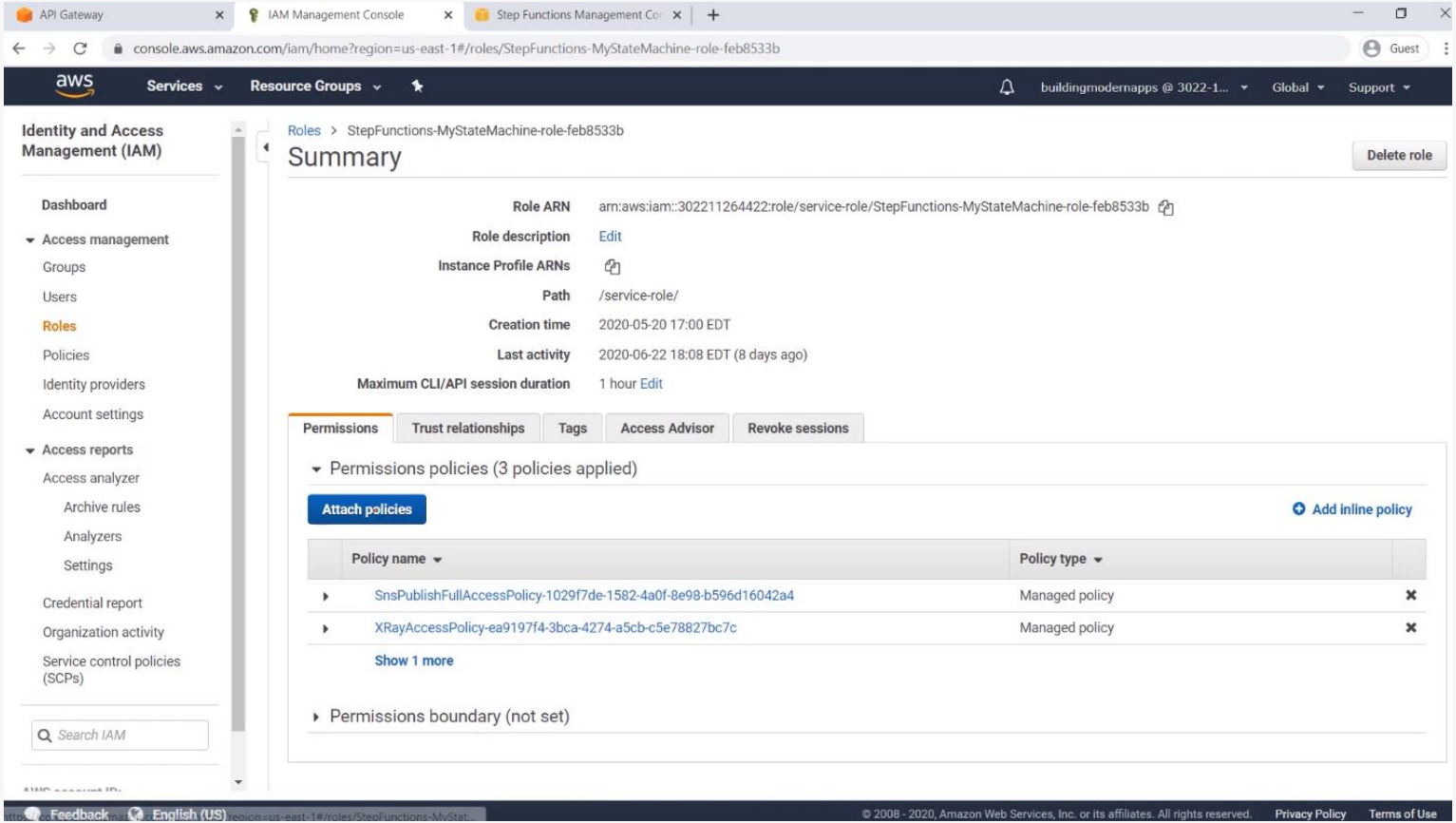
give it a name and create the policy



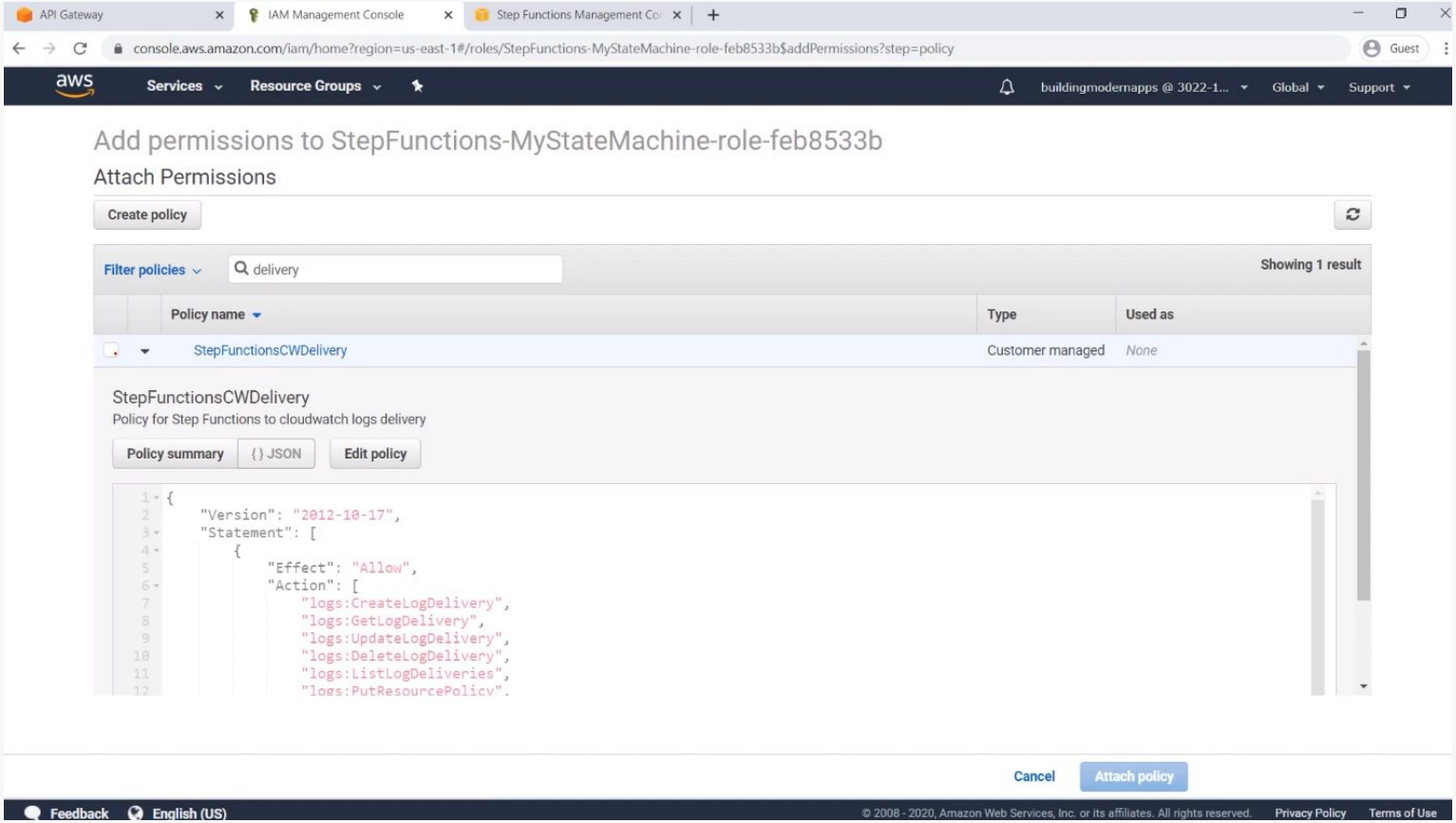
just to check the name of the Step Function’s role that we want to edit its policies



go to the Rules and select that role

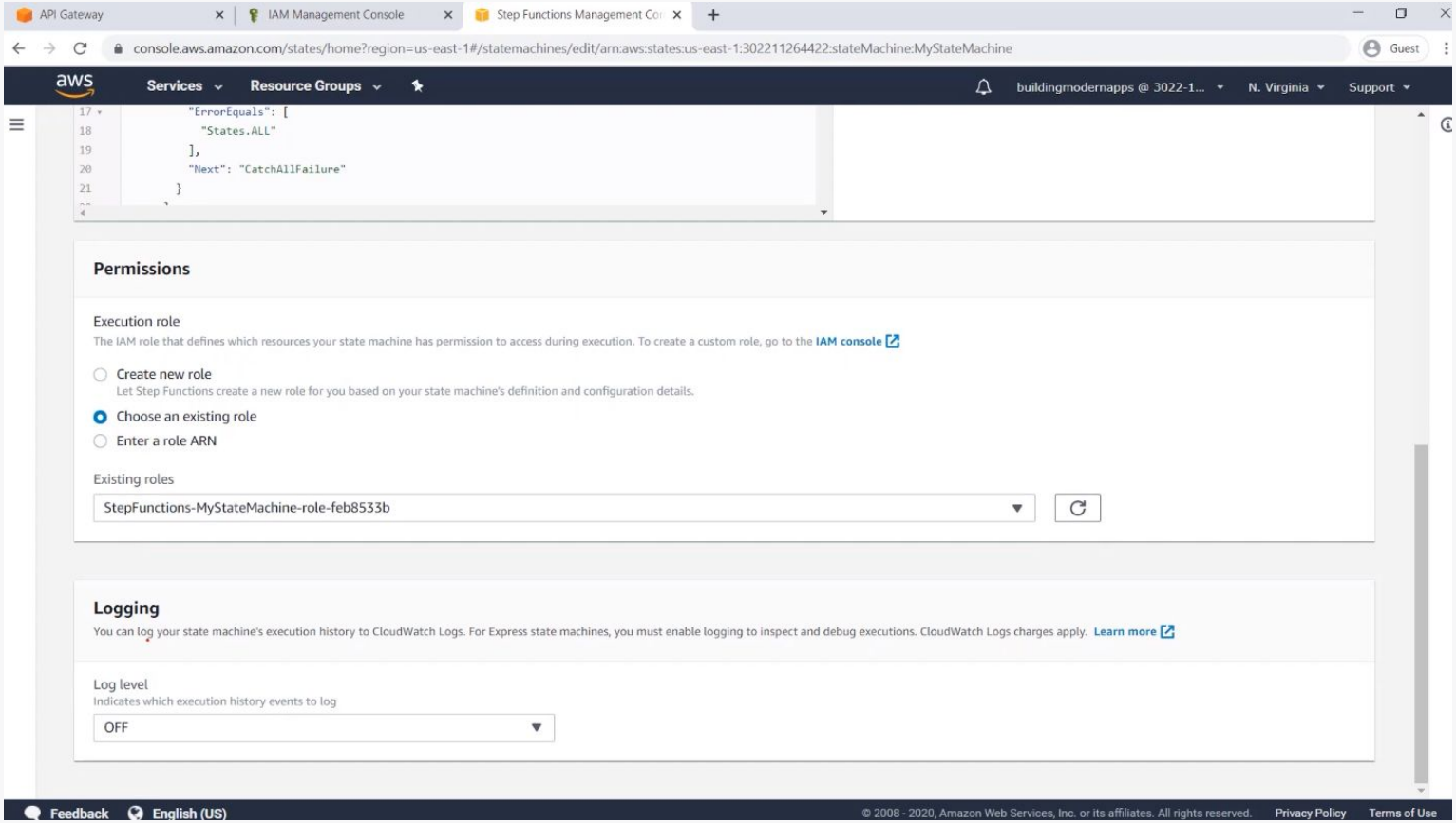


inside that role, click on the attach policy to add a new policy to this role

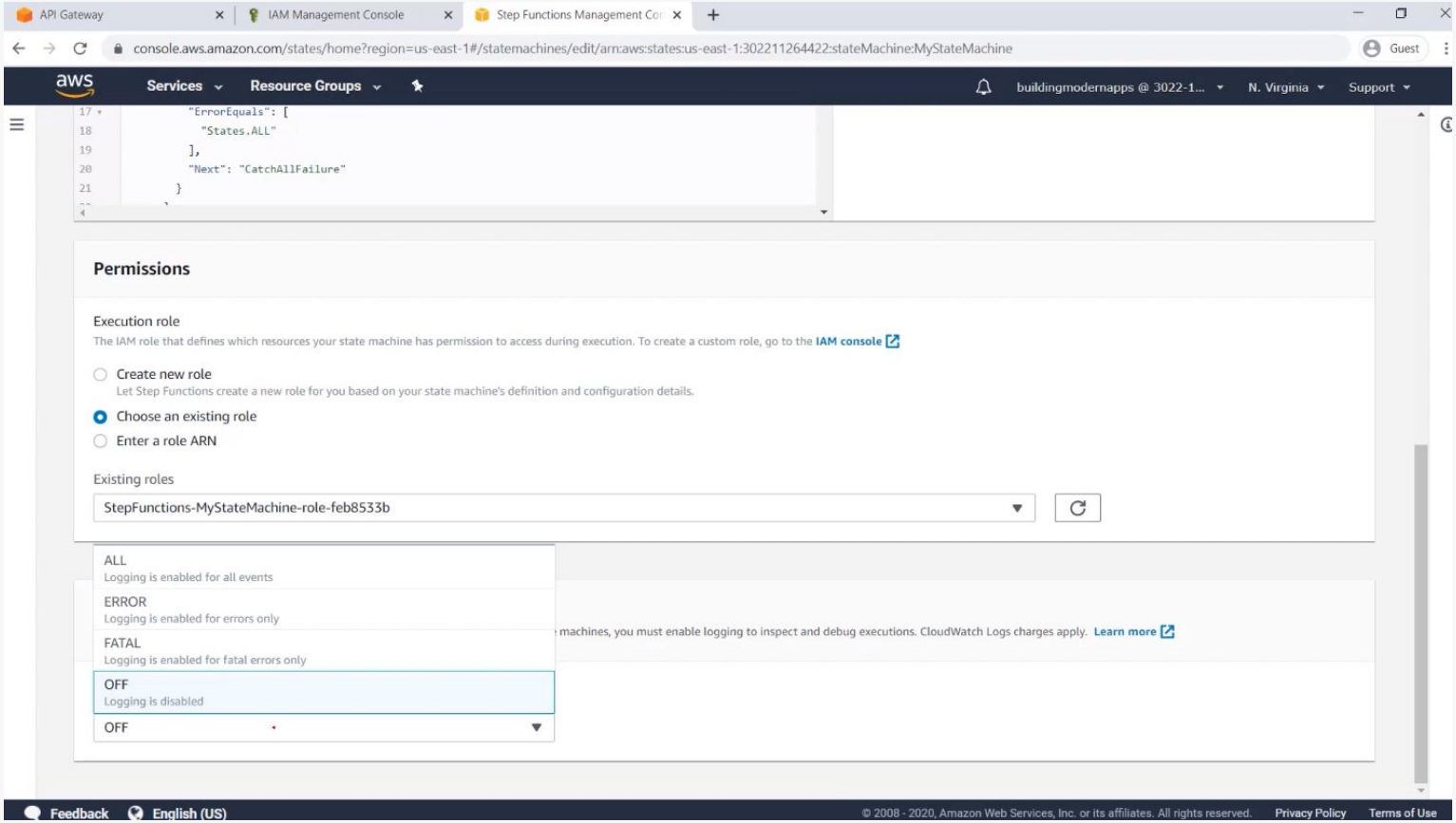


add the new policy that we just created and then click on the attach policy

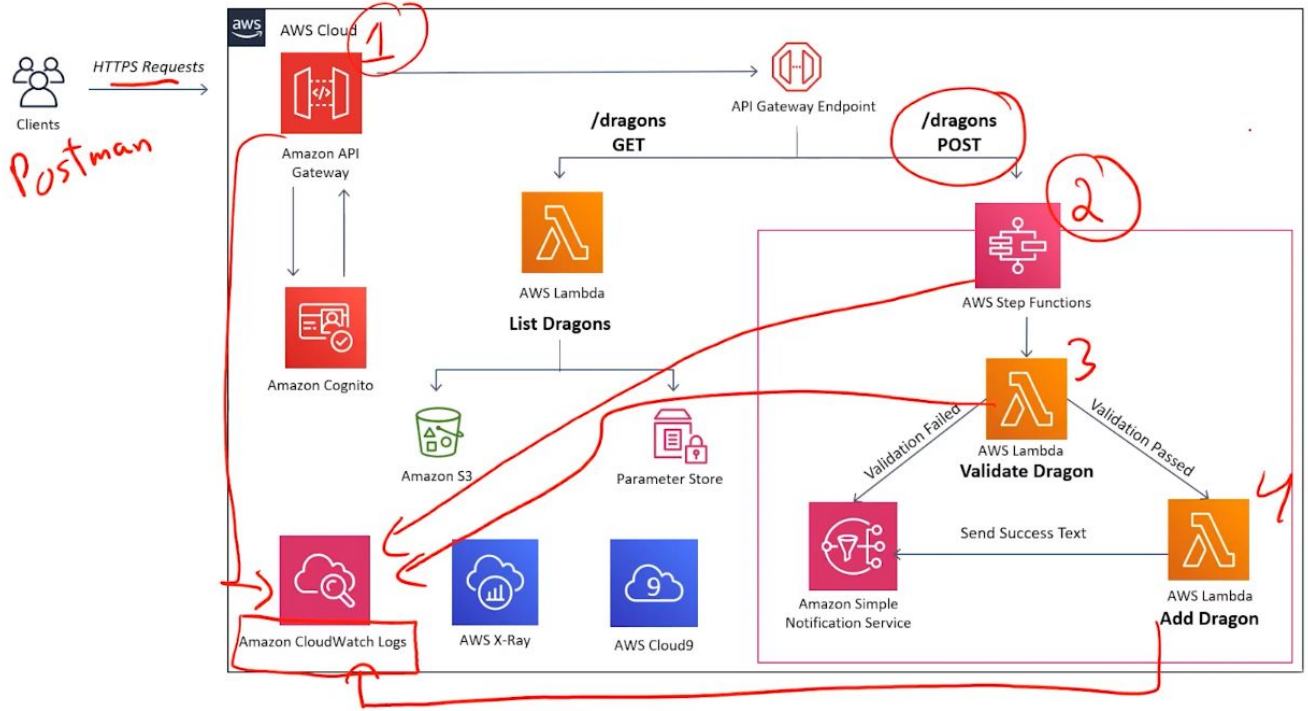
The next step is to configure the state machine to talk to CloudWatch Logs.



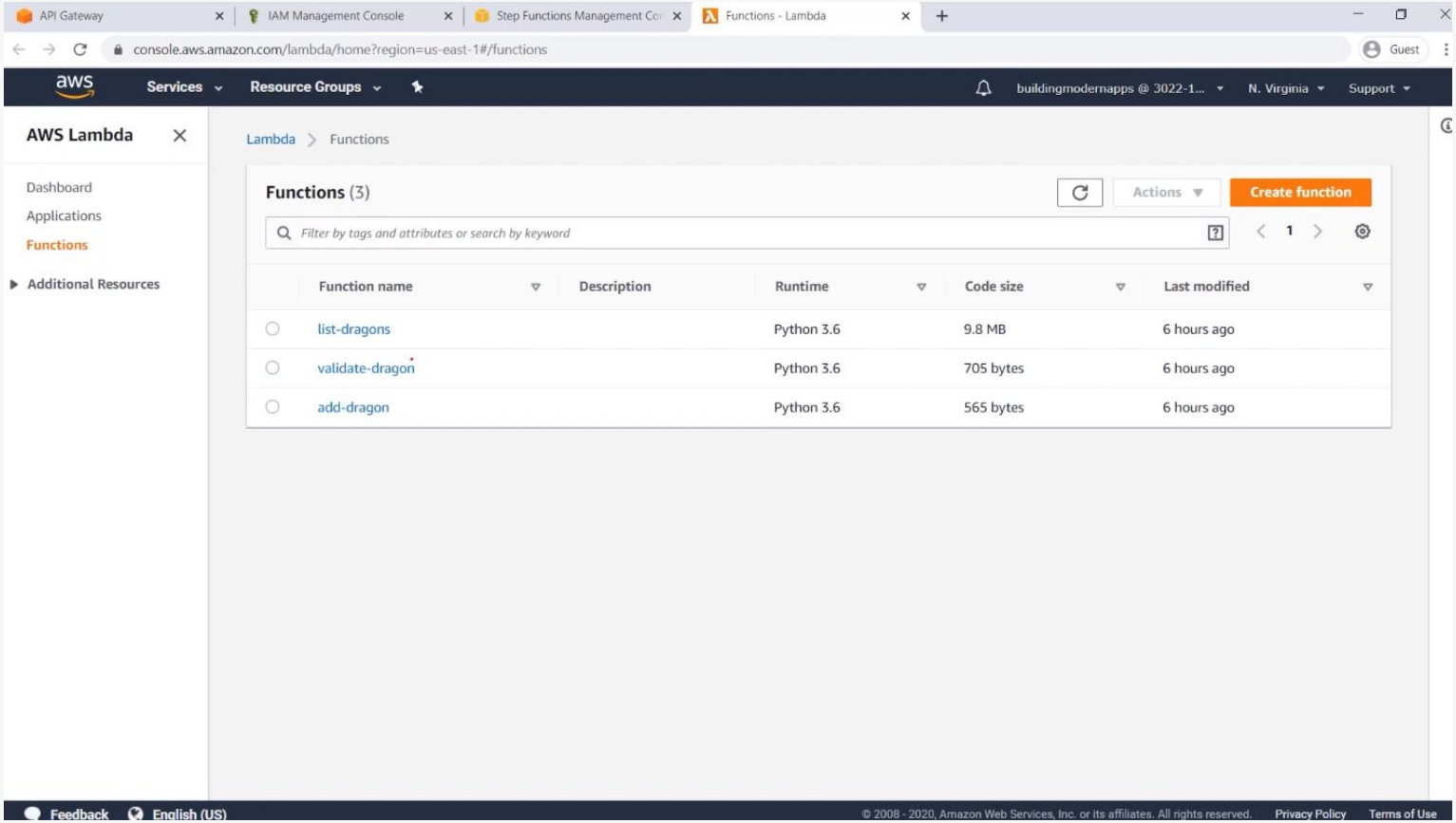
in the Step Function, go the logging



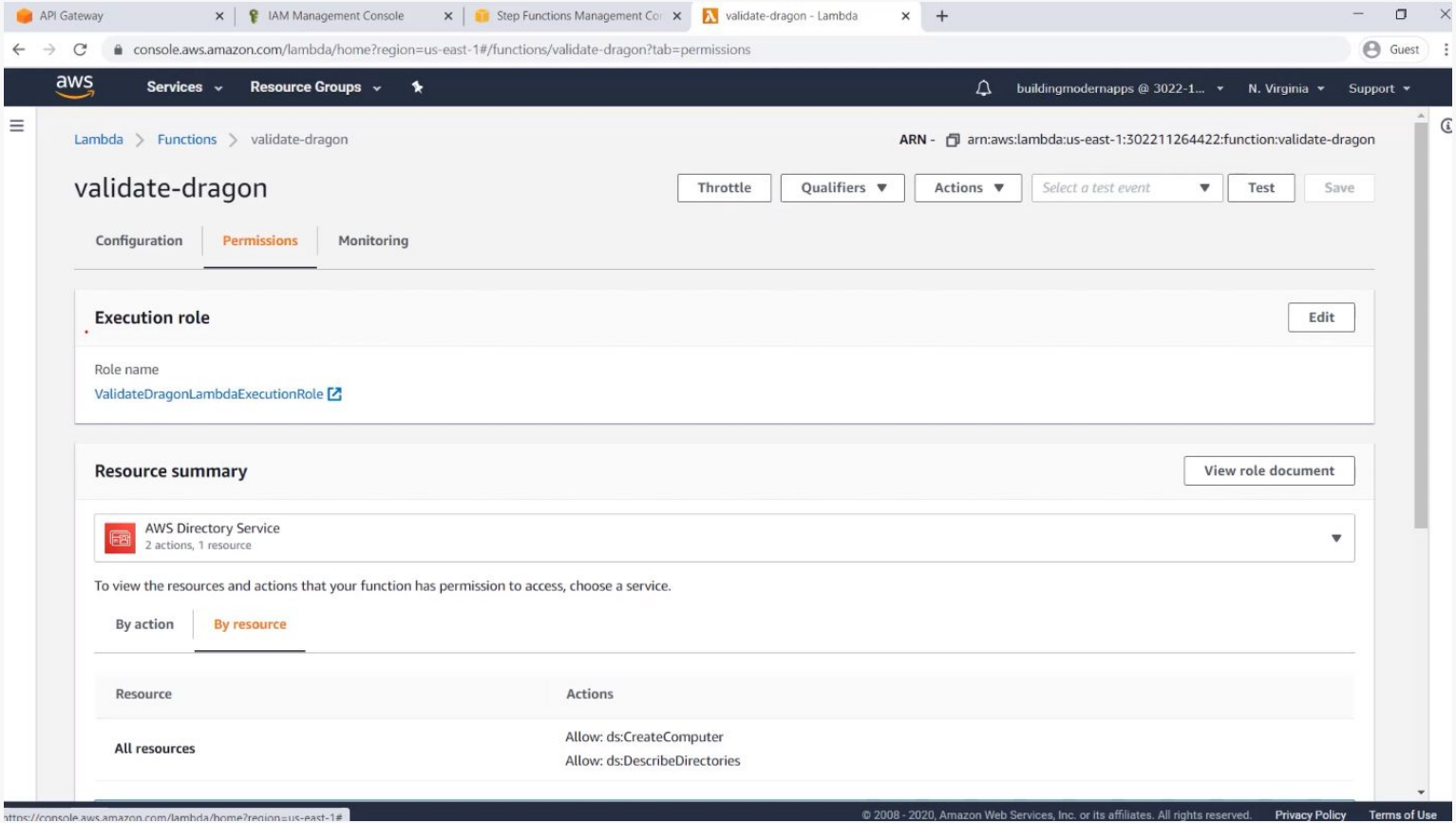
For troubleshooting purposes, we normally select error, but in this case, we select ALL to see the all logs. Save the setting and this step is done.



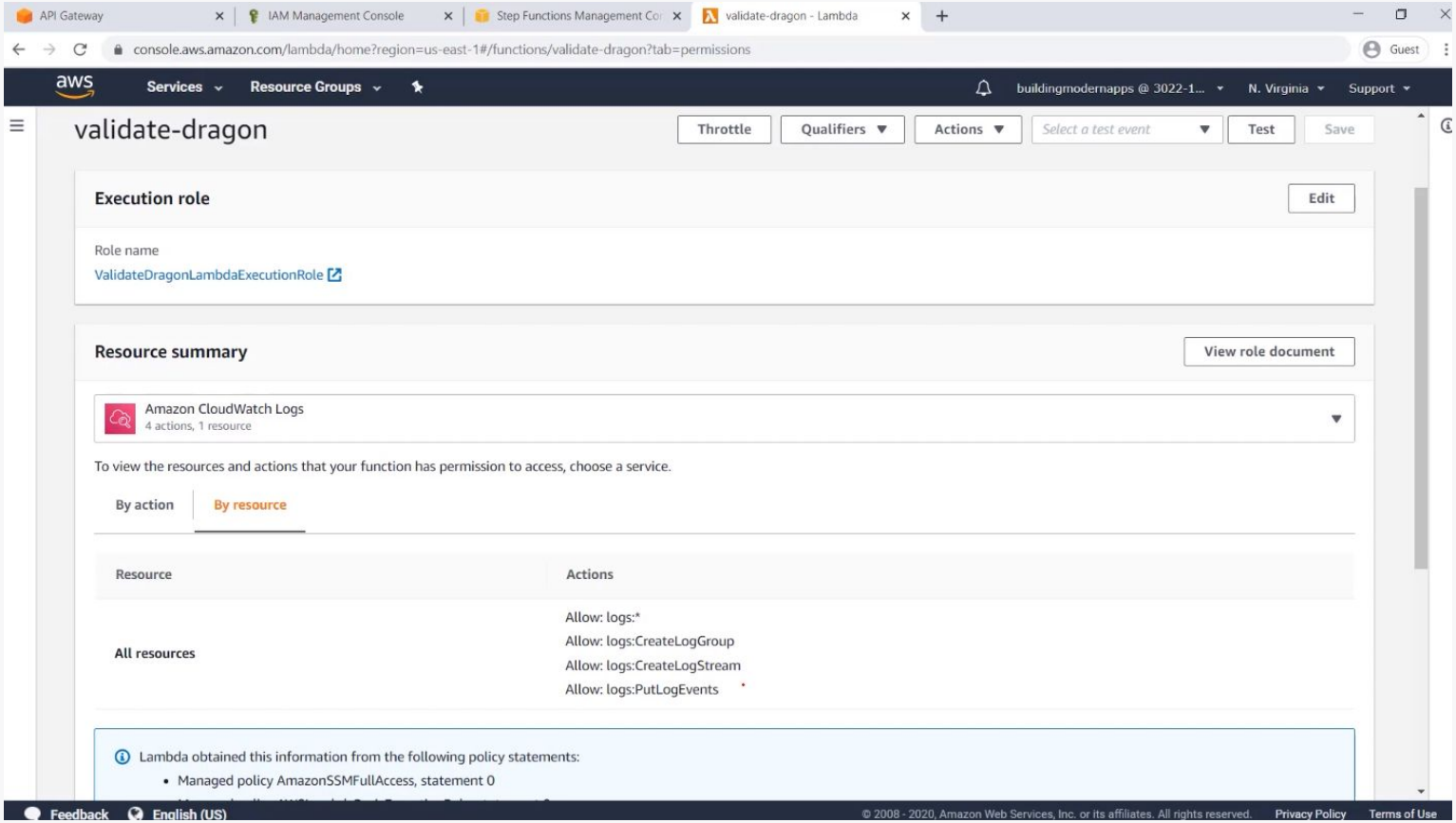
the next step is to configure the Validate Dragon Lambda function



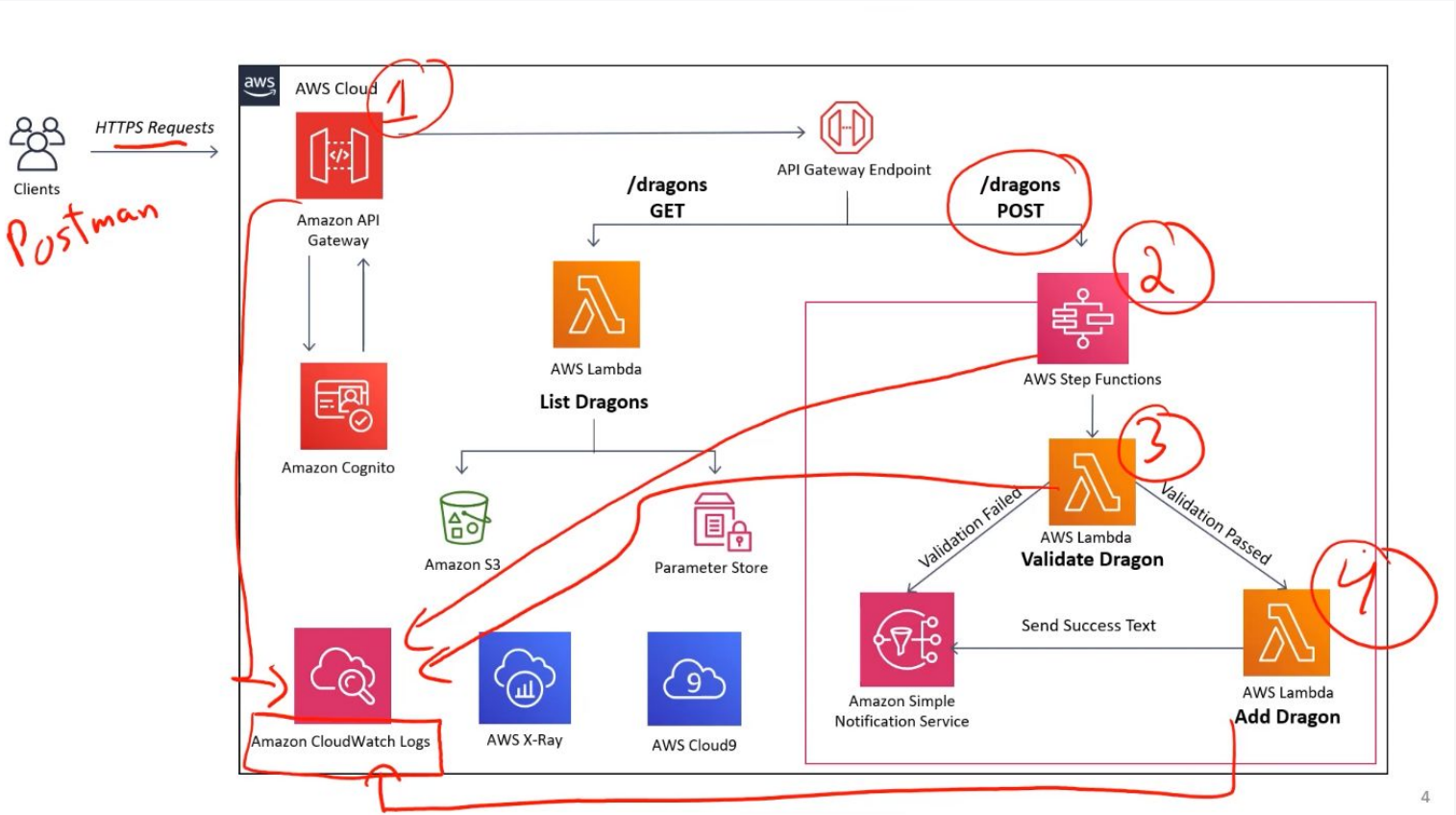
AWS Lambda Functions console



in the Permissions tab, there is already a Role and a Resource Summary

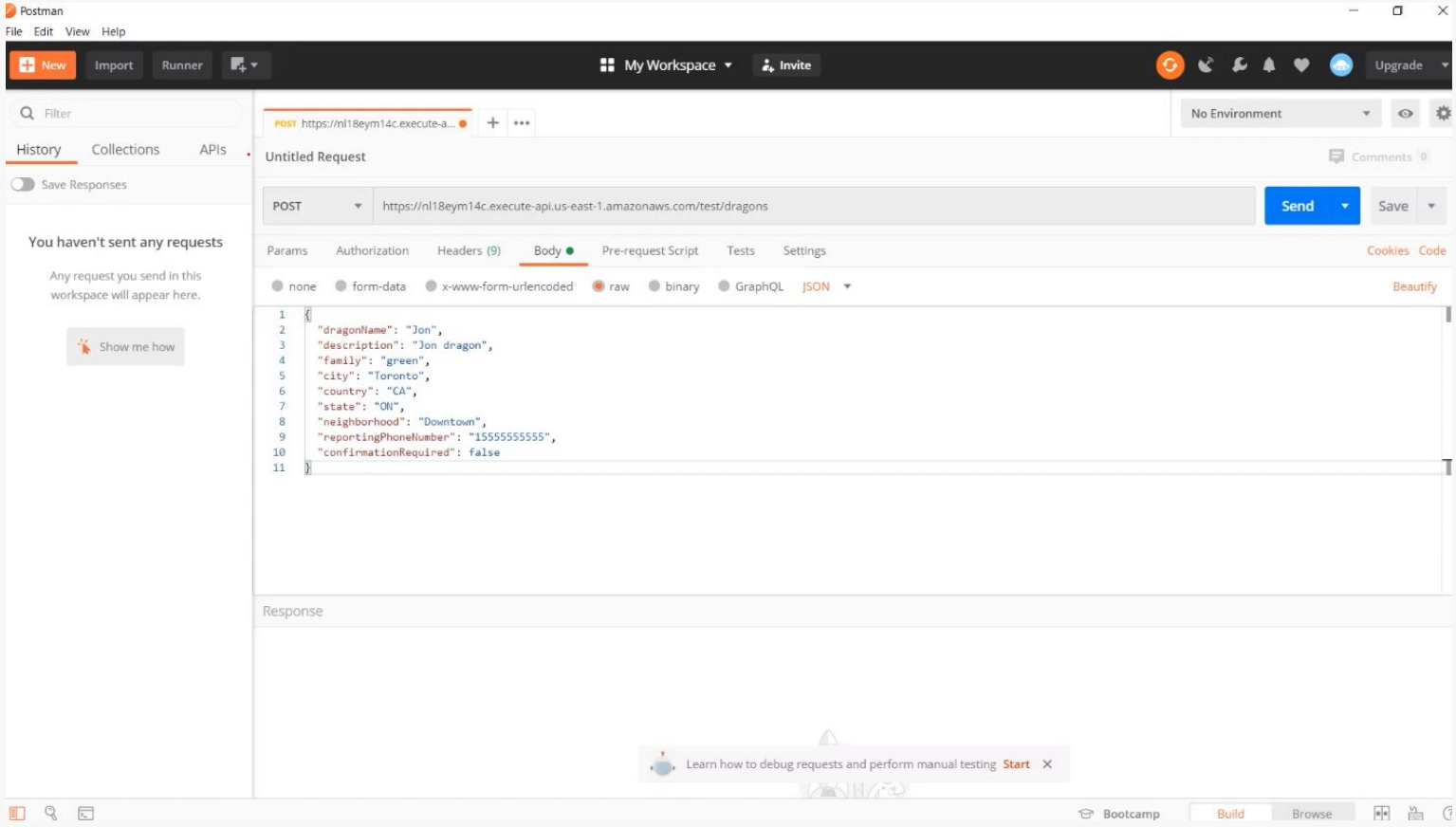


if you select the Amazon CloudWatch Logs, you can see that the cloudwatch is already configured

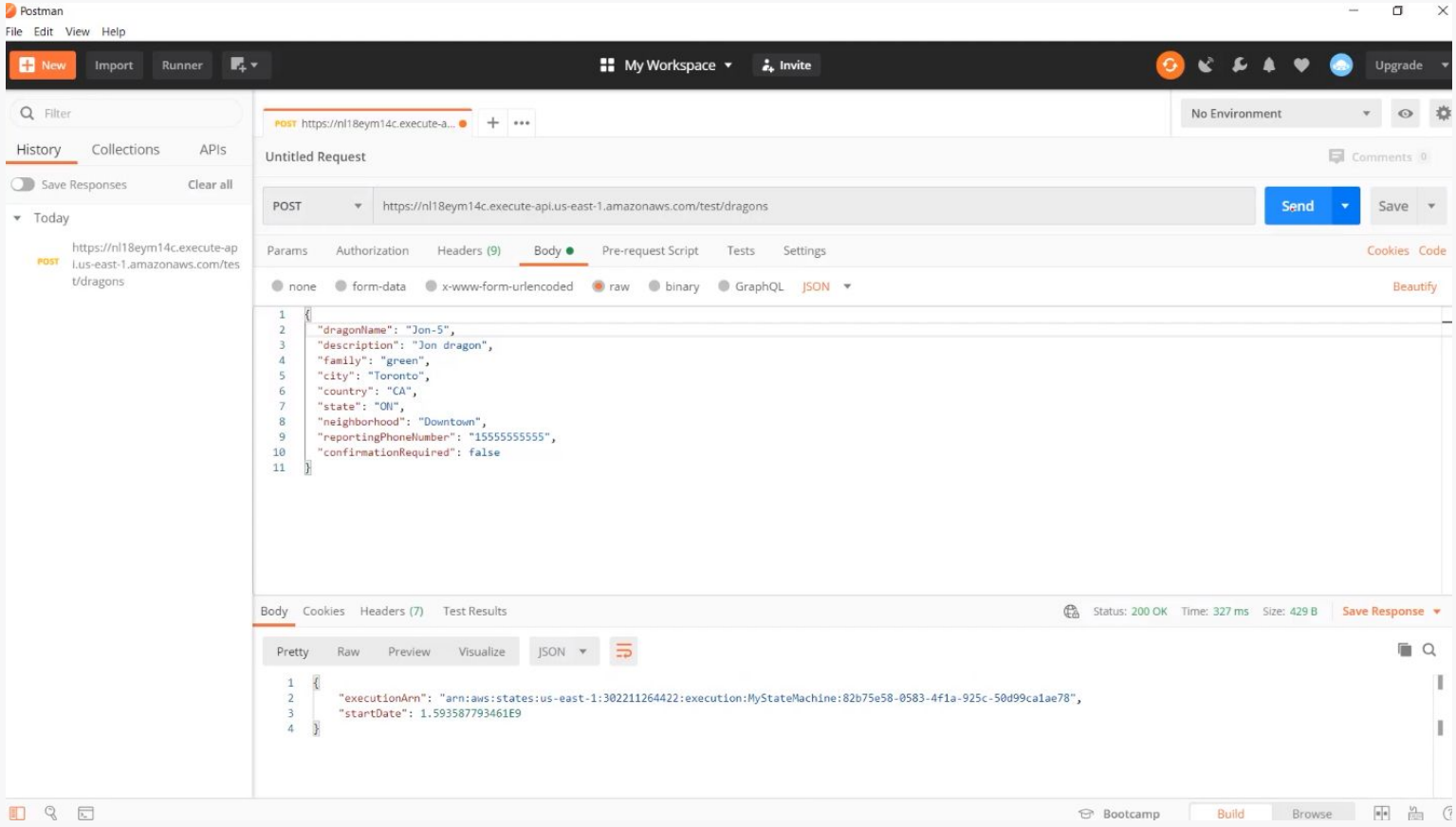


next step is configure the Add Dragon which already is configured.

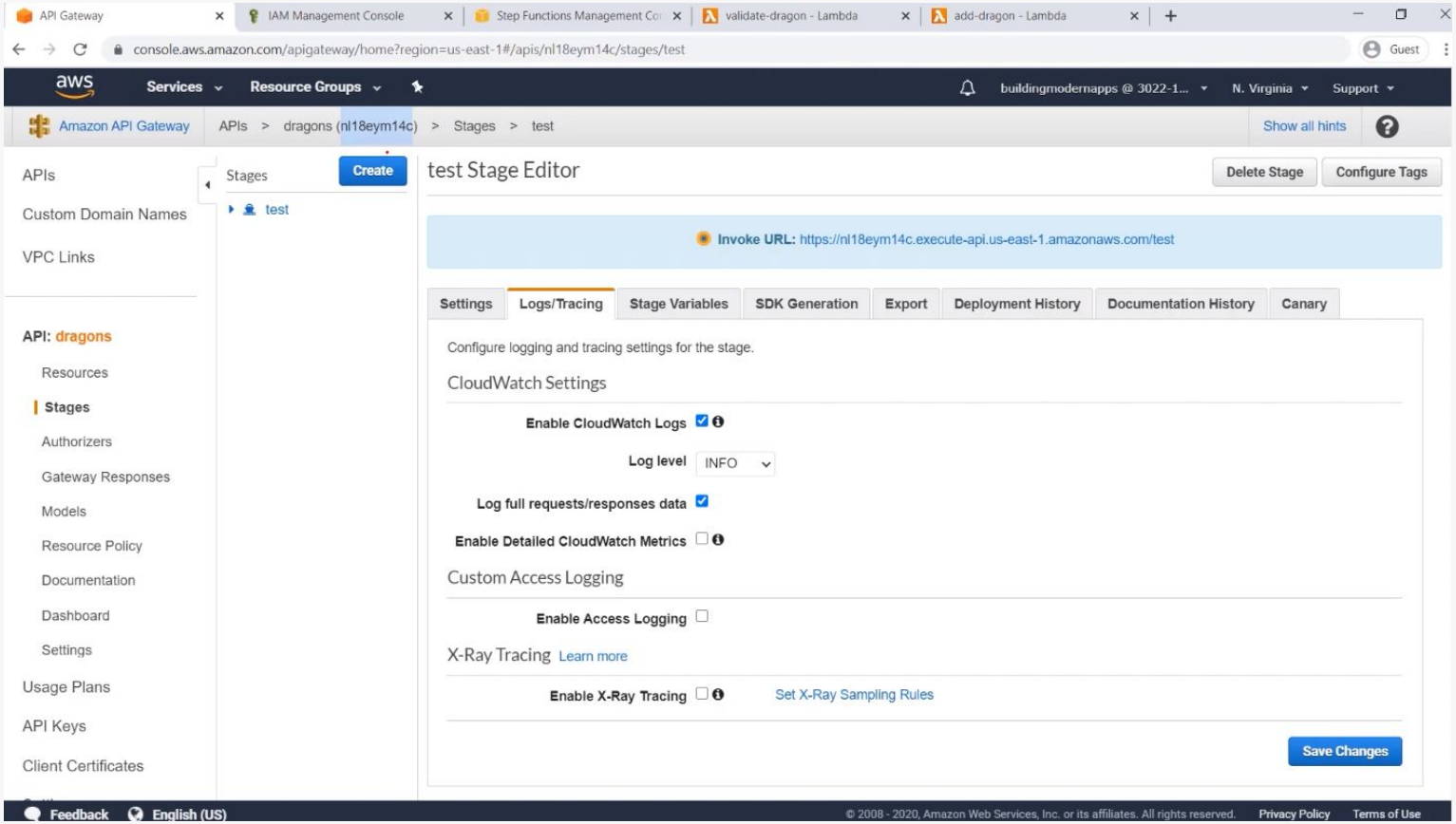
So all that's left now is to use postman to look at the logs.



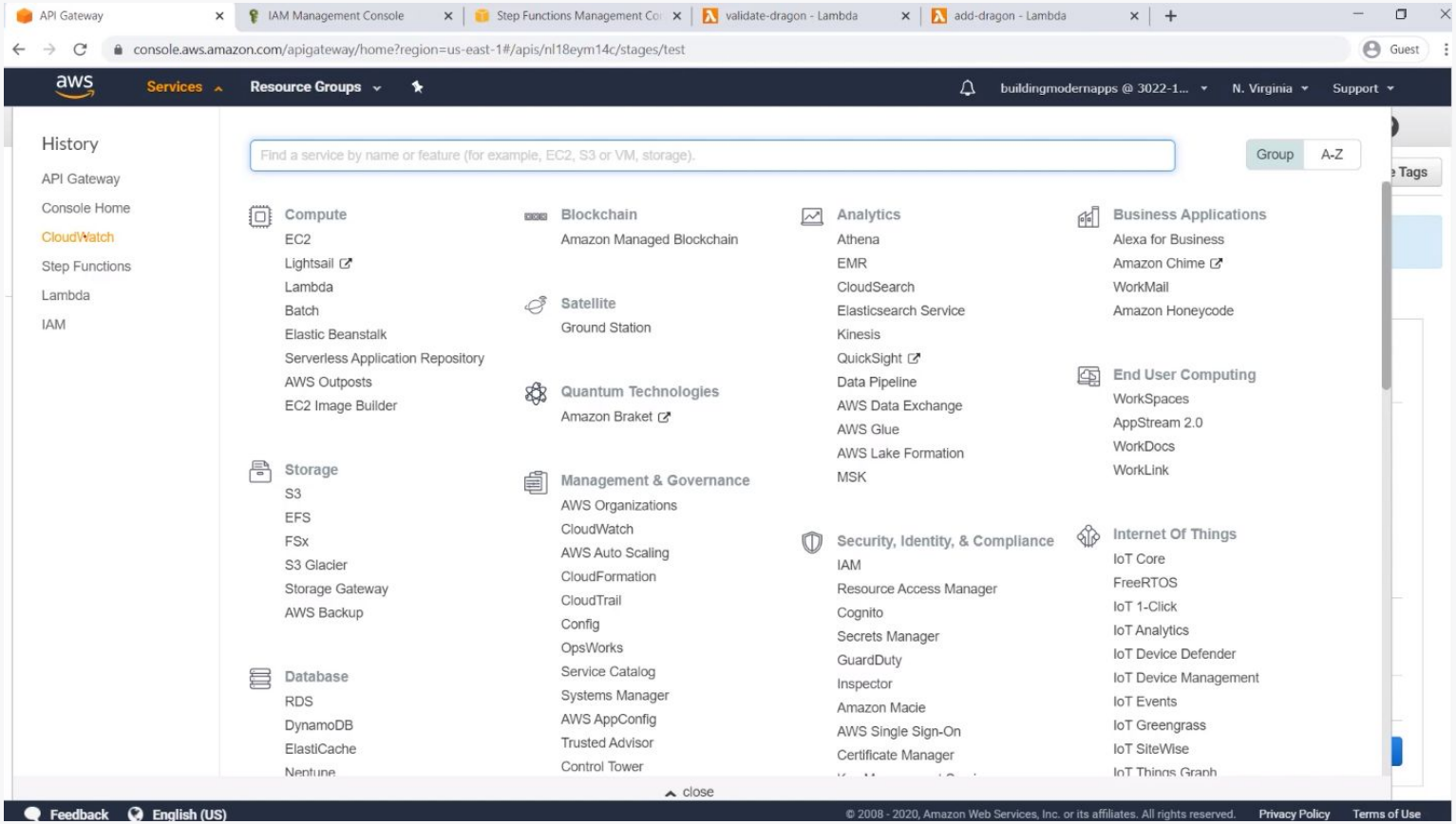
an example of Post Method in the Postman



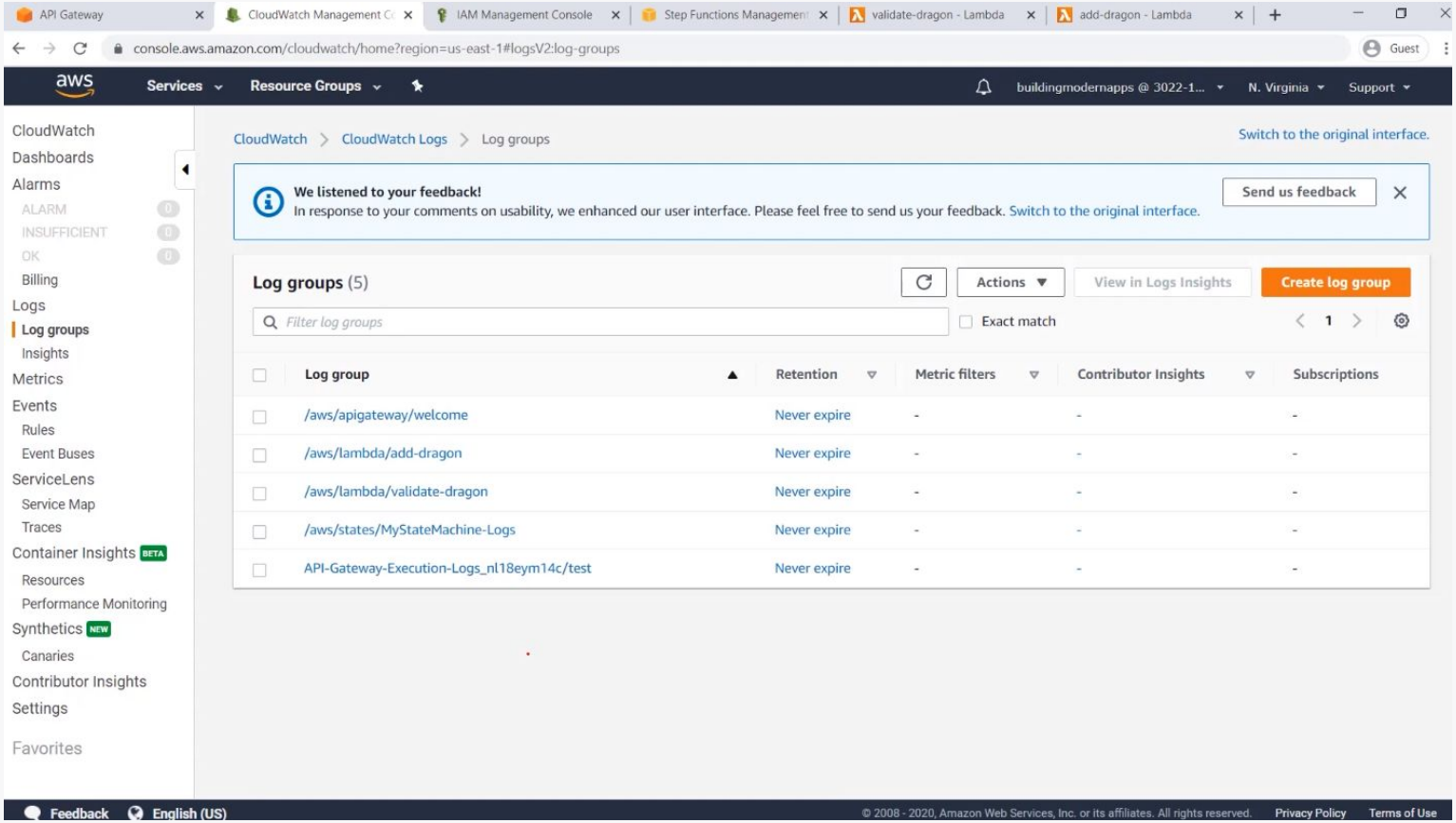
the response is the executionARN which means the step function received the request.



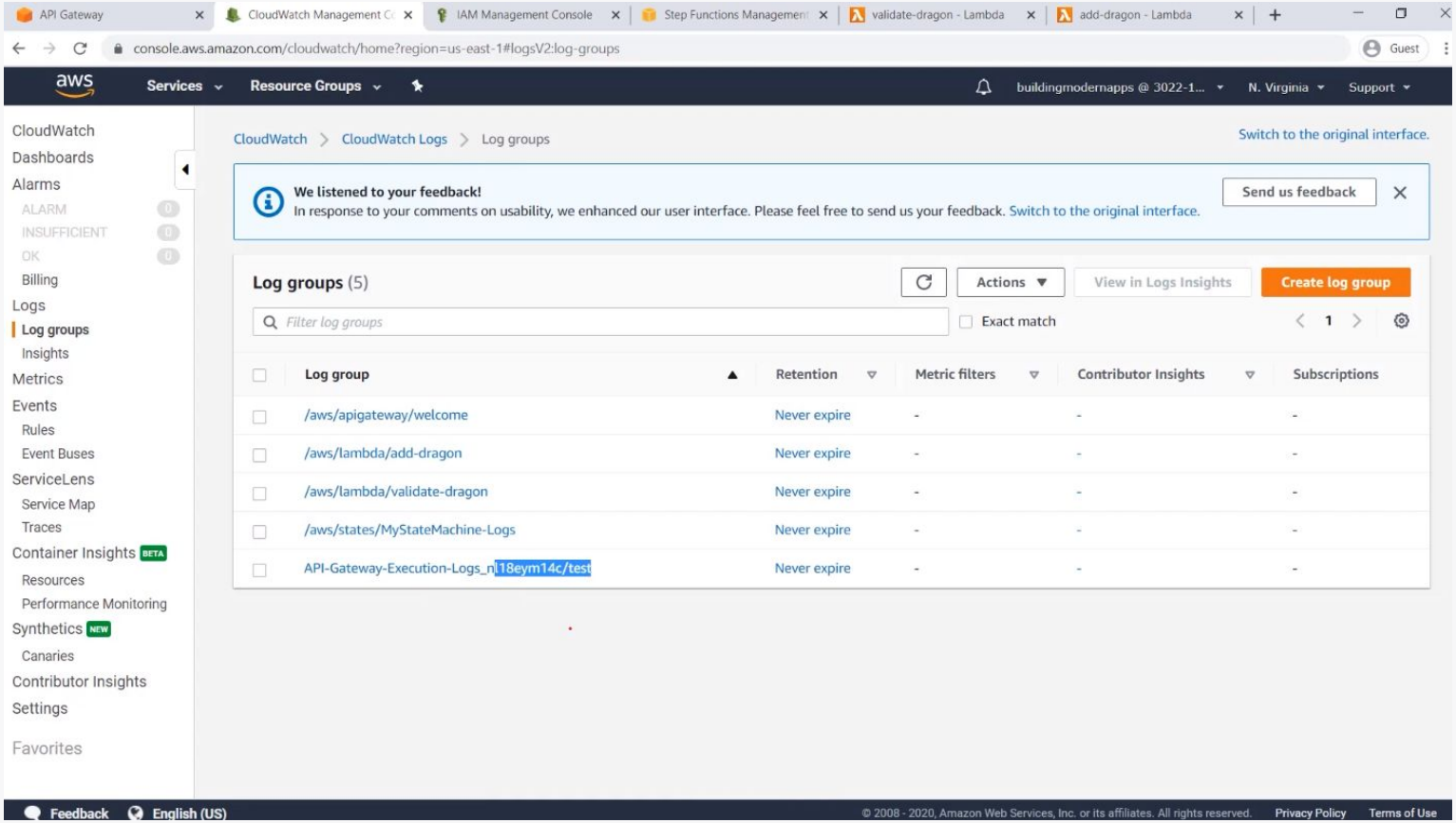
now, lets look at the logs in API Gateway. Just keep the Dragons API id in mind.



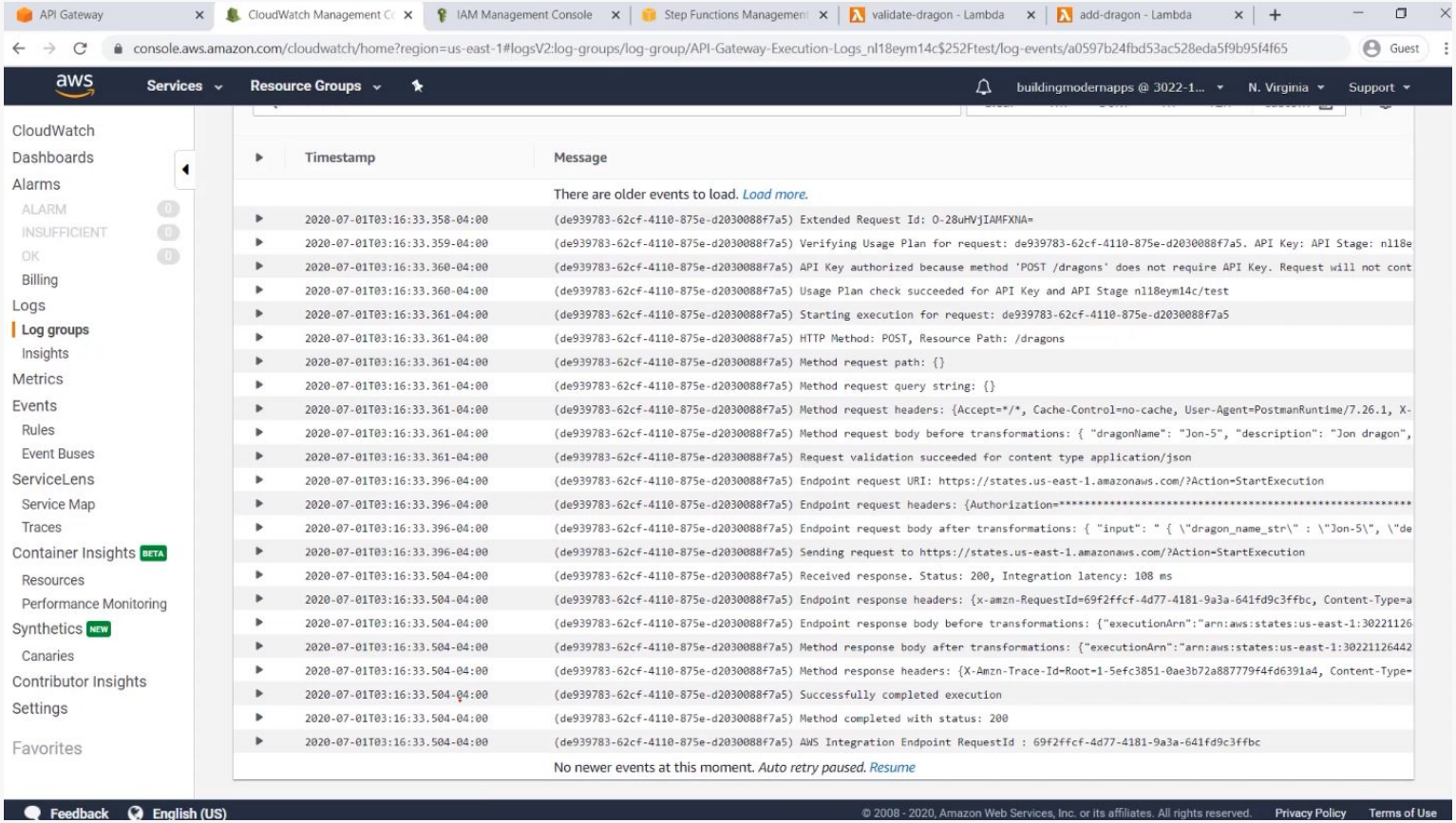
now go to the CloudWatch console



go to the Log Groups



find that log



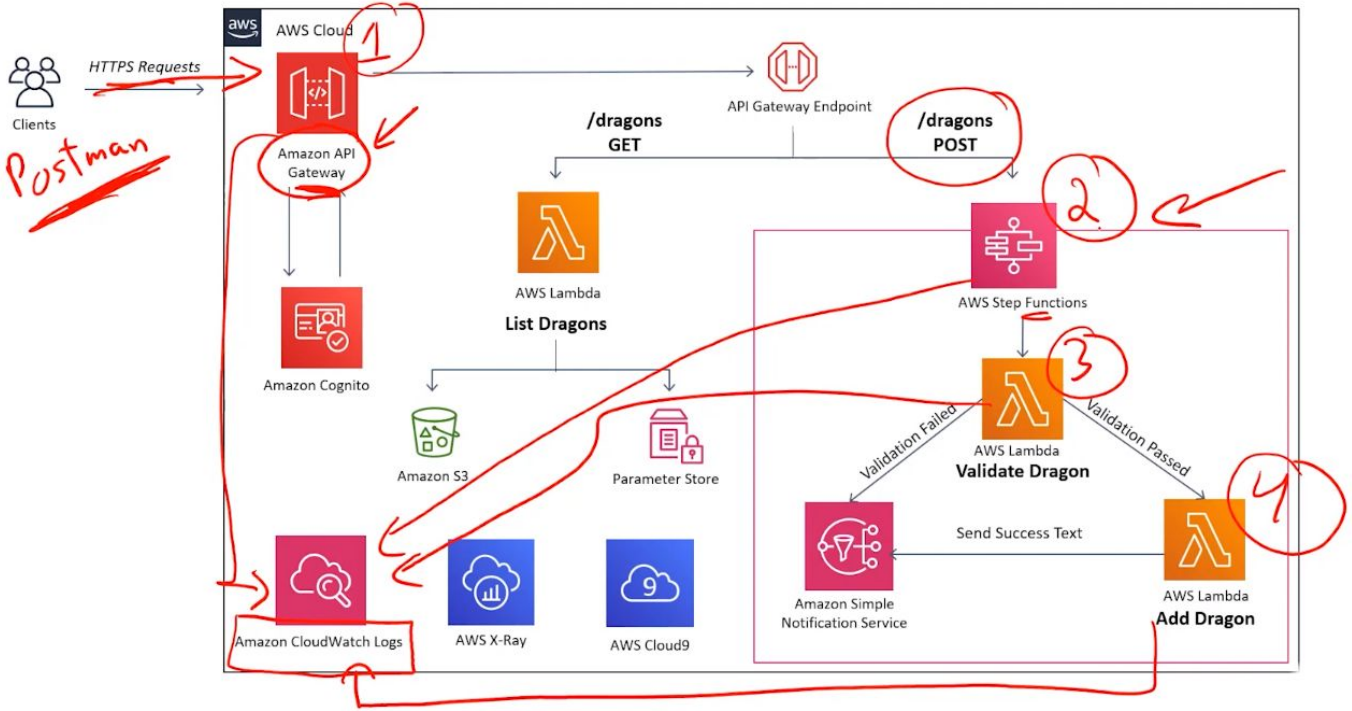
we can see all logs

	Message
	There are older events to load. *Load more.*
58-04:00	(de939783-62cf-4110-875e-d2030088f7a5) Extended Request Id: O-28uHVjIAMFXNA=
59-04:00	(de939783-62cf-4110-875e-d2030088f7a5) Verifying Usage Plan for request: de939783-62cf-4110-875e-d2030088f7a5. API Key: API St
60-04:00	(de939783-62cf-4110-875e-d2030088f7a5) API Key authorized because method 'POST /dragons' does not require API Key. Request wil
60-04:00	(de939783-62cf-4110-875e-d2030088f7a5) Usage Plan check succeeded for API Key and API Stage nl18eym14c/test
61-04:00	(de939783-62cf-4110-875e-d2030088f7a5) Starting execution for request: de939783-62cf-4110-875e-d2030088f7a5
61-04:00	(de939783-62cf-4110-875e-d2030088f7a5) HTTP Method: POST, Resource Path: /dragons
61-04:00	(de939783-62cf-4110-875e-d2030088f7a5) Method request path: {}
61-04:00	(de939783-62cf-4110-875e-d2030088f7a5) Method request query string: {}
61-04:00	(de939783-62cf-4110-875e-d2030088f7a5) Method request headers: {Accept=*/*, Cache-Control=no-cache, User-Agent=PostmanRuntime/

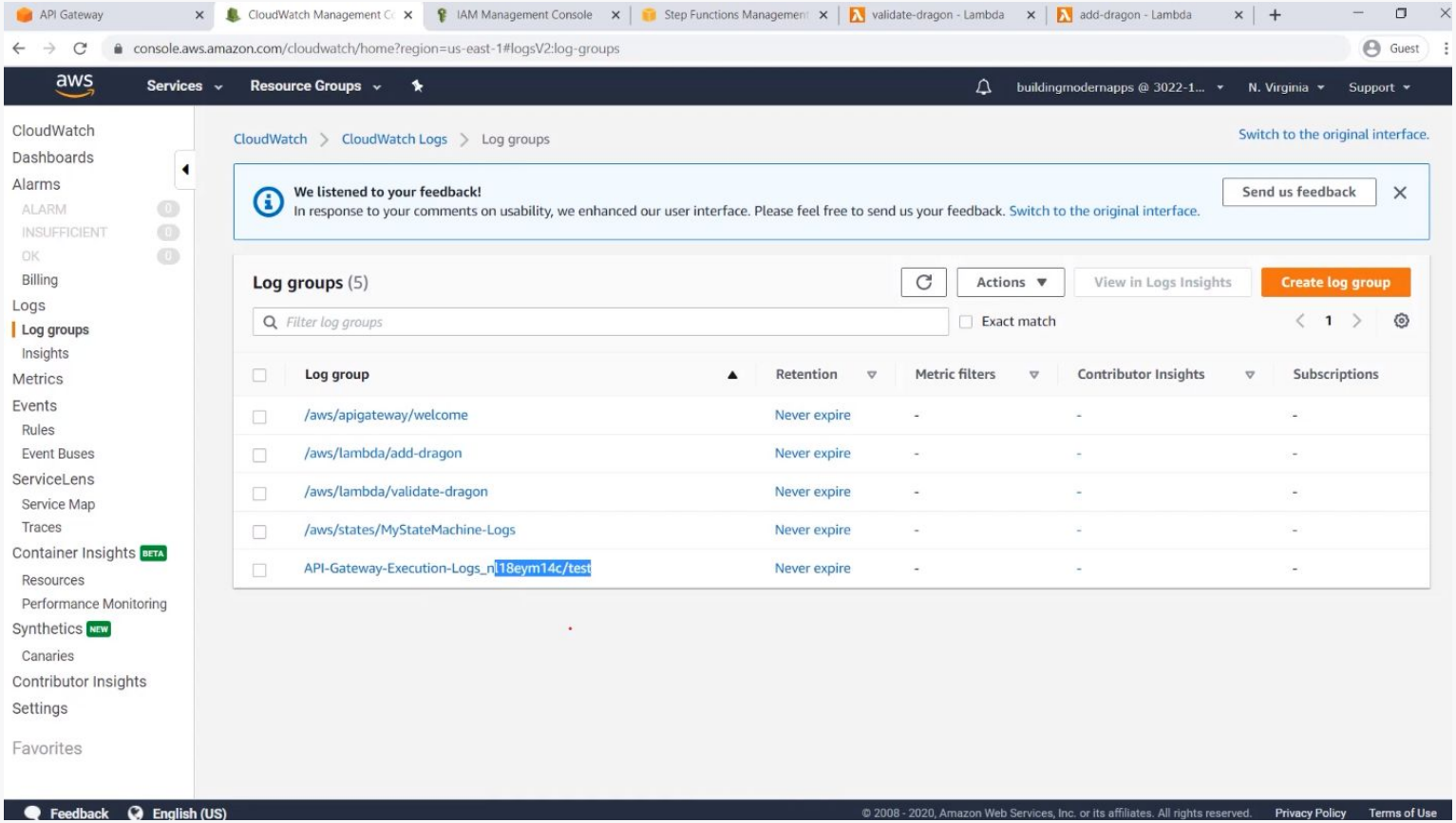
61-04:00	(de939783-62cf-4110-875e-d2030088f7a5) Method request body before transformations:
	{
	"dragonName": "Jon-5",
	"description": "Jon dragon",
	"family": "green",
	"city": "Toronto",
	"country": "CA",
	"state": "ON",
	"neighborhood": "Downtown",
	"reportingPhoneNumber": "15555555555",
	"confirmationRequired": false
	}

61-04:00	(de939783-62cf-4110-875e-d2030088f7a5) Request validation succeeded for content type application/json
96-04:00	(de939783-62cf-4110-875e-d2030088f7a5) Endpoint request URI: https://states.us-east-1.amazonaws.com/?Action=StartExecution

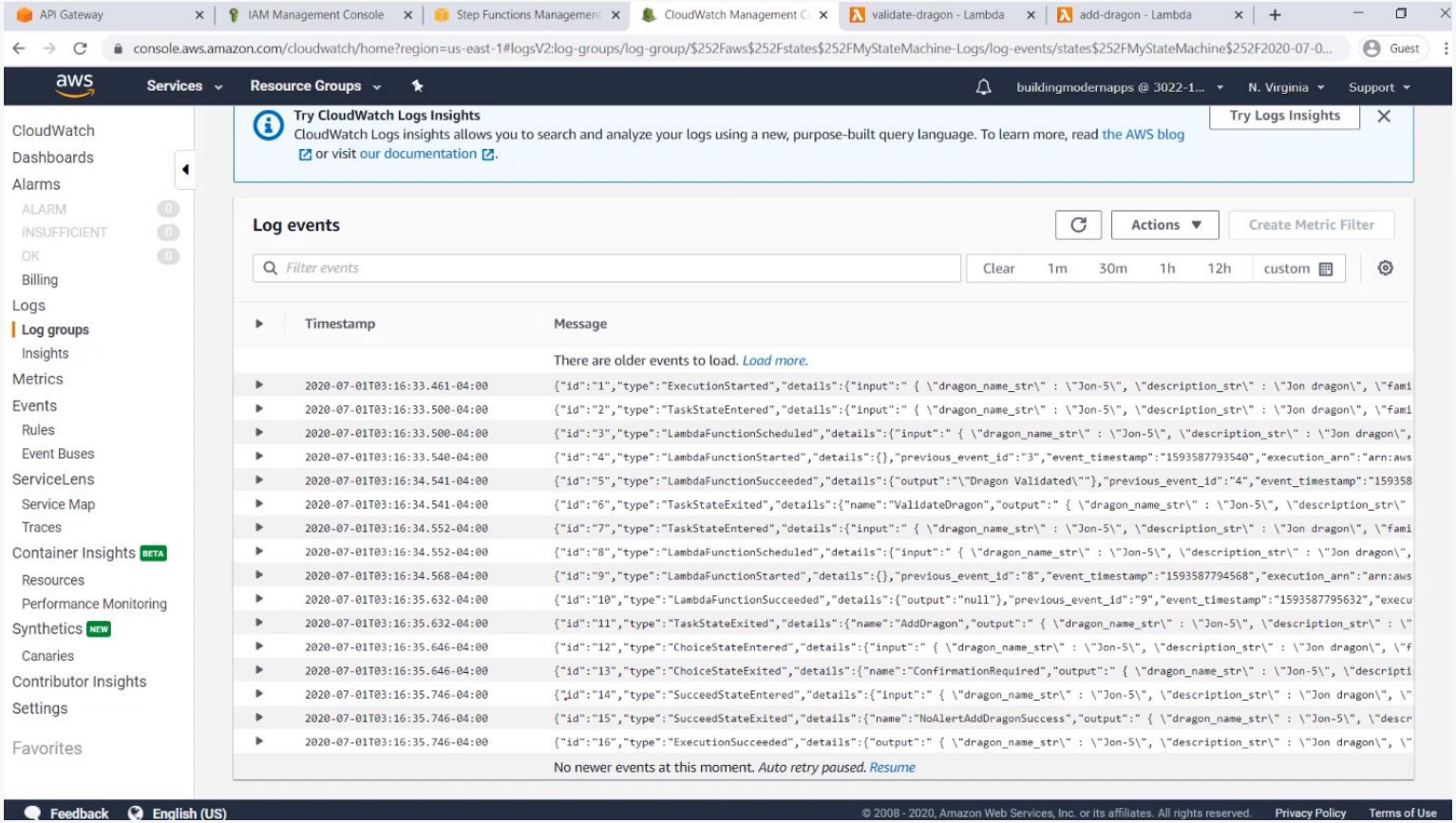
even each log’s details



now, we can look at the AWS Step Function logs



You can select the State Machine logs and Lambda Function logs in one place



Step Machine logs

Message

There are older events to load. *Load more.*


```
{
    "id": "1",
    "type": "ExecutionStarted",
    "details": {
        "input": " { \"dragon_name_str\" : \"Jon-5\", \"description_str\" : \"Jon dragon\", \"family_str\" : \"green\", \"location_
        "roleArn": "arn:aws:iam::302211264422:role/service-role/StepFunctions-MyStateMachine-role-feb8533b"
    },
    "previous_event_id": "0",
    "event_timestamp": "1593587793461",
    "execution_arn": "arn:aws:states:us-east-1:302211264422:execution:MyStateMachine:82b75e58-0583-4f1a-925c-50d99ca1ae78"
}
```

{"id":"2","type":"TaskStateEntered","details":{"input":" { \"dragon_name_str\" : \"Jon-5\", \"description_str\" : \"Jon dragon\",

{"id":"3","type":"LambdaFunctionScheduled","details":{"input":" { \"dragon_name_str\" : \"Jon-5\", \"description_str\" : \"Jon drag

{"id":"4","type":"LambdaFunctionStarted","details":{},"previous_event_id":"3","event_timestamp":"1593587793540","execution_arn":"a

{"id":"5","type":"LambdaFunctionSucceeded","details":{"output":"\"Dragon Validated\""},"previous_event_id":"4","event_timestamp":"

{"id":"6","type":"TaskStateExited","details":{"name":"ValidateDragon","output":" { \"dragon_name_str\" : \"Jon-5\", \"description_

{"id":"7","type":"TaskStateEntered","details":{"input":" { \"dragon_name_str\" : \"Jon-5\", \"description_str\" : \"Jon dragon\",

{"id":"8","type":"LambdaFunctionScheduled","details":{"input":" { \"dragon_name_str\" : \"Jon-5\", \"description_str\" : \"Jon drag

{"id":"9","type":"LambdaFunctionStarted","details":{},"previous_event_id":"8","event_timestamp":"1593587794568","execution_arn":"a

{"id":"10","type":"LambdaFunctionSucceeded","details":{"output":"null"},"previous_event_id":"9","event_timestamp":"1593587795632",

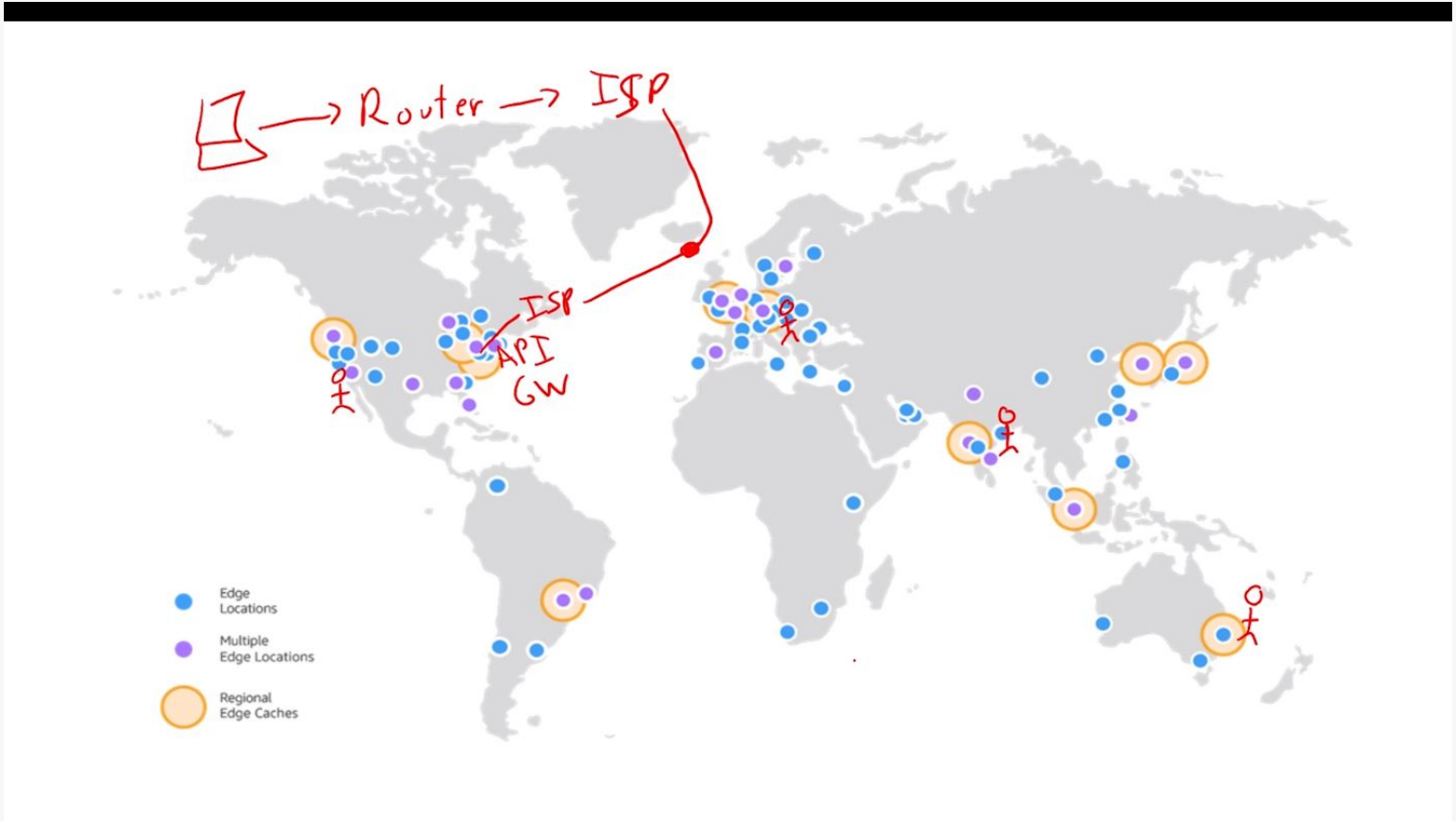
with dateils

Good Resources:

AWS Lambda function logging in Python

Setting up CloudWatch logging for a REST API in API Gateway

Edge-Optimized Endpoints



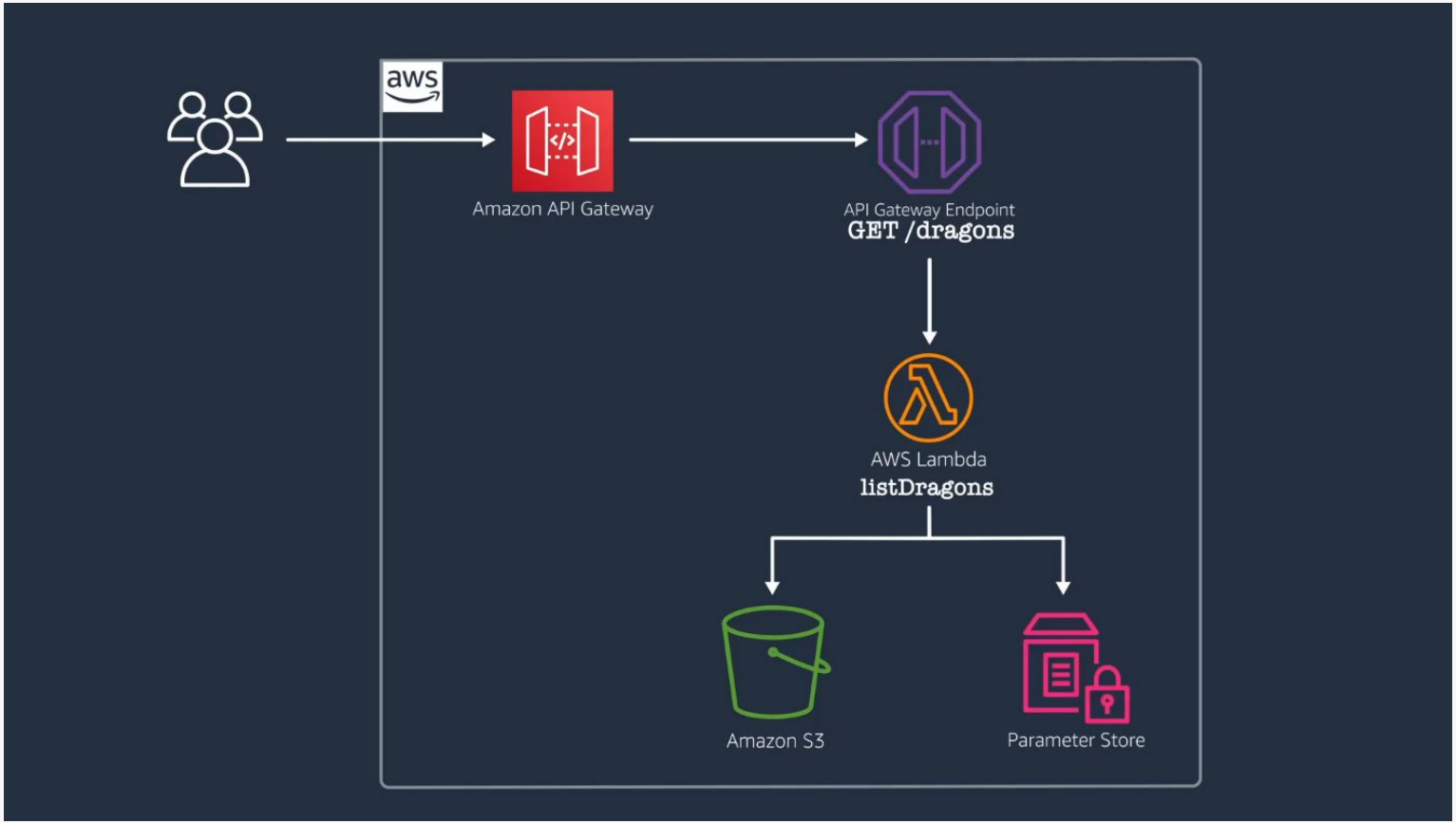
Amazon Edges

An edge optimized API endpoint is best for geographically distributed clients.

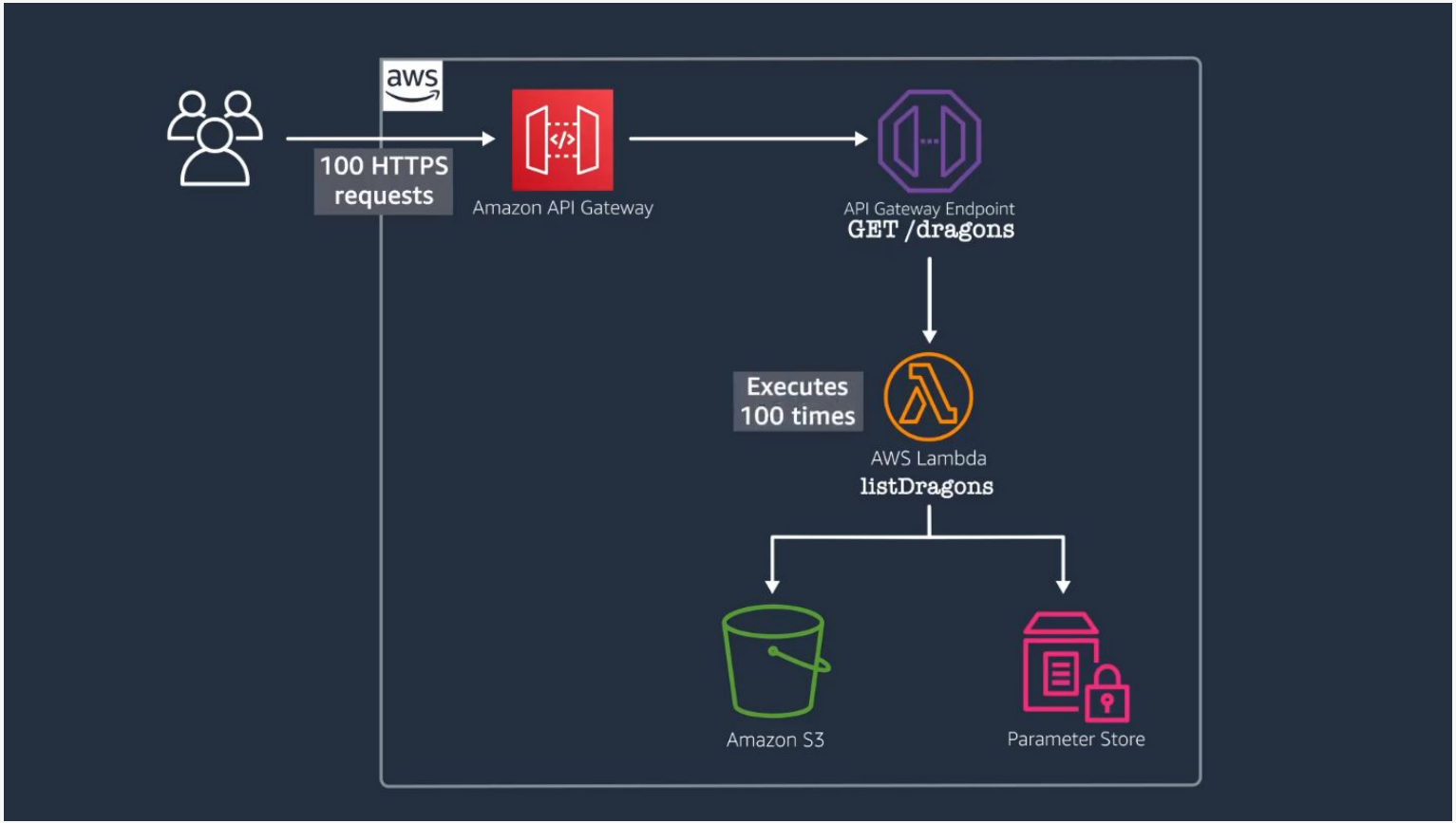
API requests are routed to the nearest CloudFront point of presence.

We can speed up the API's performance using
a feature of API Gateway called Response Caching.

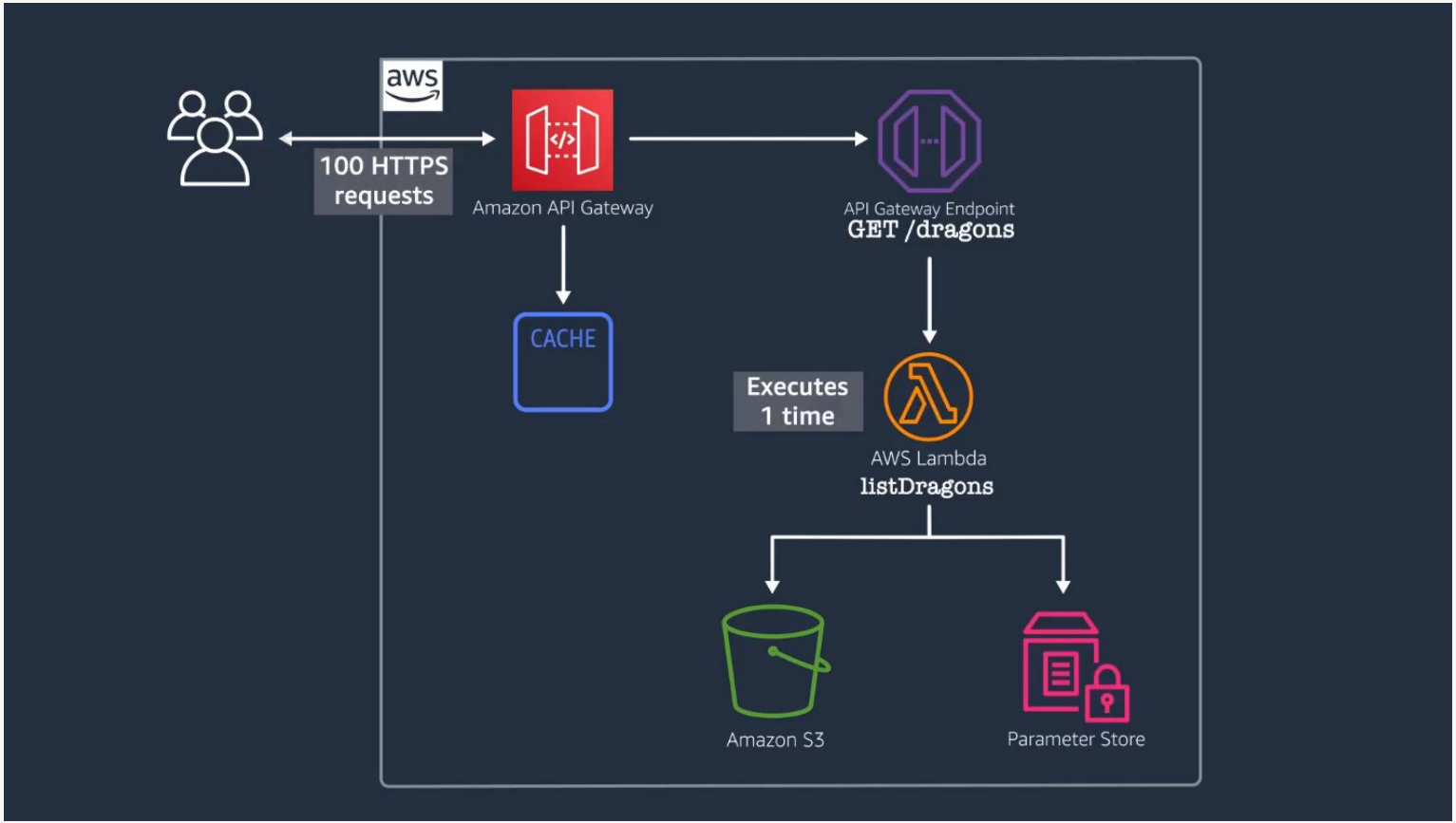
It cache the end points' responses and improves the latency.



By default, when an API is deployed to a stage, responses are not cacheed.



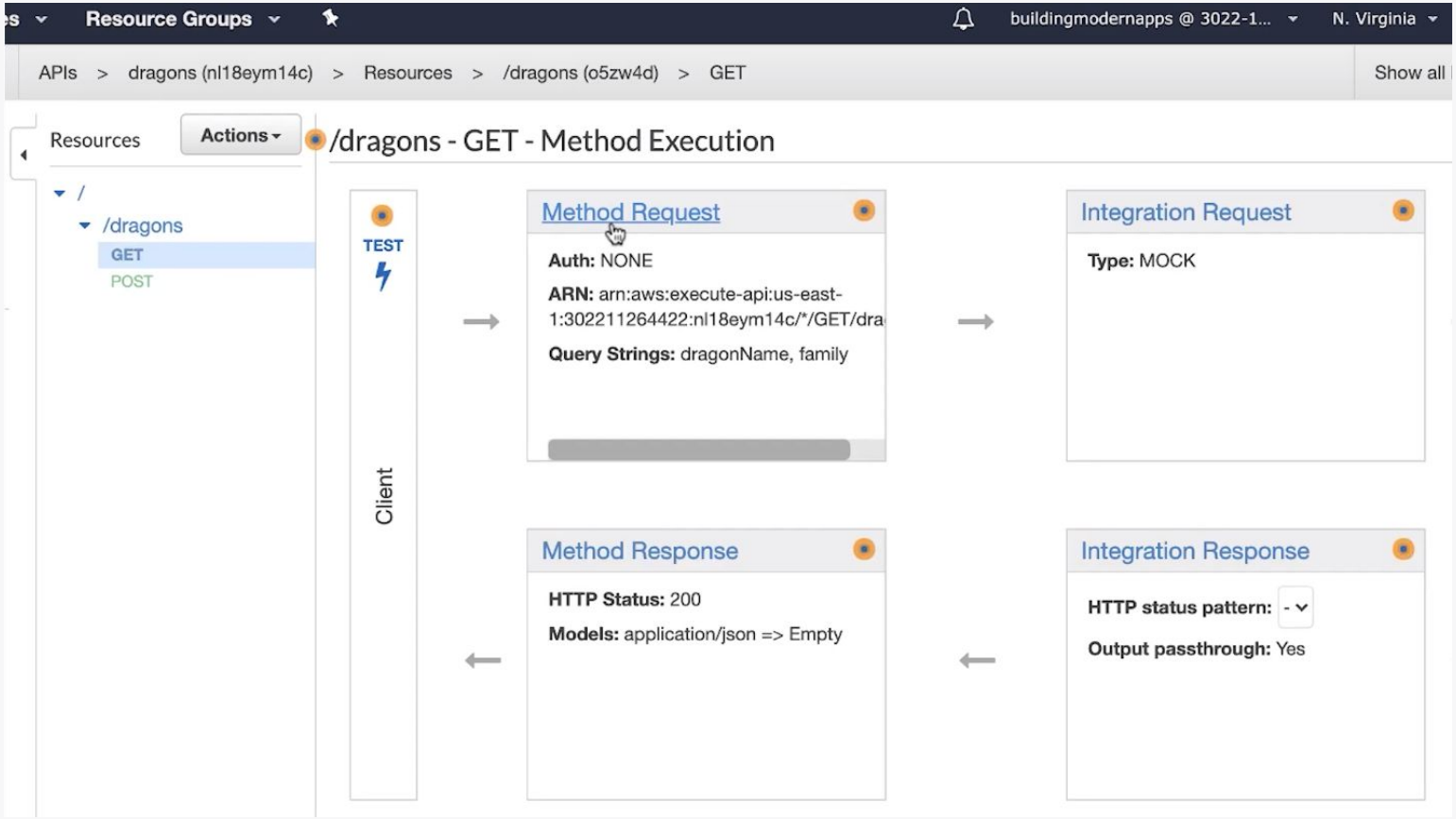
with 100 GET requests, the backend Lambda function will be executed 100 times (and 100 times for S3). We have to pay 100 times for calls.



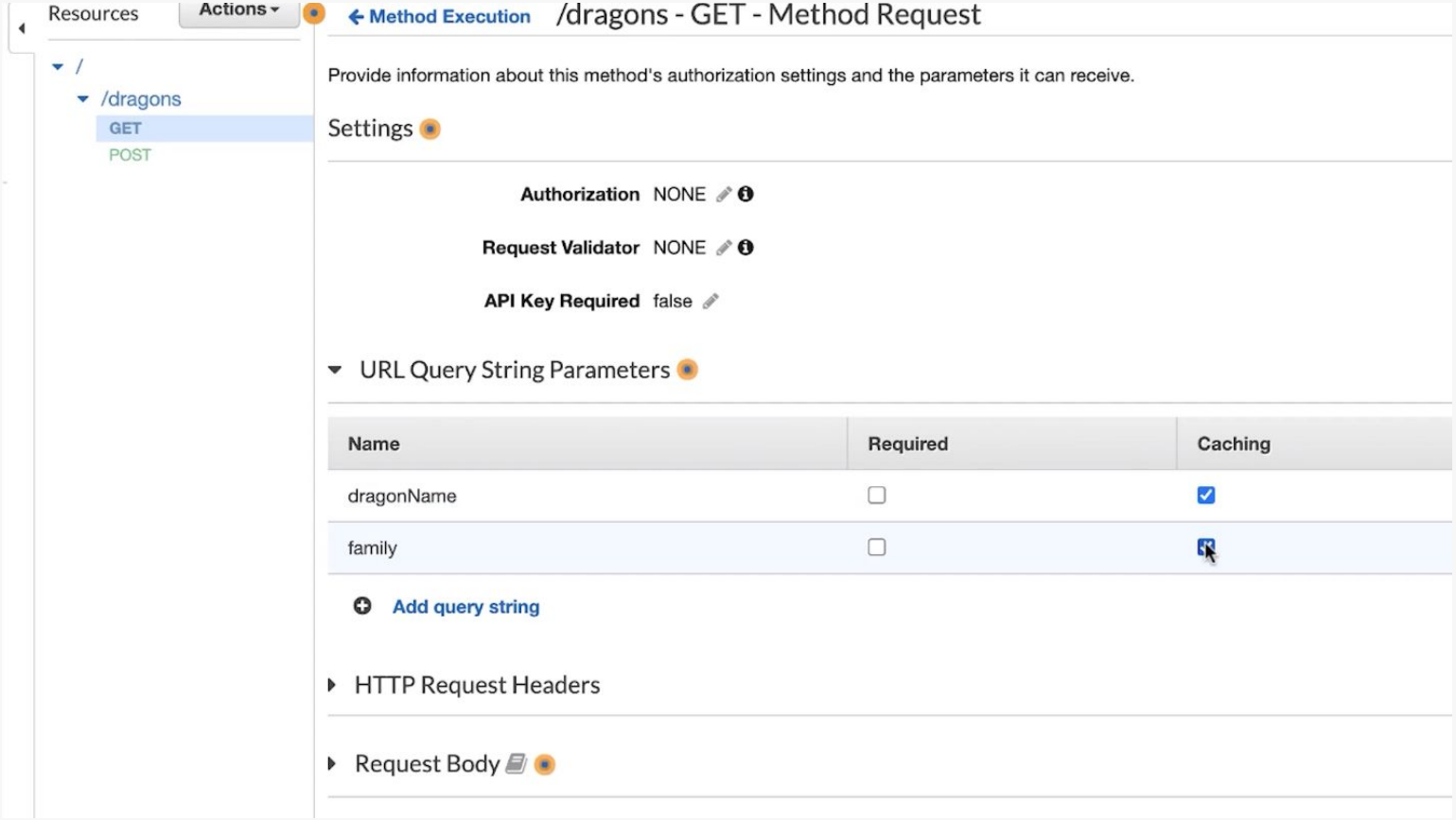
use response caching turned on for production and not for your development environment, because of cost implications.

When you enable caching, you must choose a cache capacity.

In general, a larger cache capacity gives a better performance
because you can cache more data,
but also it costs more.



For our dragon API setting up caching just on the GET method make sense.



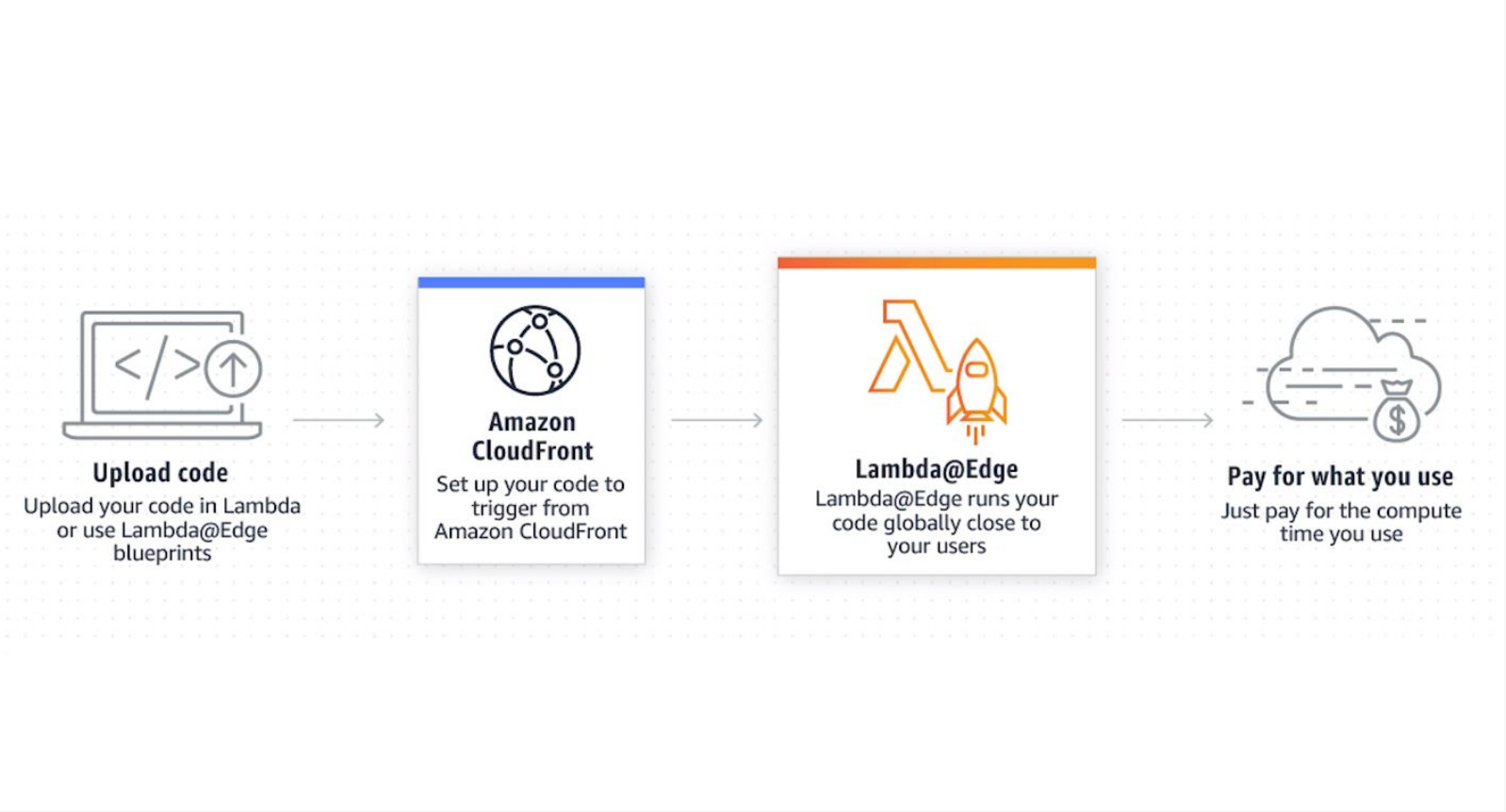
enable caching for specific query parameters

Turning on API Gateway response caching is a great way to optimize an API.

We can configure the time to live for caching in seconds per stage,
which allows to control how long objects are kept in the cache.

It is a great feature to take advantage of
in a high volume scenario where the data is relatively static.

Lambda@Edge is a feature of Amazon CloudFront
that lets to run code closer to the users,
and improve performance and reduce latency.



how to set up a lambda function to use the Lambda@Edge

Keep in mind that Lambda@Edge has service requirements and limits that are more restrictive and you should check the documentation.

Good Resources:

Low-Latency Content Delivery Network (CDN) - Amazon CloudFront

Enabling API caching to enhance responsiveness - Amazon API Gateway

Edge Computing| CDN, Global Serverless Code, Distribution | AWS Lambda@Edge

Using AWS Lambda with CloudFront Lambda@Edge

Exercise 6

https://aws-tc-largeobjects.s3.amazonaws.com/DEV-AWS-MO-BuildingRedux/exercise-6-optimizing.html